Defining and Using Classes

Version 2026.1
2026-04-20

InterSystems Corporation One Congress Street Boston MA 02114 www.intersystems.com

Defining and Using Classes
PDF generated on 2026-04-20
InterSystems IRIS Version 2026.1

For Support questions about any InterSystems products, contact:

InterSystems Worldwide Response Center (WRC)
Tel: +1-617-621-0700
Tel: +44 (0) 844 854 2917
Email: support@InterSystems.com

Table of Contents

List of Figures

List of Tables

1

Basic Ideas in Class Programming

If you are not familiar with class programming, this topic is intended to give you a sense of how this kind of programming works. If you are familiar with class programming, you might find it helpful just to skim the code examples, so that you see what class programming in InterSystems IRIS® data platform looks like.

The concepts discussed here are largely independent of language, although the examples use ObjectScript.

1.1 Objects and Properties

In class programming, a key concept is *objects*. An object is a container for a set of values that are stored together or passed together as a set. An object often corresponds to a real-life entity, such as a patient, a patient diagnosis, a transaction, and so on.

A class definition is often a template for objects of a given type. The class definition has properties to contain the values for those objects. For example, suppose that we have a class named MyApp.Clinical.PatDiagnosis; this class could have the properties Date, EnteredBy, PatientID, DiagnosedBy, Code, and others.

You use the template by creating *instances* of the class; these instances are objects. For example, suppose that a user enters a patient diagnosis into a user interface and saves that data. The underlying code would have the following logic:

1. Create a new patient diagnosis object from the patient diagnosis template.
2. Set values for the properties of the object, as needed. Some may be required, some may have default values, some may be calculated based on others, and some may be purely optional.
3. Save the object.

   This action stores the data.

The following shows an example that uses ObjectScript:

ObjectScript

```
//create the object
set diagnosis=##class(MyApp.Clinical.PatDiagnosis).%New()

//set a couple of properties by using special variables
set diagnosis.Date=$SYSTEM.SYS.TimeStamp()
set diagnosis.EnteredBy=$username

//set other properties based on variables set earlier by
//the user interface
set diagnosis.PatientID=patientid
set diagnosis.DiagnosedBy=clinicianid
set diagnosis.Code=diagcode

//save the data
//the next line tries to save the data and returns a status to indicate
//whether the action was successful
set status=diagnosis.%Save()
//always check the returned status
if $$$ISERR(status) {do $System.Status.DisplayError(status) quit status}
```

Note the following points:

- To refer to a property of the object, you use the syntax *object_variable*`.`*property_name*, for example: `diagnosis.DiagnosedBy`
- **%New()** and **%Save()** are methods of the MyApp.Clinical.PatDiagnosis class.

  The next section discusses types of methods and why you invoke them in different ways as seen here.

1.2 Methods

A method is a procedure (in most cases; also see Variable Availability and Scope). Methods can invoke each other and can refer to properties and parameters.

There are two kinds of methods in a class: instance methods and class methods. These have different purposes and are used in different ways.

1.2.1 Instance Methods

An *instance method* has meaning only when invoked from an instance of the class, usually because you are doing something to or something with that instance. For example:

ObjectScript

```
set status=diagnosis.%Save()
```

For example, suppose that we are defining a class that represents patients. In this class, we could define instance methods to perform the following actions:

- Calculate the BMI (body mass index) for the patient
- Print a report summarizing information for the patient
- Determine whether the patient is eligible for a specific procedure

Each of these actions requires knowledge of data stored for the patient, which is why most programmers would write them as instance methods. Internally, the implementation of an instance method typically refers to properties of that instance. The following shows an example definition of an instance method that refers to two properties:

Class Member

```
Method GetBMI() as %Numeric
{
 Set bmi=..WeightKg / (..HeightMeter**2)
 Quit bmi
}
```

To use this method, your application code might include lines like this:

ObjectScript

```
 //open the requested patient given an id selected earlier
 set patient=##class(MyApp.Clinical.PatDiagnosis).%OpenId(id)

 //get value to display in BMI Display field
 set BMIDisplay=patient.GetBMI()
```

1.2.2 Class Methods

The other type of method is the *class method* (called a *static method* in other languages). To invoke this type of method, you use syntax that does not refer to an instance. For example:

ObjectScript

```
 set patient=##class(MyApp.Clinical.PatDiagnosis).%New()
```

There are three very general reasons to write class methods:

- You need to perform an action that creates an instance of the class.

  By definition, this action cannot be an instance method.

- You need to perform an action that affects multiple instances.

  For example, you might need to reassign a group of patients to a different primary care physician.

- You need to perform an action that does not affect any instance.

  For example, you can write a method that returns the time of day, or a random number, or a string formatted in a particular way.

1.2.3 Methods and Variable Scope

A method typically sets values of variables. In nearly all cases, these variables are available only within this method. For example, consider the following class:

Class Definition

```
Class GORIENT.VariableScopeDemo
{

ClassMethod Add(arg1 As %Numeric, arg2 As %Numeric) As %Numeric
{
    Set ans=arg1+arg2
    Quit ans
}

ClassMethod Demo1()
{
   set x=..Add(1,2)
   write x
}

ClassMethod Demo2()
{
   set x=..Add(2,4)
```

```
   write x
}

}
```

The **Add()** method sets a variable named `ans` and then returns the value contained in that variable.

The method **Demo1()** invokes the method **Add()**, with the arguments 1 and 2, and then writes the answer. The method **Demo2()** is similar but uses different hardcoded arguments.

If the method **Demo1()** or **Demo2()** tried to refer to the variable `ans`, that variable would be undefined in that context and InterSystems IRIS would throw an error.

Similarly, **Add()** cannot refer to the variable `x`. Also the variable `x` within **Demo1()** is a different variable from the variable `x` within **Demo2()**.

These variables have limited scope because that is the default behavior of InterSystems IRIS classes (and is the usual behavior in other class languages).

Within class definitions, you pass values almost entirely by including them as arguments to methods. This is the convention in class programming. This convention simplifies the job of determining the scope of variables.

In contrast, when you write routines, it is necessary to understand the rules that control scoping; see Variable Availability and Scope.

1.3 Class Constants (Parameters)

Sometimes it is useful for a class to have easy access to a constant value. In InterSystems IRIS classes, such a value is a *class parameter*. Other languages use the term *class constant* instead. The following shows an example:

Class Member

```
Parameter MYPARAMETER = "ABC" ;
```

A class parameter acquires a value at compile time and cannot be changed later.

Your methods can refer to parameters; that is why you define parameters. For example:

ObjectScript

```
 set myval=..#MYPARAMETER * inputvalue
```

1.4 Class Definitions and Types

The following shows an example of a class definition, which we will use to discuss types in class definitions:

Class Definition

```
Class MyClass Extends %Library.Persistent
{
Parameter MYPARAMETER = "ABC" ;

Property DateOfBirth As %Library.Date;

Property Home As Sample.Address;

Method CurrentAge() As %Library.Integer
{
 //details
}

ClassMethod Addition(x As %Library.Integer, y As %Library.Integer) As %Library.Integer
{
 //details
}

}
```

This class definition defines one parameter (`MYPARAMETER`), two properties (`DateOfBirth` and `Home`), one instance method (`CurrentAge()`), and one class method (`Addition()`).

In class programming, you can specify *types* in the following key places:

- For the class itself. The element after `Extends` is a type.

  Each type is the name of class.

- For parameters. In this case and in the remaining cases, the element after `As` is a type.
- For properties. For the `Home` property, the type is a class that itself contains properties.

  In this case, the type has an *object value*. In the example here, this is an *object-valued property*.

  Object-valued properties can contain other object-valued properties.

- For the return value of a method.
- For the value of any arguments used by a method.

1.5 Inheritance

In most class-based languages, a major feature is *inheritance*: one class can inherit from other classes and thus acquire the parameters, properties, methods, and other elements of those other classes. Collectively, the parameters, properties, methods, and other elements are known as *class members*.

1.5.1 Terminology and Basics

When class A inherits from class B, we use the following terminology:

- Class A is a *subclass* of class B. Alternatively, class A *extends* class B.

  Sometimes it is said that class A is a *subtype* of class B.

- Class B is a *superclass* of class A.

  Sometimes it is said that class A is the *child class* and class B is the *parent class*. This terminology is common but can be misleading, because the words *parent* and *child* are used in quite a different sense when discussing SQL tables.

When a class inherits from other classes, it acquires the class members of those other classes, including members that the superclasses have themselves inherited. The subclass can override the inherited class members.

It is possible for multiple superclasses of one class to define methods with the same name, properties with the same name, and so on. Therefore it is necessary to have rules for deciding which superclass contributes the definition that is used in the subclass. (See Inheritance.)

In the InterSystems IRIS class library, superclasses usually have different purposes and have members with different names, and conflicts of member names are not common.

1.5.2 Example

The following shows an example:

```
/// Finds files in a FilePath directory and submits all that match a FileSpec wildcard to
/// an associated BusinessService for processing within InterSystems IRIS
Class EnsLib.File.InboundAdapter Extends (Ens.InboundAdapter, EnsLib.File.Common)
```

This example is presented solely to demonstrate how a class can combine logic from different superclasses. This EnsLib.File.InboundAdapter class inherits from two classes that do quite different things:

- Ens.InboundAdapter, which contains the basic logic for something known as an inbound adapter, a concept in InterSystems IRIS.
- EnsLib.File.Common, which contains logic for working with sets of files in a given directory.

In EnsLib.File.InboundAdapter, the methods use logic from both of these classes, as well as their superclasses.

1.5.3 Use of Inherited Class Members

When you see the definition of a class in an editing tool, you do not see the inherited members that it contains but your code can refer to them.

For example, suppose that class A has two properties, each of which has a default value, as follows:

Class Definition

```
Class Demo.A
{
Property Prop1 as %Library.String [InitialExpression = "ABC"];

Property Prop2 as %Library.String [InitialExpression = "DEF"];
}
```

Class B could look like this:

Class Definition

```
Class Demo.B Extends Demo.A
{
Method PrintIt()
{
 Write ..Prop1,!
 Write ..Prop2,!
}

}
```

As noted earlier, a subclass can override the inherited class members. For example, class C could also inherit from class A but could override the default value of one of its properties:

Class Definition

```
Class Demo.C Extends Demo.A
{
Property Prop2 as %Library.String [InitialExpression = "GHI"];

}
```

1.5.4 Use of Subclasses

If class B inherits from class A, you can use an instance of class B in any location where you can use an instance of class A.

For example, suppose that you have a utility method like the following:

Class Member

```
ClassMethod PersonReport(person as MyApp.Person) {
 //print a report that uses properties of the instance
}
```

You can use an instance of `MyApp.Person` as input to this method. You can also use an instance of any subclass of `MyApp.Person`. For example:

ObjectScript

```
 //id variable is set earlier in this program
 set employee=##class(MyApp.Employee).%OpenId(id)
 do ##class(Util.Utils).PersonReport(employee)
```

Similarly, the return value of a method (if it returns a value) can be an instance of a subclass of the specified type. For example, suppose that `MyApp.Employee` and `MyApp.Patient` are both subclasses of `MyApp.Person`. You could define a method as follows:

Class Member

```
ClassMethod ReturnRandomPerson() as MyApp.Person
{
 Set randomnumber = $RANDOM(10)
 If randomnumber > 5 {
     set person=##class(MyApp.Employee).%New()
 }
 else {
     set person=##class(MyApp.Patient).%New()
 }
 quit person
}
```

1.6 Classes as Containers of Methods

As noted earlier, a class definition is often a template for objects. Another possibility is for a class to be a container for a set of class methods that belong together. In this case, you never create an instance of this class. You only invoke class methods in it.

For examples, see the classes in the %SYSTEM package.

1.7 Abstract Classes

It is also useful to define abstract classes. An *abstract class* typically defines a generic interface and cannot be instantiated. A method definition within the class declares the signature of the method, but not its implementation.

You define an abstract class to describe an interface. Then you or other developers create subclasses, and in those subclasses, implement the methods. The implementation must match the signature specified in the abstract class. This system enables you to develop multiple, parallel classes with slightly different purposes but identical interfaces. Many of the system classes have common interfaces for this reason.

It is also possible to specify that a method is abstract even if the class is not.

1.8 See Also

- Defining Classes
- Creating Class Documentation

2

Defining Classes

This topic describes the basics of defining classes in InterSystems IRIS® data platform.

2.1 Introduction to Terminology

The following shows a simple InterSystems IRIS class definition, with some typical elements:

Class Definition

```
Class Demo.MyExample.MyClass Extends %RegisteredObject
{

Property Property1 As %String;

Property Property2 As %Numeric;

Method MyMethod() As %String
{
   set returnvalue=..Property1_..Property2
   quit returnvalue
}

}
```

Note the following points:

- The full class name (also called the *fully qualified class name*) is `Demo.MyExample.MyClass` and the *short class name* is `MyClass`.
- `Demo` and `MyExample` are both *package names*. The package `MyExample` *belongs to* or is *contained in* the package `Demo`. `MyExample` is a *short package name*, and `Demo.MyExample` is the *fully qualified name* of this package.
- This class *extends* the class %RegisteredObject. Equivalently, this class *inherits from* %RegisteredObject.

  %RegisteredObject is the *superclass* of this class, or this class is a *subclass* of %RegisteredObject. An InterSystems IRIS class can have multiple superclasses.

  The superclass(es) of a class determine how the class can be used.
- This class defines two properties: `Property1` and `Property2`. Property `Property1` is of type %String, and Property `Property2` is of type %Numeric.
- This class defines one method: `MyMethod()`, which returns a value of type %String.

This class refers to several system classes provided by InterSystems IRIS. These classes are %RegisteredObject (whose full name is %Library.RegisteredObject), %String (%Library.String), and %Numeric (%Library.Numeric). %RegisteredObject is a key class in InterSystems IRIS, because it defines the object interface. It provides the methods you use to create and work with object instances. %String and %Numeric are data type classes. As a consequence, the corresponding properties hold literal values (rather than other kinds of values).

2.2 Kinds of Classes

InterSystems IRIS provides a large set of class definitions that your classes can use in the following general ways:

- You can use classes as superclasses for your classes.
- You can use classes as values of properties, values of arguments to methods, values returned by methods, and so on.
- Some classes simply provide specific APIs. You typically do not use these classes in either of the preceding ways. Instead you write code that calls methods of the API.

The most common choices for superclasses are as follows:

- %RegisteredObject — This class represents the object interface in its most generic form.
- %Persistent — This class represents a persistent object. In addition to providing the object interface, this class provides methods for saving objects to the database and reading objects from the database.
- %SerialObject — This class represents an object that can be embedded in (serialized within) another object.
- Subclasses of any of the preceding classes.
- None — It is not necessary to specify a superclass when you create a class.

The most common choices for values of properties, values of arguments to methods, values returned by methods, and so on are as follows:

- Object classes (the classes contained in the previous list)
- Data type classes used for literal properties
- Collection classes
- Stream classes

2.2.1 Object Classes

The phrase *object class* refers to any subclass of %RegisteredObject. With an object class, you can create an instance of the class, specify properties of the instance, and invoke methods of the instance.

The generic term *object* refers to an instance of an object class.

There are three general categories of object classes:

- *Transient object classes* or *registered object classes* are subclasses of %RegisteredObject but not of %Persistent or %SerialObject (see the following bullets).

  For details, see Working with Registered Objects.

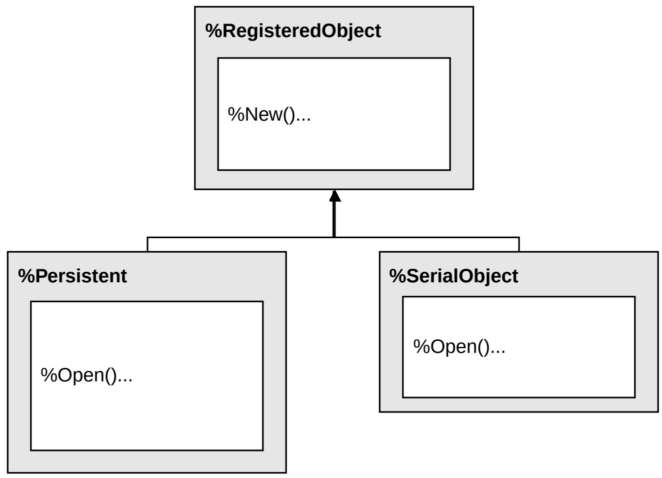
- *Persistent classes* are subclasses of %Persistent, which is a direct subclass of %RegisteredObject. The %Persistent class includes the behavior of %RegisteredObject and adds the ability to save objects to disk, reopen them, and so on.

  For details, see Introduction to Persistent Objects.

- *Serial classes* are subclasses of %SerialObject, which is a direct subclass of %RegisteredObject. The %SerialObject class includes the behavior of %RegisteredObject and adds the ability to create a string that represents the state of the object, for inclusion as a property within another object (usually either a transient object or a persistent object). The phrase *serializing an object* refers to the creation of this string.

  For details, see Defining and Using Object-Valued Properties.

The following figure shows the inheritance relationship among these three classes. The boxes list some of the methods defined in the classes:



Collection classes and stream classes are object classes with specialized behavior.

2.2.2 Data Type Classes

The phrase *data type class* refers to any class whose ClassType keyword equals `datatype` or any subclass of such a class. These classes are not object classes (a data type class cannot define properties, and you cannot create an instance of the class). The purpose of a data type class (more accurately a data type generator class) is to be used as the type of a property of an object class.

2.3 Kinds of Class Members

An InterSystems IRIS class definition can include the following items, all known as *class members*:

- Parameters — A parameter defines a constant value for use by this class. The value is set at compilation time, in most cases.
- Methods — InterSystems IRIS supports two types of methods: instance methods and class methods. An *instance method* is invoked from a specific instance of a class and performs some action related to that instance; this type of method is useful only in object classes. A *class method* is a method that can be invoked whether or not an instance of its class is in memory; this type of method is called a *static method* in other languages.
- Properties — A property contains data for an instance of the class. Properties are useful only in object classes. The following section provides more information.
- Class queries — A class query defines an SQL query that can be used by the class and specifies a class to use as a container for the query. Often (but not necessarily), you define class queries in a persistent class, to perform queries on the stored data for that class. You can, however, define class queries in any class.
- Other kinds of class members that are relevant only for persistent classes:

  - Storage definitions
  - Indices
  - Foreign keys
  - SQL triggers

- XData blocks — An XData block is a named unit of data defined within the class, typically for use by a method in the class. These have many possible applications.
- Projections — A class projection provides a way to extend the behavior of the class compiler.

  The projection mechanism is used by the Java projections; hence the origin of the term *projection*.

The language used to define the various class members (not including any ObjectScript, Python, SQL or other code used to implement the members) is sometimes referred to as the Class Definition Language (CDL).

2.4 Kinds of Properties

Formally, there are two kinds of properties: attributes and relationships.

Attributes hold values. Attribute properties are usually referred to simply as *properties*. Depending on the property definition, the value that it holds can be any of the following:

- A literal value such as `"MyString"` and 1. Properties that hold literal values are based on data type classes and are also called data type properties. See Defining and Using Literal Properties.
- A stream. A stream is an object that contains a value that would be too long for a string. See Working with Streams.
- A collection. InterSystems IRIS provides the ability to define a property as either a list or an array. The list or array items can be literal values or can be objects. See Working with Collections.
- Some other kind of object. See Defining and Using Object-Valued Properties.

Relationships hold associations between objects. Relationship properties are referred to as *relationships*. Relationships are supported only in persistent classes. See Defining and Using Relationships.

2.5 Defining a Class: The Basics

This section discusses basic class definitions in more detail. It discusses the following topics:

- Choosing a superclass
- Specifying class keywords
- Include files
- Introduction to defining class parameters
- Introduction to defining properties
- Introduction to defining methods

Typically, you use an Integrated Development Environment (IDE) to define classes. You can also define classes programmatically using the class definition classes or via an XML class definition file. If you define an SQL table using SQL DDL statements, the system creates a corresponding class definition.

2.5.1 Choosing a Superclass

When you define a class, one of your earliest design decisions is choosing the class (or classes) which to base your class. If there is only a single superclass, include `Extends` followed by the superclass name, at the start of the class definition.

Class Definition

```
Class Demo.MyClass Extends Superclass
{

//...

}
```

If there are multiple superclasses, specify them as a comma-separated list, enclosed in parentheses.

Class Definition

```
Class Demo.MyClass Extends (Superclass1, Superclass2, Superclass3)
{

//...

}
```

It is not necessary to specify a superclass when you create a class. It is common to use %RegisteredObject as the superclass even if the class does not represent any kind of object, because doing so gives your class access to many commonly used macros, but you can instead directly include the include files that contain them.

2.5.2 Include Files

You can create your own include files and include them in class definitions as needed.

To include an include file at the beginning of a class definition, use syntax of the following form. Note that you must omit the `.inc` extension of the include file:

```
Include MyMacros
```

For example:

```
Include %occInclude

Class Classname
{
}
```

To include multiple include files at the beginning of a class definition, use syntax of the following form:

```
Include (MyMacros, YourMacros)
```

Include files are inherited. That is, a subclass has access to all the same include files as its superclasses.

Note that this syntax does not have a leading pound sign (in contrast to the syntax required in a routine). Also, the `Include` directive is not case-sensitive, so you could use `INCLUDE` instead, for example. The include file name is case-sensitive.

See also #include.

2.5.3 Specifying Class Keywords

In some cases, it is necessary to control details of the code generated by the class compiler. For one example, for a persistent class, you can specify an SQL table name, if you do not want to (or cannot) use the default table name. For another example, you can mark a class as final, so that subclasses of it cannot be created. The class definitions support a specific set of keywords for such purposes. If you need to specify class keywords, include them within square brackets after the superclass, as follows:

```
Class Demo.MyClass Extends Demo.MySuperclass [ Keyword1, Keyword2, ...]
{

//...

}
```

For example, the available class keywords include Abstract and Final. For an introduction, see Compiler Keywords. InterSystems IRIS also provides specific keywords for each kind of class member.

2.5.4 Introduction to Defining Class Parameters

A class parameter defines a constant value for all objects of a given class. To add a class parameter to a class definition, add an element like one of the following to the class:

Class Member

```
Parameter PARAMNAME as Type;

Parameter PARAMNAME as Type = value;

Parameter PARAMNAME as Type [ Keywords ] = value;
```

Keywords represents any parameter keywords. For an introduction to keywords, see Compiler Keywords. For parameter keywords; see Parameter Keywords. These are optional.

2.5.5 Introduction to Defining Properties

An object class can include properties.

To add a property to a class definition, add an element like one of the following to the class:

Class Member

```
Property PropName as Classname;

Property PropName as Classname [ Keywords ] ;

Property PropName as Classname(PARAM1=value,PARAM2=value) [ Keywords ] ;

Property PropName as Classname(PARAM1=value,PARAM2=value) ;
```

PropName is the name of the property, and *Classname* is an optional class name (if you omit this, the property is assumed to be of type %String).

Keywords represents any property keywords. For an introduction to keywords, see Compiler Keywords. For property keywords; see Property Keywords. These are optional.

Depending on the class used by the property, you might also be able to specify *property parameters*, as shown in the third and fourth variations.

Notice that the property parameters, if included, are enclosed in parentheses and precede any property keywords. Also notice that the property keywords, if included, are enclosed in square brackets.

2.5.6 Introduction to Defining Methods

You can define two kinds of methods in InterSystems IRIS classes: class methods and instance methods.

To add a class method to a class definition, add an element like the following to the class:

```
ClassMethod MethodName(arguments) as Classname [ Keywords]
{
//method implementation
}
```

MethodName is the name of the method and *arguments* is a comma-separated list of arguments. *Classname* is an optional class name that represents the type of value (if any) returned by this method. Omit the `As` *Classname* part if the method does not return a value.

Keywords represents any method keywords. For an introduction to keywords, see Compiler Keywords. For method keywords, see Method Keywords. These are optional.

To add an instance method, use the same syntax with `Method` instead of `ClassMethod`:

```
Method MethodName(arguments) as Classname [ Keywords]
{
//method implementation
}
```

Instance methods are relevant only in object classes.

2.6 Naming Conventions

Class and class members follow naming conventions, described briefly here.

For complete information, see Rules and Guidelines for Identifiers and What Is Accessible in Your Namespaces.

2.6.1 General Rules

Every identifier must be unique within its context (for example, no two classes in a given namespace can have the same full name).

Identifiers preserve case: you must exactly match the case of a name; at the same time, two classes cannot have names that differ only in case. For example, the identifiers `id1` and `ID1` are considered identical for purposes of uniqueness.

2.6.2 Class Names

A full class name consists of two parts: a package name and a class name: the class name follows the final . character in the name. A class name must be unique within its package; a package name must be unique within an InterSystems IRIS namespace. A full class name (that is, starting with the package name) must start with either a letter or the `%` character. Note that any class whose package name starts with a `%` character is available in all namespaces.

Because persistent classes are automatically projected as SQL tables, a class definition must specify a table name that is *not* an SQL reserved word; if the name of a persistent class is an SQL reserved word, then the class definition must also specify a valid, non-reserved word value for its SQLTableName keyword.

For details on packages, see Packages.

2.6.3 Class Member Names

Every class member (such as a property or method) must have a name that is unique within its class. InterSystems strongly recommends that you do not give two members the same name, even if they are different types of members; there could be unexpected results.

Further, a member of a persistent class cannot use an SQL reserved word as its identifier. It can define an alias, however, using the SQLName or SQLFieldName keyword of that member (as appropriate).

Member names can be delimited, which allows them to include characters that are otherwise not permitted. To create a delimited member name, use double quotes for the first and last characters of the name. For example:

Class Member

```
Property "My Property" As %String;
```

2.7 Inheritance

As with other class-based languages, you can combine multiple class definitions via inheritance. An InterSystems IRIS class definition can *extend* (or *inherit from*) multiple other classes. Those classes, in turn, can extend other classes.

If one class inherits from another, the inheriting class is known as a *subclass* and the class or classes it is derived from are known as *superclasses*.

Note that InterSystems IRIS classes cannot inherit from classes defined in Python (meaning a class definition contained in a .py file) and vice versa.

The following shows an example class definition that uses two superclasses:

Class Definition

```
Class User.MySubclass Extends (%Library.Persistent, %Library.Populate)
{
}
```

In addition to a class inheriting methods from its superclasses, the properties inherit additional methods from system property behavior classes and, in the case of a data type attribute, from the data type class.

For example, if there is a class defined called Person:

Class Definition

```
Class MyApp.Person Extends %Library.Persistent
{
Property Name As %String;
Property DOB As %Date;
}
```

It is simple to derive a new class, Employee, from it:

Class Definition

```
Class MyApp.Employee Extends Person
{
Property Salary As %Integer;
Property Department As %String;
}
```

This definition establishes the Employee class as a subclass of the Person class. In addition to its own class parameters, properties, and methods, the Employee class includes all of these elements from the Person class.

2.7.1 Use of Subclasses

You can use a subclass in any place in which you might use its superclass. For example, using the above defined Employee and Person classes, it is possible to open an Employee object and refer to it as a Person:

ObjectScript

```
Set x = ##class(MyApp.Person).%OpenId(id)
Write x.Name
```

We can also access Employee-specific attributes or methods:

ObjectScript

```
Write x.Salary // displays the Salary property (only available in Employee instances)
```

2.7.2 Primary Superclass

The leftmost superclass that a subclass extends is known as its *primary superclass*. A class inherits all the members of its primary superclass, including applicable class keywords, properties, methods, queries, indexes, class parameters, and the parameters and keywords of the inherited properties and inherited methods. Except for items marked as *Final*, the subclass can override (but not delete) the characteristics of its inherited members.

Important: Indexes (an option in persistent classes) are inherited *only* from the primary superclass.

2.7.3 Multiple Inheritance

A class can inherit its behavior and class type from more than one superclass. To establish multiple inheritance, list multiple superclasses within parentheses. The leftmost superclass is the primary superclass and specifies all the class keyword values.

For example, if class X inherits from classes A, B, and C, its definition includes:

Class Definition

```
Class X Extends (A, B, C)
{
}
```

The default inheritance order for the class compiler is from left to right, which means that differences in member definitions among superclasses are resolved in favor of the leftmost superclass (in this case, A superseding B and C, and B superseding C.)

Specifically, for class X, the values of the class parameter values, properties, and methods are inherited from class A (the first superclass listed), then from class B, and, finally, from class C. X also inherits any class members from B that A has not defined, and any class members from C that neither A nor B has defined. If class B has a class member with the same name as a member already inherited from A, then X uses the value from A; similarly, if C has a member with the same name as one inherited from either A or B, the order of precedence is A, then B, then C.

InterSystems recommends using left inheritance in all new programming, but you can specify the Inheritance keyword to override this. For reasons of history, most InterSystems IRIS classes contain `Inheritance = right`.

2.7.4 Additional Topics

Also see %ClassName() and the Most Specific Type Class (MSTC).

2.8 Introduction to Compiler Keywords

As shown in Defining a Class: The Basics, you can include keywords in a class definition or in the definition of a class member. These keywords, also known as *class attributes*, generally affect the compiler. This section introduces some common keywords and discusses how InterSystems IRIS presents them.

2.8.1 Example

The following example shows a class definition with some commonly used keywords:

```
/// This sample persistent class represents a person.
Class MyApp.Person Extends %Persistent [ SqlTableName = MyAppPerson ]
{

/// Define a unique index for the SSN property.
Index SSNKey On SSN [ Unique ];

/// Name of the person.
Property Name As %String [ Required ];

/// Person's Social Security number.
Property SSN As %String(PATTERN = "3N1""-""2N1""-""4N") [ Required ];

}
```

This example shows the following keywords:

- For the class definition, the Extends keyword specifies the superclass (or superclasses) from which this class inherits.

  Note that the Extends keyword has a different name when you view the class in other ways; see Class Documentation.

- For the class definition, the SqlTableName keyword determines the name of the associated table, if the default name is not to be used. This keyword is meaningful only for persistent classes.
- For the index definition, the Unique keyword causes InterSystems IRIS to enforce uniqueness on the property on which the index is based (`SSN` in this example).
- For the two properties, the Required keyword causes InterSystems IRIS to require non-null values for the properties.

PATTERN is not a keyword but instead is a property parameter; notice that *PATTERN* is enclosed in parentheses, rather than square brackets.

Apart from keywords related to storage (which are not generally documented), you can find details on the keywords in the Class Definition Reference. The reference information demonstrates the syntax that applies when you view a class in the usual edit mode.

2.9 See Also

- Macros and Include Files
- Compiling and Deploying Classes

- Creating Class Documentation
- Top Level Class Syntax and Keywords
- Working with Registered Objects
- Introduction to Persistent Objects

3

Compiling and Deploying Classes

You can compile classes using an IDE (as documented elsewhere), and you can compile them programmatically as described in this topic. This topic also provides details on the compiler behavior. Finally, this topic describes how to put compiled classes into deployed mode.

3.1 Compiling Programmatically

You can compile classes programmatically by using the **Compile()** method of the %SYSTEM.OBJ class:

ObjectScript

```
Do $System.OBJ.Compile("MyApp.MyClass")
```

The %SYSTEM.OBJ class provides additional methods you can use for compiling multiple classes.

3.2 Class Compiler Basics

InterSystems IRIS® data platform class definitions must be compiled before they can be used.

The class compiler differs from the compilers available with other programming languages, such Java, in two significant ways: first, the results of compilation are placed into a shared repository (database), not a file system. Second, it automatically provides support for persistent classes.

Specifically, the class compiler does the following:

1. It generates a list of *dependencies* — classes that must be compiled first. Depending on the compile options used, any dependencies that have been modified since last being compiled will also be compiled.
2. It resolves *inheritance* — it determines which methods, properties, and other class members are inherited from superclasses. It stores this inheritance information into the class dictionary for later reference.
3. For persistent and serial classes, it determines the storage structure needed to store objects in the database and creates the necessary runtime information needed for the SQL representation of the class.
4. It executes any method generators defined (or inherited) by the class.
5. It creates one or more routines that contain the runtime code for the class. The class compiler groups methods by implementation language and generates separate routines, each containing methods of one implementation language.

6. It compiles all of the generated routines into executable code.
7. It creates a class descriptor. This is a special data structure (stored as a routine) that contains all the runtime dispatch information needed to support a class (names of properties, locations of methods, and so on).

3.3 Class Compiler Notes

3.3.1 Compilation Order

When you compile a class, the system also recompiles other classes if the class that you are compiling contains information about dependencies. On some occasions, you may need to control the order in which the classes are compiled. To do so, use the System, DependsOn, and CompileAfter keywords.

To find the classes that the compiler will recompile when you compile a given class, use the **$SYSTEM.OBJ.GetDependencies()** method. For example:

```
TESTNAMESPACE>d $system.OBJ.GetDependencies("Sample.Address",.included)

TESTNAMESPACE>zw included
included("Sample.Address")=""
included("Sample.Customer")=""
included("Sample.Employee")=""
included("Sample.Person")=""
included("Sample.Vendor")=""
```

The signature of this method is as follows:

```
classmethod GetDependencies(ByRef class As %String,
                            Output included As %String,
                            qspec As %String) as %Status
```

Where:

- *class* is either a single class name (as in the example), a comma-separated list of class names, or a multidimensional array of class names. (If it is a multidimensional array, be sure to pass this argument by reference.) It can also include wildcards.
- *included* is a multidimensional array of the names of the classes that will be compiled when *class* is compiled.
- *qspec* is a string of compiler flags and qualifiers. If you omit this, the method uses the current compiler flags and qualifiers.

3.3.2 Compiling Classes That Include Bitmap Indexes

When compiling a class that contains a bitmap index, the class compiler generates a bitmap extent index if no bitmap extent index is defined for that class. Special care is required when adding a bitmap index to a class on a production system. For more information, see Generating a Bitmap Extent Index.

3.3.3 Compiling When There Are Existing Instances of a Class in Memory

If the compiler is called while an instance of the class being compiled is open, there is no error. The already open instance continues to use its existing code. If another instance is opened after compilation, it uses the newly compiled code.

3.3.4 Compilation and <PROTECT> Errors

The process of compiling classes updates the *extent index*, which is an internal global structure that serves as the namespace-level registry. By default, this global is stored within the default globals database for the namespace. If that database is read-only, compiling classes will result in a <PROTECT> error. In such unusual cases, you can map the global location of the extent index (specifically the *^rINDEXEXT* global) to a writable database so that it is possible to compile classes; see Global Mappings.

3.4 Putting Classes in Deployed Mode

You might want to put some of your classes in *deployed mode* before you send them to customers; this process removes the source code so that it cannot be examined or modified.

For any class definitions that contain method definitions that you do not want customers to see, compile the classes and then use **$SYSTEM.OBJ.MakeClassDeployed()**. For example:

ObjectScript

```
do $system.OBJ.MakeClassDeployed("MyApp.MyClass",,1)
```

If the third argument (*fullDeploy*) is 1, the method removes the majority of the class definition, leaving the minimum so the class is runnable. For persistent, serial, or XML-enabled classes, because the runtime requires more of the class definition to be present, the method instead just removes the method implementations. Do not use *fullDeploy* = 1 with a class that contains a class query; if you do so, the queries become unusable. See %SYSTEM.OBJ for details.

For an alternative approach, see Adding Compiled Code to Customer Databases.

3.4.1 About Deployed Mode

When a class is in deployed mode, its method and trigger definitions have been removed. (Note that if the class is a data type class, its method definitions are retained because they may be needed at runtime by cached queries.)

You cannot export or compile a class that is in deployed mode, but you can compile its subclasses (if they are not in deployed mode).

There is no way to reverse or undo putting a class into this mode. You can, however, replace the class by importing the definition from a file, if you previously saved it to disk. (This is useful if you accidentally put one of your classes into deployed mode prematurely.)

3.5 See Also

- Defining Classes
- Creating Class Documentation
- Top Level Class Syntax and Keywords
- System Flags and Qualifiers

4

Creating Class Documentation

Your InterSystems IRIS® data platform server provides a web page called the *InterSystems Class Reference*, which displays automatically generated reference information for the classes provided by InterSystems, as well as for classes you have created. Informally, the Class Reference is known as *Documatic*, because it is generated by the class %CSP.Documatic.

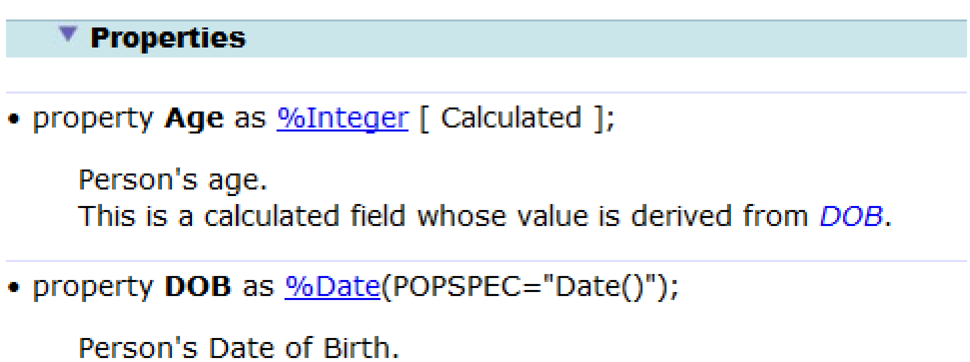
The URL has the following form, using the <baseURL> for your instance:

```
https:<baseURL>/csp/documatic/%25CSP.Documatic.cls
```

This web page has a different look and feel from the online Class Reference (because the latter is also connected to the rest of the online documentation).

4.1 Introduction to the Class Reference

The purpose of the Class Reference is to advertise, to other programmers, which parts of a class can be used, and how to use them. The following shows an example:



This reference information shows the definitions of class members, but not their actual implementations. For example, it shows method signatures but not their internal definitions. It includes links between elements so that you can rapidly follow the logic of the code. There is also a search option.

4.2 Creating Documentation to Include in the Class Reference

To create documentation to include in the Class Reference, create comments within the class definitions — specifically comments that start with `///`. If you precede the class declaration with such comments, the comments are shown at the top of the page for the class. If you precede a given class member with such comments, the comments are shown after the generated information for that class member. Once you compile the class, you can view its generated class documentation the next time you open the *Class Reference* documentation. If you add no Class Reference comments, items that you add to a class or package appear appropriately in the lists of class or package contents, but without any explanatory text.

You can extend any existing Class Reference comments by modifying the class definition. The syntax rules for Class Reference comments are strict:

- All Class Reference comments that describe a class or class member must appear in a consecutive block immediately before the declaration of the item that they describe.
- Each line in the block of comments must start with three slashes: `///`

  Tip: Note that, by default, the presentation combines the text of all the `///` lines and treats the result as single paragraph. You can insert HTML line breaks (
). Or you can use HTML formatting (such as <p> and </p>), as discussed in the subsection.

- The three slashes must begin at the first (left-most) position in the line.
- No blank lines are allowed within Class Reference comments.
- No blank lines are allowed between the last line of the Class Reference comments and the declaration for the item that they describe.
- The length of the Class Reference comment (all lines combined) must be less than the string length limit (which is extremely long).

Class Reference comments allow plain text, plus any standard HTML element and a small number of specialized elements.

4.3 Using HTML Markup in Class Documentation

You can use HTML tags within the comments in a class. In your markup, adhere to as strict an HTML standard as you can, for example XHTML, so that any browser can display the result. Note that the %CSP.Documatic class generates a complete HTML page, and the class documentation is contained *within* the `<body>` element on that page. Therefore, do not include the HTML tags `<html>`, `<body>`, `<head>`, or `<meta>` in your markup; these tags will all be ignored. Also, the class name is displayed as a `<h1>` heading, so if you use headings, use `<h2>` and lower headings. Because search engines favor HTML pages that have exactly one `<h1>`, if you include `<h1>` within your markup, that heading is converted to `<h2>`.

In addition to standard HTML, you can use the following tags: `<CLASS>`, `<METHOD>`, `<PROPERTY>`, `<PARAMETER>`, `<QUERY>`, and `<EXAMPLE>`. (As with standard HTML tags, the names of these tags are not case-sensitive.) The most commonly used tags are described here. See the documentation for %CSP.Documatic for details of the others.

<CLASS>

Use to tag class names. If the class exists, the contents are displayed as a link to the class' documentation. For example:

```
/// This uses the <CLASS>MyApp.MyClass</CLASS> class.
```

<EXAMPLE>

Use to tag programming examples. This tag affects the appearance of the text. Note that each `///` line becomes a separate line in the example (in contrast to the usual case, where the lines are combined into a single paragraph). For example:

```
/// <EXAMPLE>
/// set o=..%New()
/// set o.MyProperty=42
/// set o.OtherProp="abc"
/// do o.WriteSummary()
/// </EXAMPLE>
```

<METHOD>

Use to tag method names. If the method exists, the contents are displayed as a link to the method's documentation. The method cannot be internal and is assumed to be available in the same class. For example:

```
/// This is identical to the <METHOD>Unique</METHOD> method.
```

<PROPERTY>,

Use to tag property names. If the property exists, the contents are displayed as a link to the property's documentation. The property cannot be internal and is assumed to be available in the same class. For example:

```
/// This uses the value of the <PROPERTY>State</PROPERTY> property.
```

Here is a multi-line description using HTML markup:

```
/// The <METHOD>Factorial</METHOD> method returns the factorial
/// of the value specified by <VAR>x</VAR>.
```

4.4 See Also

- Defining Classes

5
Package Options

This topic discusses packages in more detail.

For persistent classes, the package is represented in SQL as an SQL schema. For details, see Projection of Packages to Schemas.

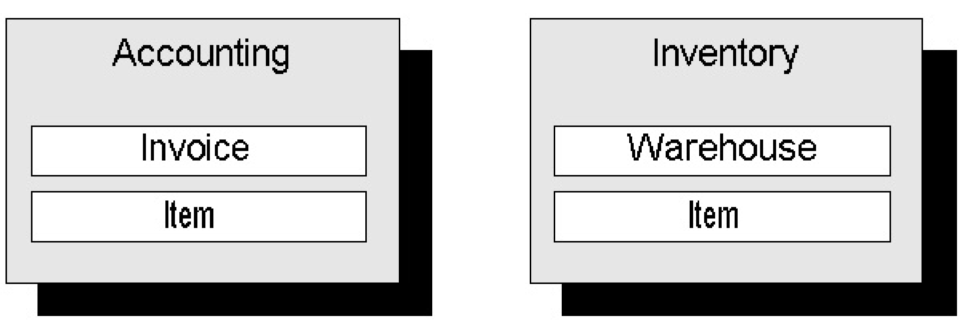
Important: When InterSystems IRIS® data platform encounters a reference to a class that does not include a package name and where the class name starts with "%", InterSystems IRIS assumes the class is in the %Library package.

5.1 Overview of Packages

InterSystems IRIS supports *packages*, which group related classes within a specific database. Packages provide the following benefits:

- They give developers an easier way to build larger applications and to share code with one another.
- They make it easier to avoid name conflicts between classes.
- They give a logical way to represent SQL schemas within the object dictionary in a clean, simple way: A package corresponds to a schema.

A package is simply a way to group related classes under a common name. For example, an application could have an Accounting system and an Inventory system. The classes that make up these applications could be organized into an Accounting package and an Inventory package:



Any of these classes can be referred to using their full name (which consists of package and class name):

ObjectScript

```
Do ##class(Accounting.Invoice).Method()
Do ##class(Inventory.Item).Method()
```

If the package name can be determined from context (see below), then the package name can be omitted:

ObjectScript

```
Do ##class(Invoice).Method()
```

As with classes, a package definition exists within an InterSystems IRIS database. For information on mapping a package from a database to a namespace, see Package Mapping.

5.2 Package Names

A package name is a string. It may contain . (period) characters, but no other punctuation. Each period-delimited piece of the package name is a subpackage, and there can be multiple subpackages. If you give a class the name Test.Subtest.TestClass, then this indicates that the name of the package is Test, the name of the subpackage is Subtest, and the name of the class is TestClass. The fully qualified class name is Test.Subtest.TestClass and this class is within the package whose fully qualified package name is Test.Subtest.

There are several limitations on the length and usage of package names:

- A package name is subject to a length limit. See Rules and Guidelines for Identifiers.
- Within a namespace, each package name must be unique without regards to case. Hence, there cannot be both "ABC" and "abc" packages in a namespace, and the "abc.def" package and subpackage are treated as part of the "ABC" package. (Similarly, two classes in the same package cannot have names differing only by case.)

For complete information, see Rules and Guidelines for Identifiers and What Is Accessible in Your Namespaces.

5.3 Defining Packages

Packages are implied by the name of the classes. When you create a class, the package is automatically defined. Similarly, when the last class in a package is deleted, the package is also automatically deleted.

The following shows an example in which the package name is `Accounting`, the class name is `Invoice`, and the fully qualified class name is `Accounting.Invoice`:

Class Definition

```
Class Accounting.Invoice
{
}
```

5.4 Package Mapping

By definition, each package is part of a particular database. Frequently, each database is associated with a namespace, where the database and the namespace share a common name. To make a package definition in a database available to a namespace not associated with that database, use *package mapping*. This procedure is described in Adding Mappings to a Namespace.

Mapping a package across namespace maps the package definition, not its data.

5.5 Package Use When Referring to Classes

There are two ways to refer to classes:

- Use the fully qualified name (that is, *Package.Class*). For example:

  ObjectScript

  ```
  // create an instance of Lab.Patient
  Set patient = ##class(Lab.Patient).%New()
  ```

- Use the short class name and let the class compiler resolve which package it belongs to.

  By default, when you use a short class name, InterSystems IRIS assumes that the class is either in the package of the class whose code you are using (if any), or in the `%Library` package, or in the `User` package.

  If you want the compiler to search for classes in other packages, import those packages.

  Note: It is an error to use a short class that is ambiguous; that is, if you have the same short class name in two or more packages and import all of them, you will get an error when the compiler attempts to resolve the package name. To avoid this error, use full names.

5.6 Importing Packages

When you import packages, InterSystems IRIS looks for any short class names in those packages. In a class definition, you can import a package via the class Import directive or the ObjectScript #IMPORT directive. This section explains these directives, discusses the effect on the User package and the effect on subclasses, and presents some tips.

5.6.1 Class Import Directive

You can include the class Import directive at the top of a class definition, before the `Class` line. The syntax for this directive is as follows:

```
Import packages
```

```
Class name {}
```

Where *packages* is either a single package or a comma-separated list of packages, enclosed in parentheses. The word `Import` is not case-sensitive, but is usually capitalized as shown here.

Remember that in a class context, the current package is always implicitly imported.

5.6.2 ObjectScript #IMPORT Directive

In ObjectScript method, an #IMPORT directive imports a package so that you can use short class names to refer to classes in it. The syntax for this directive is as follows:

```
#import packagename
```

Where *packagename* is the name of the package. The word `#import` is not case-sensitive. For example:

ObjectScript

```
#import Lab
 // Next line will use %New method of Lab.Patient, if that exists
 Set patient = ##class(Patient).%New()
```

You can have multiple #IMPORT directives:

ObjectScript

```
#import Lab
#import Accounting

 // Look for "Patient" within Lab & Accounting packages.
 Set pat = ##class(Patient).%New()

 // Look for "Invoice" within Lab & Accounting packages.
 Set inv = ##class(Invoice).%New()
```

The order of #IMPORT directives has no effect on how the compiler resolves short class names.

5.6.3 Explicit Package Import Affects Access to User Package

Once your code imports any packages explicitly, the User package is *not* automatically imported. If you need that package, you must import it explicitly as well. For example:

ObjectScript

```
#import MyPackage
#import User
```

The reason for this logic is because there are cases where you may *not* want the User package to be imported.

5.6.4 Package Import and Inheritance

A class inherits any explicitly imported packages from the superclasses.

The name of a class is resolved in the context where it was first used and not with the current class name. For example, suppose you define in User.MyClass a method **MyMethod()** and then you create a MyPackage.MyClass class that inherits from User.MyClass and compile this. InterSystems IRIS compiles the inherited **MyMethod()** method in MyPackage.MyClass — but resolves any class names in this method in the context of User.MyClass (because this is where this method was defined).

5.6.5 Tips for Importing Packages

By importing packages, you can make more adaptable code. For example, you can create code such as:

ObjectScript

```
#import Customer1
 Do ##class(Application).Run()
```

Now change App.MAC to:

ObjectScript

```
#import Customer2
 Do ##class(Application).Run()
```

When you recompile App.MAC, you will be using the Customer2.Application class. Such code requires planning: you have to consider code compatibility as well as the effects on your storage structures.

5.7 See Also

- Projection of Packages to Schemas
- Adding Mappings to a Namespace
- Defining Classes

6

Defining and Referring to Class Parameters

This topic describes how to define class parameters and how to refer to them programmatically. Also see Common Property Parameters.

6.1 Introduction to Class Parameters

A class parameter defines a special constant value available to all objects of a given class. When you create a class definition (or at any point before compilation), you can set the values for its class parameters. By default, the value of each parameter is the null string, but you can specify a non-null value as part of the parameter definition. At compile-time, the value of the parameter is established for all instances of a class. With rare exceptions, this value cannot be altered at runtime.

Class parameters are typically used for the following purposes:

- To customize the behavior of the various data type classes (such as providing validation information).
- To define user-specific information about a class definition. A class parameter is simply an arbitrary name-value pair; you can use it to store any information you like about a class.
- To define a class-specific constant value.
- To provide parameterized values for method generator methods to use. A method generator is a special type of method whose implementation is actually a short program that is run at compile-time in order to generate the actual runtime code for the method. Many method generators use class parameters.

6.2 Defining Class Parameters

To add a class parameter to a class definition, add an element like one of the following to the class:

```
Parameter PARAMNAME;

Parameter PARAMNAME as Type;

Parameter PARAMNAME as Type = value;

Parameter PARAMNAME as Type [ Keywords ] = value;
```

Where

- *PARAMNAME* is the name of the parameter. Note that by convention, parameters in InterSystems IRIS® data platform system classes are nearly all in uppercase; this convention provides an easy way to distinguish parameters from other class members, merely by name. There is no requirement for you to do the same.
- *Type* is a parameter type. See the next section.
- *value* is the value of the parameter. In most cases, this is a literal value such as 100 or `"MyValue"`. For some types, this can be an ObjectScript expression. See the next section.
- *Keywords* represents any parameter keywords. These are optional. For an introduction to keywords, see Compiler Keywords. For parameter keywords; see Parameter Keywords.

6.3 Parameter Types and Values

It is not necessary to specify a parameter type. If you do, note that this information is primarily meant for use by the development environment.

The parameter types include BOOLEAN, STRING, and INTEGER. Note that these are not InterSystems IRIS class names. See Common Property Parameters.

Except for the types COSEXPRESSION and CONFIGVALUE, the compiler ignores the parameter types.

6.3.1 Class Parameter to Be Evaluated at Compile Time (Curly Braces)

You can define a parameter as an ObjectScript expression that it is evaluated at compile time. To do so, specify no type and specify the value in curly braces:

Class Member

```
Parameter PARAMNAME = {ObjectScriptExpression};
```

where *PARAMNAME* is the parameter being defined and *ObjectScriptExpression* is the ObjectScript expression that is evaluated at compile time. This expression can use the *%classname* variable but otherwise cannot refer to the current class or any of its members. The expression can invoke code in other classes, provided that those classes are compiled before this class.

For example:

Class Member

```
Parameter COMPILETIME = {$zdatetime($h)};
```

6.3.2 Class Parameter to Be Evaluated at Runtime (COSEXPRESSION)

You can define a parameter as an ObjectScript expression that it is evaluated at runtime. To do so, specify its type as COSEXPRESSION and specify an ObjectScript expression as the value:

Class Member

```
Parameter PARAMNAME As COSEXPRESSION = "ObjectScriptExpression";
```

where *PARAMNAME* is the parameter being defined and *ObjectScriptExpression* is the ObjectScript content that is evaluated at runtime.

An example class parameter definition would be:

Class Member

```
Parameter DateParam As COSEXPRESSION = "$H";
```

6.3.3 Class Parameter to Be Updated at Runtime (CONFIGVALUE)

You can define a parameter so that it can modified outside of the class definition. To do so, specify its type as CONFIGVALUE. In this case, you can modify the parameter via the **$SYSTEM.OBJ.UpdateConfigParam()** method. For example, the following changes the value of the parameter *MYPARAM* (in the class MyApp.MyClass) so that its new value is 42:

```
set sc=$system.OBJ.UpdateConfigParam("MyApp.MyClass","MYPARAM",42)
```

Note that **$SYSTEM.OBJ.UpdateConfigParam()** affects the generated class descriptor as used by any new processes, but does not affect the class definition. If you recompile the class, InterSystems IRIS regenerates the class descriptor, which will now use the value of this parameter as contained in the class definition, thus overwriting the change made via **$SYSTEM.OBJ.UpdateConfigParam()**.

6.4 Referring to Parameters of a Class

To refer to a parameter of a class, you can do any of the following:

- Within a method of the associated class, use the following expression:

  ```
  ..#PARMNAME
  ```

  You can use this expression with the DO and SET commands, or you can use it as part of another expression. The following shows one possibility:

  ```
   set object.PropName=..#PARMNAME
  ```

  In the next variation, a method in the class checks the value of a parameter and uses that to control subsequent processing:

  ```
   if ..#PARMNAME=1 {
     //do something
   } else {
     //do something else
   }
  ```

- To access a parameter in any class, use the following expression:

  ```
  ##class(Package.Class).#PARMNAME
  ```

  where *Package.Class* is the name of the class and *PARMNAME* is the name of the parameter.

  This syntax accesses the given class parameter and returns its value. You can use this expression with commands such as DO and SET, or you can use it as part of another expression. The following shows an example:

  ObjectScript

  ```
   w ##class(%XML.Adaptor).#XMLENABLED
  ```

  displays whether methods generated by the XML adaptor are XML enabled, which by default is set to 1.

- To access the parameter, where the parameter name is not determined until runtime, use the $PARAMETER function:

  ```
  $PARAMETER(classnameOrOref,parameter)
  ```

  where *classnameOrOref* is either the fully qualified name of a class or an OREF of an instance of the class, and *parameter* evaluates to the name of a parameter in the associated class.

  For information on OREFs, see Working with Registered Objects.

6.5 See Also

- Parameter Syntax and Keywords
- Common Property Parameters
- Defining Classes

7
Defining and Calling Methods

This topic describes the rules and options for creating methods in InterSystems IRIS® data platform classes and for calling those methods.

Instance methods apply only to object classes and are not discussed here.

7.1 Introduction to Methods

A method is an executable element defined by a class. InterSystems IRIS supports two types of methods: instance methods and class methods. An *instance method* is invoked from a specific instance of a class and typically performs some action related to that instance. A *class method* is a method that can be invoked without reference to any object instance; these are called static methods in other languages.

The term *method* usually refers to an instance method. The more specific phrase *class method* is used to refer to class methods.

Because you cannot execute an instance method without an instance of an object, instance methods are useful only when defined in object classes. In contrast, you can define class methods in any kind of class.

7.2 Defining Methods

To add a class method to a class, add an element like the following to the class definition:

```
ClassMethod MethodName(Arguments) as Classname [ Keywords]
{
//method implementation
}
```

Where:

- *MethodName* is the name of the method. For rules, see Naming Conventions.
- *Arguments* is a comma-separated list of arguments. For details, see Specifying Method Arguments.
- *Classname* is an optional class name that represents the type of value (if any) returned by this method. Omit the `As` *Classname* part if the method does not return a value.

  The class can be a data type class, an object class, or (less commonly) a class of no type. The class name can be a fully qualified class name or a short class name. For details, see Package Use When Referring to Classes.

- *Keywords* represents any method keywords. These are optional. See Compiler Keywords.
- The method implementation depends on the implementation language and type of method; see Specifying the Implementation Language and Types of Methods. By default, the method implementation consists of zero or more lines of ObjectScript.

To add an instance method to a class, use the same syntax with `Method` instead of `ClassMethod`:

```
Method MethodName(arguments) as Classname [ Keywords]
{
 //method implementation
}
```

Instance methods are relevant only in object classes.

7.3 Specifying the Implementation Language

You have the choice of implementation language when creating a method. In fact, within a single class, it is possible to have multiple methods implemented in different languages. All methods interoperate regardless of implementation language. To specify which language you will write a method in, use the following syntax:

Method/ObjectScript

```
Method MyMethod() [ Language = objectscript ]
{
    // implementation details written in ObjectScript
}
```

Method/Python

```
Method MyMethod() [ Language = python ]
{
    # implementation details written in Python
}
```

You must write the method's language in all lowercase letters, as in the example.

If a method does not use the Language keyword the compiler will assume that the method is written in the language specified by the Language keyword of the class to which it belongs. For this keyword, the default is `objectscript` (ObjectScript); the other options are `python`, `tsql`, and `ispl`. For details, see Language (Method Keyword).

7.4 Specifying Method Arguments: Basics

A method can take any number of arguments. The method definition must specify the arguments that it takes. It can also specify the type and default value for each argument. (In this context, type refers to any kind of class, not specifically data type classes.)

Consider the following generic class method definition:

```
ClassMethod MethodName(Arguments) as Classname [ Keywords]
{
 //method implementation
}
```

Here *Arguments* has the following general form:

```
argname1 as type1 = value1, argname2 as type2 = value2, argname3 as type3 = value3, [and so on]
```

Where:

- *argname1*, *argname2*, *argname3*, and so on are the names of the arguments. These names must follow the rules for variable names.
- *type1*, *type2*, *type3*, and so on are class names. This part of the method definition is intended to describe, to programmers who might use this method, what type of value to pass for the corresponding argument. Generally it is a good idea to explicitly specify the type of each method argument.

  Typically the types are data type classes or object classes.

  The class name can be a fully qualified class name or a short class name. For details, see Package Use When Referring to Classes.

  You can omit this part of the syntax. If you do, also omit the `as` part.
- *value1*, *value2*, *value3*, and so on are the default values of the arguments. The method automatically sets the argument equal to this value if the method is called without providing a value for the argument.

  Each value can either be a literal value (`"abc"` or 42) or an ObjectScript expression enclosed in curly braces. For example:

  Class Member

  ```
  ClassMethod Test(flag As %Integer = 0)
  {
   //method implementation
  }
  ```

  For another example:

  Class Member

  ```
  ClassMethod Test(time As %Integer = {$horolog} )
  {
   //method implementation
  }
  ```

  You can omit this part of the syntax. If you do, also omit the equals sign (=).

For instance, here is a **Calculate()** method with three arguments:

Class Member

```
ClassMethod Calculate(count As %Integer, name, state As %String = "CA")
{
 // ...
}
```

where *count* and *state* are declared as %Integer and %String, respectively. By default, arguments are of the %String data type, so that an argument of unspecified type is a %String. This is the case for *name* in the above example.

7.5 Indicating How Arguments Are to Be Passed

The method definition also indicates, to programmers who might use the method, how each argument is expected to be passed. Arguments can be passed by value (the default technique) or by reference.

To indicate that an argument *should* be passed by reference, include the ByRef modifier in the method signature, before the name of the argument. The following shows an example that uses ByRef for both its arguments:

Class Member

```
/// Swap value of two integers
Method Swap(ByRef x As %Integer, ByRef y As %Integer)
{
    Set temp = x
    Set x = y
    Set y = temp
}
```

Similarly, to indicate that an argument *should* be passed by reference *and* is intended to have no incoming value, include the Output modifier in the method signature, before the name of the argument. For example:

Class Member

```
Method CreateObject(Output newobj As MyApp.MyClass) As %Status
{
    Set newobj = ##class(MyApp.MyClass).%New()
    Quit $$$OK
}
```

Important: The ByRef and Output keywords provide information for the benefit of anyone using the InterSystems Class Reference. They do not affect the behavior of the code. It is the responsibility of the writer of the method to enforce any rules about how the method is to be invoked.

To pass an argument by reference in ObjectScript, place a period before the variable name when invoking the method. In Python, use `iris.ref()` on the value you want to pass and call the method on the reference. Both of these are shown in the following example:

ObjectScript

```
 Do MyMethod(arg1, .arg2, .arg3)
```

Python

```
arg2=iris.ref("peanut butter")
arg3=iris.ref("jelly")
MyMethod(arg1,arg2,arg3)
```

For more information on passing such arguments in ObjectScript, see Passing ByRef or Output Arguments.

7.6 Skipping Arguments

When invoking a method you can skip arguments if there are suitable defaults for them. ObjectScript and Python each have their own syntax to skip arguments.

In ObjectScript, you can skip over an argument by providing no value for that argument and maintaining the comma structure. For example, the following is valid:

ObjectScript

```
 set myval=##class(mypackage.myclass).GetValue(,,,,,,4)
```

In an InterSystems IRIS class, a Python method's signature must list the required arguments first, followed by any arguments with default values. When calling the method, you must provide arguments in the order of the method's signature. Therefore, once you skip an argument you must also skip all arguments following it. For example, the following is valid:

Member/Python

```
ClassMethod Skip(a1, a2 As %Integer = 2, a3 As %Integer = 3) [ Language = python ]
{
    print(a1, a2, a3)
}
```

```
TESTNAMESPACE>do ##class(mypackage.myclass).Skip(1)
1 2 3
```

7.7 Specifying a Variable Number of Arguments

You can define a method that accepts variable numbers of arguments. To do so, include `...` after the name of the last argument within the method signature, as in the following example.

Method/ObjectScript

```
ClassMethod MultiArg(Arg1... As %List) [ Language = objectscript ]
{
 Set args = $GET(Arg1, 0)
 Write "Invocation has ",
     args,
     " element",
     $SELECT((args=1):"", 1:"s"), !
 For i = 1 : 1 : args
 {
     Write "Argument[", i , "]: ", $GET(Arg1(i), "<NULL>"), !
 }
}
```

Method/Python

```
ClassMethod MultiArg(Arg1... As %List) [ Language = Python ]
{
    print("Invocation has", len(Arg1), "elements")
    for i in range(len(Arg1)):
        print("Argument[" + str(i+1) + "]: " + Arg1[i])
}
```

For information on passing arguments in this case, see Passing a Variable Number of Arguments.

7.8 Specifying Default Values

To specify an argument's default value in either an ObjectScript or a Python method, use the syntax as shown in the following example:

Class Member

```
Method Test(flag As %Integer = 0)
{
 //method details
}
```

When a method is invoked, it uses its default values (if specified) for any missing arguments. If a method is written in Python, then any arguments with default values must be defined at the end of the argument list.

In ObjectScript, another option is to use the **$GET** function to set a default value. For example:

Class Member

```
Method Test(flag As %Integer)
{
  set flag=$GET(flag,0)
 //method details
}
```

This technique, however, does not affect the class signature.

7.9 Returning a Value

Except in the case of an expression method, to define a method so that it returns a value, use either of the following in the method (if you implement the method in ObjectScript):

```
 Return returnvalue
```

Or:

```
 Quit returnvalue
```

Where *returnvalue* is a suitable value for the method to return. This should be consistent with the declared return type of the method. If the return type is a data type class, the method should return a literal value. If the return type is an object class, the method should return an instance of that class (specifically an OREF). For details, see Working with Registered Objects.

For example:

Class Member

```
ClassMethod Square(input As %Numeric) As %Numeric
{
    Set returnvalue = input * input
    Return returnvalue
}
```

For another example, this method returns an object instance:

Class Member

```
ClassMethod FindPerson(id As %String) As Person
{
    Set person = ##class(Person).%OpenId(id)
    Return person
}
```

The syntax for returning a value is different depending on the implementation language of the method.

7.10 Overriding Inherited Methods

A class inherits methods (both class and instance methods) from its superclass or superclasses, which you can override. If you do so, you must ensure that the signature in your method definition matches the signature of the method you are overriding. Each argument of the subclass method must use the same data type as the superclass method's argument, or a subclass of that data type. The method in the subclass can, however, specify additional arguments that are not defined in the superclass.

You can override a method written in ObjectScript with a Python method and vice versa as long as the method signatures match.

Within a method in a subclass, you can refer to the method that it overrides in a superclass. To do so in ObjectScript, use the *##super()* syntax. For example:

```
//overrides method inherited from a superclass
Method MyMethod() [ Language = objectscript ]
{
  //execute MyMethod as implemented in the superclass
  do ##super()
  //do more things....
}
```

Note: *##super* is not case-sensitive.

7.11 Restricting Access by Using Privilege Checks

You can restrict access to a method by using the Requires keyword. Only users or processes that have the specified privilege (or privileges) can call the method.

Specify a privilege by naming a resource and the appropriate level of permission (Use, Read, or Write), separated with a colon. Use a comma-delimited list to specify more than one privilege.

In the following example, the **MyAction** method requires the Use permission on the Service_FileSystem resource:

Class Member

```
ClassMethod MyAction() [ Requires = Service_FileSystem: Use ]
{
  write "You have access to this method."
}
```

Calling the method without the required privilege results in a <PROTECT> error:

```
<PROTECT> *Method MyAction' requires resource 'Service_FileSystem: Use'
```

If a method inherits the Requires keyword from a superclass, you can add to the list of required privileges by setting a new value for the keyword. You cannot remove required privileges in this manner.

For details, see Privileges and Permissions and see the reference for the Requires keyword.

7.12 Types of Methods (CodeMode Options)

InterSystems IRIS supports four types of methods, which the class compiler handles differently:

- Code methods (the most default and the most common)
- Expression methods
- Call methods
- Method generators

7.12.1 Code Methods

A code method is a method whose implementation is simply lines of code. This is the most typical type of method and is the default.

The method implementation can contain any code that is valid for the implementation language.

Note: InterSystems IRIS comes with a set of system-defined methods that perform simple, common tasks. If a user-defined method performs one of these tasks, then the compiler does not generate any executable code for it. Rather, it associates the user-defined method with the system-defined method, so that invoking the user-defined method results in a call to the system-defined method, with an associated performance benefit. Also, the debugger does not step through such a system-defined method.

7.12.2 Expression Methods

An expression method is a method whose implementation consists of only an expression. When executed, the method returns the value of that expression. Expression methods are typically used for simple methods (such as those found in data type classes) that need rapid execution speed.

For example, it is possible to convert the **Speak()** method of the Dog class from the previous example into an expression method:

Class Member

```
Method Speak() As %String [CodeMode = expression]
{
    "Woof, Woof"
}
```

Assuming *dog* refers to a Dog object, this method could be used as follows:

ObjectScript

```
 Write dog.Speak()
```

Which could result in the following code being generated:

ObjectScript

```
 Write "Woof, Woof"
```

It is a good idea to give default values to all formal variables of an expression method. This prevents potential substitution problems caused by missing actual variables at runtime.

Note: InterSystems IRIS does not allow macros or call-by-reference arguments within expression methods.

7.12.3 Call Methods

A call method is a special mechanism to create method wrappers around existing InterSystems IRIS routines. This is typically useful when migrating legacy code to object-based applications.

Defining a method as a call method indicates that a specified routine is called whenever the method is invoked. The syntax for a call method is:

Class Member

```
Method Call() [ CodeMode = call ]
{
    Tag^Routine
}
```

where Tag^Routine specifies a tag name within a routine.

7.12.4 Method Generators

A method generator is actually a small program that is invoked by the class compiler during class compilation. Its output is the actual runtime implementation of the method. Method generators provide a means of inheriting methods that can produce high performance, specialized code that is customized to the needs of the inheriting class or property. Within the InterSystems IRIS library, method generators are used extensively by the data type and storage classes.

For details, see Defining Method and Trigger Generators.

7.13 Projecting a Method As an SQL Stored Procedure

You can define a class method (but not an instance method) so that it is also available as an SQL stored procedure. To do so, include the SqlProc keyword in the method definition.

The default name of the procedure is as *CLASSNAME_METHODNAME* To specify a different name, specify the SqlName keyword.

For details, see Defining and Using Stored Procedures.

7.14 Calling Class Methods

This section discusses how to call class methods in ObjectScript. This section applies to all kinds of classes. Note that instance methods, which apply only to object classes, are not discussed here.

- To call a class method of any class (if that method is not private), use the following expression:

  ```
  ##class(Package.Class).Method(Args)
  ```

  Where *Package.Class* is the name of the class, *Method* is the name of the method, and *Args* is any arguments of the method.

  `##class` is not case-sensitive.

  This expression invokes the given class method and obtains its return value (if any). You can use this expression with commands such as DO and SET, or you can use it as part of another expression. The following shows some variations:

  #### ObjectScript

  ```
  do ##class(Package.Class).Method(Args)
  set myval= ##class(Package.Class).Method(Args)
  write ##class(Package.Class).Method(Args)
  set newval=##class(Package.Class).Method(Args)_##class(Package2.Class2).Method2(Args)
  ```

  You can omit the package. If you do so, the class compiler determines the correct package name to use (this name resolution is explained in Packages).

- (Within a class method) to call another class method of that class (which could be an inherited method), use the following expression:

  ```
  ..MethodName(args)
  ```

  You can use this expression with the DO command. If the method returns a value, you can use SET, or you can use it as part of another expression. The following shows some variations:

  ```
  do ..MethodName()
  set value=..MethodName(args)
  ```

  Note: You cannot use this syntax in a class method to refer to a property or an instance method, because those references require the instance context.

- To execute a class method, where the method name is not determined until runtime, use the ObjectScript $CLASSMETHOD function:

  ```
  $CLASSMETHOD(classname, methodname, Arg1, Arg2, Arg3, ... )
  ```

  where *classname* evaluates to the fully qualified name of a class, *methodname* evaluates to the name of a class method in that class, and *Arg1*, *Arg2*, *Arg3*, and so on are any arguments to that method. For example, the following executes the **PrintPersons** method from the Sample.Person class:

  ObjectScript

  ```
  set cls="Sample.Person"
  set clsmeth="PrintPersons"
  do $CLASSMETHOD(cls,clsmeth)
  ```

If the given method does not exist or if it is an instance method instead, the system generates the <METHOD DOES NOT EXIST> error. If the given method is private, the system generates the <PRIVATE METHOD> error.

7.14.1 Passing Arguments to a Method

The default way to pass arguments to methods is by value. In this technique, simply include the arguments as variables, literal values, or other expressions within the method call, as shown in the preceding examples.

It is also possible to pass arguments by reference.

This works as follows: the system has a memory location that contains the value of each local variable. The name of the variable acts as the address to the memory location. When you pass a local variable to a method, you pass the variable by value. This means that the system makes a copy of the value, so that the original value is not affected. You can pass the memory address instead; this technique is known as *calling by reference*. It is also the only way to pass a multidimensional array as an argument.

In ObjectScript, to pass an argument by reference, precede that argument with a period. For example:

ObjectScript

```
set MyArg(1)="value A"
set MyArg(2)="value B"
set status=##class(MyPackage.MyClass).MyMethod(.MyArg)
```

In this example, we pass a value (a multidimensional array) by reference so that the method could receive the value. In other cases, it is useful to pass an argument by reference so that you can use its value after running the method. For example:

```
set status=##class(MyPackage.MyClass).GetList(.list)
//use the list variable in subsequent logic
```

In other cases, you might specify a value for the variable, invoke a method that modifies it (and that returns it by reference), and then use the changed value.

7.15 Casting a Method

To cast a method of one class as a method of another class, the syntax is either of the following (in ObjectScript):

```
Do ##class(Package.Class1)Class2Instance.Method(Args)
Set localname = ##class(Package.Class1)Class2Instance.Method(Args)
```

You can cast both class methods and instance methods.

For example, suppose that two classes, MyClass.Up and MyClass.Down, both have **Go()** methods. For **MyClass.Up**, this method is as follows

Class Member

```
Method Go()
{
    Write "Go up.",!
}
```

For **MyClass.Down**, the **Go()** method is as follows:

Class Member

```
Method Go()
{
    Write "Go down.",!
}
```

You can then create an instance of MyClass.Up and use it to invoke the **MyClass.Down.Go** method:

```
>Set LocalInstance = ##class(MyClass.Up).%New()

>Do ##class(MyClass.Down)LocalInstance.Go()
Go down.
```

It is also valid to use ##class as part of an expression, as in

ObjectScript

```
 Write ##class(Class).Method(args)*2
```

without setting a variable equal to the return value.

A more generic approach is to use the $METHOD and $CLASSMETHOD functions, which are for instance and class methods, respectively.

7.16 Overriding an Inherited Method

A class inherits methods (both class and instance methods) from its superclass or superclasses. Except for methods that are marked Final, you can override these definitions by providing a definition within this class. If you do so, note the following rules:

- If the method is a class method in the superclass, you cannot override it as an instance method in the subclass, and vice versa.

- The return type of the subclass method must be either the same as the original return type or a subclass of the original return type.
- The method in the subclass *can* have more arguments than the method in the superclass. (Also see Number of Arguments.)
- The method in the subclass can specify different default values for the arguments.
- The types of the arguments in the subclass method must be consistent with the types of the arguments in the original method. Specifically, any given argument must be either the same as the original type or a subclass of the original type.

  Note that if an argument has no specified type, the compiler treats the argument as %String. Thus if an argument in the superclass method has no type, the corresponding argument of a subclass method can be %String, can be a subclass of %String, or can have no type.
- The method in the subclass should receive argument values in the same way as the method in the superclass. For example, if a given argument is passed by reference in the superclass, the same argument should be passed by reference in the subclass.

  If the method signatures are inconsistent in this regard, it is harder for other developers to know how to use the methods appropriately. Note, however, that the compiler does not issue an error.

If your method implementation needs to call the method of the same name as defined in the superclass, you can use the syntax ##super(), which is discussed in the subsections. This discussion applies to code that is written in ObjectScript.

7.16.1 ##super()

Within a method, use the following expression to call the method of the same name as defined in the nearest superclass:

```
##super()
```

You can use this expression with the DO command. If the method returns a value, you can use SET, or you can use it as part of another expression. The following shows some variations:

```
 do ##super()
 set returnvalue=##super()_"additional string"
```

Note: *##super* is not case-sensitive.

This is useful if you define a method that should invoke the existing method of the superclass and then perform some additional steps such as modifying its returned value.

7.16.2 ##super and Method Arguments

##super also works with methods that accept arguments. If the subclass method does not specify a default value for an argument, make sure that the method passes the argument by reference to the superclass.

For example, suppose the code for the method in the superclass (**MyClass.Up.SelfAdd()**) is:

Class Member

```
ClassMethod SelfAdd(Arg As %Integer)
{
    Write Arg,!
    Write Arg + Arg
}
```

then its output is:

```
>Do ##Class(MyClass.Up).SelfAdd(2)
2
4
>
```

The method in the subclass (**MyClass.Down.SelfAdd()**) uses *##super* and passes the argument by reference:

Class Member

```
ClassMethod SelfAdd(Arg1 As %Integer)
{
    Do ##super(.Arg1)
    Write !
    Write Arg1 + Arg1 + Arg1
}
```

then its output is:

```
>Do ##Class(MyClass.Down).SelfAdd(2)
2
4
6
>
```

In **MyClass.Down.SelfAdd()**, notice the period before the argument name. If we omitted this and we invoked the method without providing an argument, we would receive an `<UNDEFINED>` error.

7.16.3 Calls That ##super Affects

##super only affects the current method call. If that method makes any other calls, those calls are relative to the current object or class, *not* the superclass. For example, suppose that MyClass.Up has **MyName()** and **CallMyName()** methods:

```
Class MyClass.Up Extends %Persistent
{

ClassMethod CallMyName()
{
    Do ..MyName()
}

ClassMethod MyName()
{
    Write "Called from MyClass.Up",!
}

}
```

and that MyClass.Down overrides those methods as follows:

```
Class MyClass.Down Extends MyClass.Up
{

ClassMethod CallMyName()
{
    Do ##super()
}

ClassMethod MyName()
{
    Write "Called from MyClass.Down",!
}

}
```

then invoking the **CallMyName()** methods have the following results:

```
USER>d ##class(MyClass.Up).CallMyName()
Called from MyClass.Up

USER>d ##class(MyClass.Down).CallMyName()
Called from MyClass.Down
```

MyClass.Down.CallMyName() has different output from **MyClass.Up.CallMyName()** because its **CallMyName()** method includes *##super* and so calls the **MyClass.Up.CallMyName()** method, which then calls the uncast **MyClass.Down.MyName()** method.

7.16.4 Number of Arguments

In some cases, you might find it necessary to add new arguments to a method in a superclass, thus resulting in more arguments than are defined in the method in a subclass. The subclasses will still compile, because (for convenience) the compiler appends the added arguments to the method in the subclass. In most cases, you should still examine all the subclasses that extend the method, and edit the signatures to account for the additional arguments, and decide whether you want to edit the code also. Even if you do not want to edit signatures or code, you still must consider two points:

- Make sure that the added argument names are not the same as the names of any variables used in the method in the subclass. The compiler appends the added arguments to the method in the subclass. If these arguments happen to have the same names as variables used in the method of the subclass, unintended results will occur.
- If the method in the subclass uses the added arguments (because this method uses *##super*), make sure that the method in the superclass specifies default values for the added arguments.

7.17 See Also

- Defining Classes
- Method Syntax and Keywords

8
Working with Registered Objects

The %RegisteredObject class is the basic object API in InterSystems IRIS® data platform. This topic describes how to use this API. Information in this topic applies to all subclasses of %RegisteredObject.

8.1 Introduction to Object Classes

An *object class* is any class that inherits from %RegisteredObject. With an object class, you can do the following things:

- Create instances of the class. These instances are known as objects.
- Set properties of those objects.
- Invoke methods of those objects (instance methods).

These tasks are possible only with object classes.

The classes %Persistent and %SerialObject are subclasses of %RegisteredObject. For an overview, see Object Classes.

8.2 OREF Basics

When you create an object, the system creates an in-memory structure, which holds information about that object, and also creates an *OREF* (object reference), which is a pointer to that structure.

The object classes provide several methods that create OREFs. When you work with any of the object classes, you use OREFs extensively. You use them when you specify values of properties of an object, access values of properties of the object, and call instance methods of the object. Consider the following example:

```
MYNAMESPACE>set person=##class(Sample.Person).%New()

MYNAMESPACE>set person.Name="Carter,Jacob N."

MYNAMESPACE>do person.PrintPerson()

Name: Carter,Jacob N.
```

In the first step, we call the **%New()** method of a class named Sample.Person; this creates an object and returns an OREF that points to that object. We set the variable `person` equal to this OREF. In the next step, we set the Name property of object. In the third step, we invoke the **PrintPerson()** instance method of the object. (Note that the Name property and the

PrintPerson() method are both just examples—these are defined in the Sample.Person class but are not part of the general object interface.)

An OREF is transient; the value exists only while the object is in memory and is not guaranteed to be constant over different invocations.

CAUTION: An OREF is only valid within the namespace where it was created; hence, if there are existing OREFs and the current namespace changes, any OREF from the previous namespace is no longer valid. If you attempt to use OREFs from other namespaces, there might not be an immediate error, but the results cannot be considered valid or usable, and may cause disastrous results in the current namespace.

8.2.1 INVALID OREF Error

In simple expressions, if you try to set a property, access a property, or invoke an instance method of a variable that is not an OREF, you receive an `<INVALID OREF>` error. For example:

```
MYNAMESPACE>write p2.PrintPerson()

WRITE p2.PrintPerson()
^
<INVALID OREF>
MYNAMESPACE>set p2.Name="Dixby,Jase"

SET p2.Name="Dixby,Jase"
^
<INVALID OREF>
```

Note: The details are different when the expression has a chain of OREFs; see Introduction to Dot Syntax.

8.2.2 Testing an OREF

InterSystems IRIS provides a function, **$ISOBJECT**, which you can use to test whether a given variable holds an OREF. This function returns 1 if the variable contains an OREF and returns 0 otherwise. If there is an chance that a given variable might not contain an OREF, it is good practice to use this function before trying to set a property, access a property, or invoke an instance method of the variable.

8.2.3 OREFs, Scope, and Memory

Any given OREF is a *pointer* to an in-memory object to which other OREFs might also point. That is, the OREF (which is a variable) is distinct from the in-memory object (although, in practice, the terms *OREF* and *object* are often used interchangeably).

InterSystems IRIS manages the in-memory structure automatically as follows. For each in-memory object, InterSystems IRIS maintains a *reference count* — the number of references to that object. Whenever you set a variable or object property to refer to a object, its reference count is automatically incremented. When a variable stops referring to an object (if it goes out of scope, is killed, or is set to a new value), the reference count for that object is decremented. When this count goes to 0, the object is automatically destroyed (removed from memory) and its **%OnClose()** method (if present) is called.

For example, consider the following method:

Class Member

```
Method Test()
{
    Set person = ##class(Sample.Person).%OpenId(1)

    Set person = ##class(Sample.Person).%OpenId(2)
}
```

This method creates an instance of Sample.Person and places a reference to it into the variable *person*. Then it creates another instance of Sample.Person and replaces the value of *person* with a reference to it. At this point, the first object is no longer referred to and is destroyed. At the end of the method, *person* goes out of scope and the second object is destroyed.

8.2.4 Removing an OREF

If needed, to remove an OREF, use the KILL command:

```
kill OREF
```

Where *OREF* is a variable that contains an OREF. This command removes the variable. If there are no further references to the object, this command also removes the object from memory, as discussed earlier.

8.2.5 OREFs, the SET Command, and System Functions

For some system functions (for example, $Piece, $Extract, and $List), InterSystems IRIS supports an alternative syntax that you can use to modify an existing value. This syntax combines the function with the SET command as follows:

```
SET function_expression = value
```

Where *function_expression* is a call to the system function, with arguments, and *value* is a value. For example, the following statement sets the first part of the `colorlist` string equal to `"Magenta"`:

```
SET $PIECE(colorlist,",",1)="Magenta"
```

It is not supported to modify OREFs or their properties in this way.

Also, $DATA(), $GET(), and $INCREMENT() can only take a property as the argument if the property is multidimensional. These operations can be done within an object method using the instance variable syntax `i%PropertyName`. Properties cannot be used with the MERGE command.

8.3 Creating New Objects

To create a new instance of a given object class, use the class method **%New()** of that class. This method creates an object and returns an OREF. The following shows an example:

ObjectScript

```
Set person = ##class(MyApp.Person).%New()
```

The **%New()** method accepts an argument, which is ignored by default. If present, this argument is passed to **%OnNew()** callback method of the class, if defined. If **%OnNew()** is defined, it can use the argument to initialize the newly created object in some way. For details, see Implementing Callback Methods.

If you have complex requirements that affect how you create new objects of given class, you can provide an alternative method to be used to create instances of that class. Such a method would call **%New()** and then would initialize properties of the object as needed. Such a method is sometimes called a *factory method*.

8.4 Viewing Object Contents

The WRITE command writes output of the following form for an OREF:

```
n@Classname
```

Where *Classname* is the name of the class, and *n* is an integer that indicates a specific instance of this class in memory. For example:

```
MYNAMESPACE>write p
8@Sample.Person
```

If you use the ZWRITE command with an OREF, InterSystems IRIS displays more information about the associated object.

```
MYNAMESPACE>zwrite p
p=<OBJECT REFERENCE>[8@Sample.Person]
+----------------- general information ---------------
|      oref value: 1
|      class name: Sample.Person
|           %%OID: $lb("3","Sample.Person")
| reference count: 2
+----------------- attribute values ------------------
|       %Concurrency = 1  <Set>
|                DOB = 33589
|               Name = "Clay,George O."
|                SSN = "480-57-8360"
+----------------- swizzled references ---------------
|    i%FavoriteColors = ""  <Set>
|    r%FavoriteColors = ""  <Set>
|             i%Home = $lb("5845 Washington Blvd","St Louis","NM",55683)  <Set>
|             r%Home = ""  <Set>
|           i%Office = $lb("3413 Elm Place","Pueblo","WI",98532)  <Set>
|           r%Office = ""  <Set>
|           i%Spouse = ""
|           r%Spouse = ""
+-----------------------------------------------------
```

Notice that this information displays the class name, the OID, the reference count, and the current values (in memory) of properties of the object. In the section `swizzled references`, the items with names starting `i%` are instance variables. (The items with names starting `r%` are for internal use only.)

8.5 Introduction to Dot Syntax

With an OREF, you can use *dot syntax* to refer to properties and methods of the associated object. This section introduces dot syntax, which is also discussed in later sections, along with alternative ways to refer to properties and methods of objects.

The general form of dot syntax is as follows:

```
oref.membername
```

For example, to specify the value of a property for an object, you can use a statement like this:

ObjectScript

```
 Set oref.PropertyName = value
```

where *oref* is the OREF of the specific object, *PropertyName* is the name of the property that you want to set, and *value* is an ObjectScript expression that evaluates to the desired value. This could be a constant or could be a more complex expression.

We can use the same syntax to invoke methods of the object (instance methods). An *instance method* is invoked from a specific instance of a class and typically performs some action related to that instance. In the following example, we invoke the **PrintPerson()** method on the object whose *Name* property was just set:

ObjectScript

```
set person=##class(Sample.Person).%New()
set person.Name="Carter,Jacob N."
do person.PrintPerson()
```

If the method returns a value, you can use the SET command to assign the returned value to a variable:

```
SET myvar=oref.MethodName()
```

If the method does not return a value (or if you are uninterested in the return value), use either DO or JOB:

```
Do oref.MethodName()
```

If the method accepts arguments, specify them within the parentheses.

ObjectScript

```
Set value = oref.methodName(arglist)
```

8.5.1 Cascading Dot Syntax

Depending on the class definition, a property can be object-valued, meaning that its type is an object class. In such cases, you can use a chain of OREFs to refer to a property of the properties (or to a method of the property). This is known as *cascading dot syntax*. For example, the following syntax refers to the `Street` property of the `HomeAddress` property of a `Person` object:

```
set person.HomeAddress.Street="15 Mulberry Street"
```

In this example, the `person` variable is an OREF, and the expression `person.HomeAddress` is also an OREF.

Note: When referring to a class member generally, sometimes the following informal reference is used: *PackageName.ClassName.Member*, for example, the Accounting.Invoice.LineItem property. This form never appears in code.

8.5.2 Cascading Dot Syntax with a Null OREF

When you use a chain of OREFs to refer to a property, and an intermediate object has not been set, it is often desirable to return a null string for the expression instead of receiving an `<INVALID OREF>` error on the intermediate object. Thus if the intermediate object has not been set (is equal to a null string), the value returned for the chained property reference is a null string.

For example, if *pers* is a valid instance of Sample.Person and *pers*.Spouse equals `""`, then the following statement sets the *name* variable to `""`:

```
set name=pers.Spouse.Name
```

If this behavior is not appropriate in some context, then your code must explicitly check the intermediate object reference.

8.6 Validating Objects

The %RegisteredObject class provides a method, %ValidateObject, that validates the properties of an instance. This method returns 1 if the following criteria are true:

- All required properties have values. To make a property required, you use the Required keyword. If a property is of type %Stream, the stream cannot be a null stream, that is, it cannot have a value for which the **%IsNull()** method returns 1.
- The value of each property, if not null, is valid for the associated property definition.

  For example, if a property is of type %Boolean, the value `"abc"` is not valid, but the values 0 and 1 are.
- The value of each literal property, if not null, obeys all constraints defined by the property definition. The term *constraint* refers to any property keyword that applies a constraint on a property value, such as the *MAXLEN*, *MAXVAL*, *MINVAL*, *VALUELIST*, or *PATTERN* keywords. For a complete list, see Defining and Using Literal Properties.

  For example, for a property of type %String, the default value of *MAXLEN* is 50, which constrains the property to be no more than 50 characters in length.
- For any object-valued property, all properties of the referenced object obey the preceding rules.

%ValidateObject() calls the validation logic for each property in turn. For more details, see Using and Overriding Property Methods.

If any property fails a validation check, then the method returns an error status.

Consider a `Sample.Person` class. Objects of this class include a required social security number property, SSN, that must be specified in the format *NNN-NN-NNNN*.

Class Member

```
Property SSN As %String(PATTERN = "3N1""-""2N1""-""4N") [ Required ];
```

If you create a new `Sample.Person` object but do not include this property, then **%ValidateObject()** returns an error status indicating that the property is missing and does not match the pattern.

ObjectScript

```
set person = ##class(Sample.Person).%New()
set status = person.%ValidateObject()
do $system.Status.DisplayError(status)
```

```
ERROR #5659: Property 'Sample.Person::SSN(9@Demo.Person,ID=)' required
ERROR #7209: Datatype value '' does not match PATTERN '3N1"-"2N1"-"4N'
  > ERROR #5802: Datatype validation failed on property 'Sample.Person:SSN', with value equal to ""
```

If you set the property but do not the match the specification pattern, then **%ValidateObject()** returns only the pattern matching error status.

ObjectScript

```
set person.SSN = "0000000000"
set status = person.%ValidateObject()
do $system.Status.DisplayError(status)
```

```
ERROR #7209: Datatype value '' does not match PATTERN '3N1"-"2N1"-"4N'
  > ERROR #5802: Datatype validation failed on property 'Sample.Person:SSN', with value equal to ""
```

Once you set the property correctly, the object passes the **%ValidateObject()** check and returns a status of 1.

For a persistent object to be considered valid, its properties must also not violate any defined uniqueness constraints. **%ValidateObject()** does not check for uniqueness, because its checks against only the properties within the object, not against the properties of other objects saved to the database. The check for uniqueness occurs when you call **%Save()** to save the object.

Building on the previous example, suppose that the `Sample.Person` class constrains the `SSN` property to be unique.

```
Index SSNIndex On SSN [ Unique ]
```

If you create a new person and set the same social security number as the previous person, **%ValidateObject()** returns a status of 1 and **$system.Status.DisplayError** does not display an error for this status.

ObjectScript

```
do person.%Save() // Save first object to database

set person2 = ##class(Sample.Person).%New()
set person2.SSN = "000-00-0000"
set status = person2.%ValidateObject()
do $system.Status.DisplayError(status)
```

Saving the object to the database fails, however, because of the uniqueness constraint placed on the `SSN` property.

ObjectScript

```
set status = person2.%Save()
do $system.Status.DisplayError(status)
```

```
ERROR #5808: Key not unique: Sample.Person:SSNIndex:^Sample.PersonI("SSNIndex"," 000-00-0000")
```

Note that when you save a persistent object, the system automatically calls the **%ValidateObject()** method first. If the object fails this check, and it fails the uniqueness check, InterSystems IRIS does not save it. For more details, see Introduction to Persistent Objects.

8.7 Determining an Object Type

Given an object, the %RegisteredObject class provides methods to determine its inheritance. This section refers to the following sample classes:

Class Definition

```
Class Test.Superclass Extends (%RegisteredObject, %XML.Adaptor)
{

/// properties not shown

}
```

And

Class Definition

```
Class Test.Subclass Extends (Test.Superclass, Test.OtherClass)
{

/// properties not shown

}
```

And

Class Definition

```
Class Test.Subclass2 Extends Test.Subclass
{

/// properties not shown

}
```

8.7.1 %Extends()

To check if an object inherits from a specific superclass, call its **%Extends()** method, and pass the name of that superclass as the argument. If this method returns 1, then the instance inherits from that class. If it returns 0, the instance does not inherit from that class. For example:

Terminal

```
USER>set t=##class(Test.Subclass2).%New()

USER>write t.%Extends("Test.Subclass2")
1
USER>write t.%Extends("Test.Subclass")
1
USER>write t.%Extends("Test.OtherClass")
1
USER>write t.%Extends("Test.Superclass")
1
USER>write t.%Extends("%XML.Adaptor")
1
USER>write t.%Extends("%RegisteredObject")
1
USER>write t.%Extends("%Persistent")
0
```

8.7.2 %IsA()

To check if an object has a specific class as one of its *primary* superclasses, call its **%IsA()** method, and pass the name of that superclass as the argument. If this method returns 1, the object does have the given class as one of its primary superclasses. For example:

Terminal

```
USER>write t.%IsA("Test.Subclass2")
1
USER>write t.%IsA("Test.Subclass")
1
USER>write t.%IsA("Test.OtherClass")
0
USER>write t.%IsA("Test.Superclass")
1
USER>write t.%IsA("%XML.Adaptor")
0
USER>write t.%IsA("%RegisteredObject")
1
USER>write t.%IsA("%Persistent")
0
```

8.7.3 %ClassName() and the Most Specific Type Class (MSTC)

Although an object may be an instance of more than one class, it always has a *most specific type class (MSTC)*. A class is said to be the most specific type of an object when that object is an instance of that class and is not an instance of any subclass of that class.

For example, in the case of the GradStudent class inheriting from the Student class that inherits from the Person class, for instances created by the commands:

ObjectScript

```
set MyInstance1 = ##class(MyPackage.Student).%New()
set MyInstance2 = ##class(MyPackage.GradStudent).%New()
```

MyInstance1 has Student as its MSTC, since it is an instance of both Person and Student, but not of GradStudent. *MyInstance2* has GradStudent as its MSTC, since it is an instance of GradStudent, Student, and Person.

The following rules also apply regarding the MSTC of an object:

- The MSTC of an object is based solely on primary inheritance.
- A non-instantiable class cannot ever be the MSTC of an object. An object class is non-instantiable if it is abstract.

To determine the MSTC of an object, use the **%ClassName()** method, which is inherited from %RegisteredObject

```
classmethod %ClassName(fullname As %Boolean) as %String
```

Where *fullname* is a boolean argument where 1 specifies that the method return a package name and class name and 0 (the default) specifies that the method return only the class name.

For example:

```
write myinstance.%ClassName(1)
```

(Similarly, you can use **%PackageName()** to get just the name of the package.)

8.8 Cloning Objects

To clone an object, call the **%ConstructClone()** method of that object. This method creates a new OREF.

The following Terminal session demonstrates this:

```
MYNAMESPACE>set person=##class(Sample.Person).%OpenId(1)

MYNAMESPACE>set NewPerson=person.%ConstructClone()

MYNAMESPACE>w

NewPerson=<OBJECT REFERENCE>[2@Sample.Person]
person=<OBJECT REFERENCE>[1@Sample.Person]
MYNAMESPACE>
```

Here, you can see that the `NewPerson` variable uses a different OREF than the original `person` object. `NewPerson` is a clone of `person` (or more precisely, these variables are pointers to separate but identical objects).

In contrast, consider the following Terminal session:

```
MYNAMESPACE>set person=##class(Sample.Person).%OpenId(1)

MYNAMESPACE>set NotNew=person

MYNAMESPACE>w

NotNew=<OBJECT REFERENCE>[1@Sample.Person]
person=<OBJECT REFERENCE>[1@Sample.Person]
```

Notice that here, both variables refer to the same OREF. That is, `NotNew` is not a clone of `person`.

For information on arguments to this method, see the InterSystems Class Reference for %Library.RegisteredObject.

8.9 Referring to Properties of an Instance

To refer to a property of an instance, you can do any of the following:

- Create an instance of the associated class, and use dot syntax to refer to the property of that instance, as described previously.
- Within an instance method of the associated class, use *relative dot syntax*, as follows:

  ```
  ..PropName
  ```

  You can use this expression with the SET command, or you can use it as part of another expression. The following shows some variations:

  ```
   set value=..PropName
   write ..PropName
  ```

- To access the property, where the property name is not determined until runtime, use the $PROPERTY function. If the property is multidimensional, the following arguments after the property name are used as indexes in accessing the value of the property. The signature is:

  ```
  $PROPERTY (oref, propertyName, subscript1, subscript2, subscript3... )
  ```

  Where *oref* is an OREF, *propertyName* evaluates to the name of a property method in the associated class. Also, *subscript1*, *subscript2*, *subscript3* are values for any subscripts of the property; specify these only for a multidimensional property.

  For more information, see $PROPERTY.

- (Within an instance method) use the variable $this.
- (Within an instance method) use instance variables.
- Use the applicable property accessor (getter and setter) methods. See Using and Overriding Property Methods.

8.10 Calling Methods of an Instance

To call a method of an instance, you can do any of the following:

- Create an instance of the associated class, and use dot syntax to call the method of that instance, as described previously.
- (Within an instance method) to call another instance method of that class (which could be an inherited method), use the following expression:

  ```
  ..MethodName(args)
  ```

  You can use this expression with the DO command. If the method returns a value, you can use SET, or you can use it as part of another expression. The following shows some variations:

  ```
   do ..MethodName()
   set value=..MethodName(args)
  ```

- To execute an instance method, where the method name is not determined until runtime, use the $METHOD function:

  ```
  $METHOD(oref, methodname, Arg1, Arg2, Arg3, ... )
  ```

where *oref* is an OREF, *methodname* evaluates to the name of an instance method in the associated class, and *Arg1*, *Arg2*, *Arg3*, and so on are any arguments to that method.

For more information, see $METHOD.

- (Within an instance method) use the variable $this.

8.11 Getting the Class Name from an Instance

To get the name of a class, use the $CLASSNAME function:

```
$CLASSNAME(oref)
```

where *oref* is an OREF.

For more information, see $CLASSNAME.

8.12 $this Variable (Current Instance)

The *$this* syntax provides a handle to the OREF of the current instance, such as for passing it to another class or for another class to refer to properties of methods of the current instance. When an instance refers to its properties or methods, the relative dot syntax is faster and thus is preferred.

Note: *$this* is not case-sensitive; hence, *$this*, *$This*, *$THIS*, or any other variant all have the same value.

For example, suppose there is an application with an Accounting.Order class and an Accounting.Utils class. The **Accounting.Order.CalcTax()** method calls the **Accounting.Utils.GetTaxRate()** and **Accounting.Utils.GetTaxableSubtotal()** methods, passing city and state of the current instance to the **GetTaxRate()** method and passing the list of items ordered and relevant tax-related information to **GetTaxableSubtotal()**. **CalcTax()** then uses the values returned to calculate the sales tax for the order. Hence, its code is something like:

Class Member

```
Method CalcTax() As %Numeric
{
    Set TaxRate = ##Class(Accounting.Utils).GetTaxRate($this)
    Write "The tax rate for ",..City,", ",..State," is ",TaxRate*100,"%",!
    Set TaxableSubtotal = ##class(Accounting.Utils).GetTaxableSubTotal($this)
    Write "The taxable subtotal for this order is $",TaxableSubtotal,!
    Set Tax = TaxableSubtotal * TaxRate
    Write "The tax for this order is $",Tax,!
}
```

The first line of the method uses the *##Class* syntax (described earlier) to invoke the other method; it passes the current object to that method using the *$this* syntax. The second line of the method uses the .. syntax (also described earlier) to get the values of the *City* and *State* properties. The action on the third line is similar to that on the first line.

In the Accounting.Utils, the **GetTaxRate()** method can then use the handle to the passed-in instance to get handles to various properties — for both getting and setting their values:

Class Member

```
ClassMethod GetTaxRate(OrderBeingProcessed As Accounting.Order) As %Numeric
{
    Set LocalCity = OrderBeingProcessed.City
    Set LocalState = OrderBeingProcessed.State
    // code to determine tax rate based on location and set
    // the value of OrderBeingProcessed.TaxRate accordingly
    Quit OrderBeingProcessed.TaxRate
}
```

The **GetTaxableSubtotal()** method also uses the handle to the instance to look at its properties and set the value of its TaxableSubtotal property.

Hence, if we invoke the **CalcTax()** method of *MyOrder* instance of the Accounting.Order class, we would see something like this:

```
>Do MyOrder.CalcTax()
The tax rate for Cambridge, MA is 5%
The taxable subtotal for this order is $79.82
The tax for this order is $3.99
```

8.13 i%PropertyName (Instance Variables)

This section introduces instance variables. You do not need to refer to these variables unless you override an *accessor method* for a property; see Overriding Property Methods.

When you create an instance of any class, the system creates an instance variable for each non-calculated property of that class. The instance variable holds the value of the property. For the property *PropName*, the instance variable is named *i%PropName*, and this variable name is case-sensitive. These variables are available within any instance method of the class.

For example, if a class has the properties Name and DOB, then the instance variables *i%Name* and *i%DOB* are available within any instance method of the class.

Internally, InterSystems IRIS also uses additional instance variables with names such as *r%PropName* and *m%PropName*, but these are not supported for direct use.

Instance variables have process-private, in-memory storage allocated for them. Note that these variables are not held in the local variable symbol table and are not affected by the **Kill** command.

8.14 See Also

- Defining Classes
- Introduction to Persistent Objects
- Defining and Using Properties

9
Defining and Using Properties

You can define properties in any object class. This topic provides an overview.

9.1 Kinds of Properties

An object class can include any mix of the following kinds of properties:

- Literal properties, where the property holds a literal value.
- Stream properties, where the property is a stream class that provides access to the global that holds the value. In this case, the value can be an extremely large amount of data (longer than the *string length limit*).
- Object-valued properties, where the property is an object class.
- Serial properties, where the property is a serial object class.
- Collection properties, where the property is a specialized object that provides access to a collection of values, either an array or a list. The collection can consist of either literal values or of objects.
- Relationships, where the property is a specialized object that refers to another object class and that adds a specific form of constraint to the two connected classes. These are supported only for persistent classes.

9.2 Defining Properties

To add a property to a class definition, add an element like one of the following to the class:

Class Member

```
Property PropName as Classname;

Property PropName as Classname [ Keywords ] ;

Property PropName as Classname(PARAM1=value,PARAM2=value) [ Keywords ] ;

Property PropName as Classname(PARAM1=value,PARAM2=value) ;
```

Where:

- *PropName* is the name of the property.

- *Classname* is the class on which this property is based. If you omit this, *Classname* is assumed to be %String, which has a default maximum length of 50 characters. *Classname* can be a data type class (resulting in a literal property), an object class, or a serial object class.

  Also, you can use the following syntax variations to create a list property or an array property, respectively:

  ```
  Property PropName As list of Classname;
  ```

  ```
  Property PropName As array of Classname;
  ```

- *Keywords* represents any property keywords. See Compiler Keywords.
- *PARAM1*, *PARAM2*, and so on are property parameters. The available parameter depend on the class used by the property.

Notice that the property parameters, if included, are enclosed in parentheses and precede any property keywords. The property keywords, if included, are enclosed in square brackets at the end of the property definition.

9.3 Assigning Values to Properties

To assign a value to a property, in the simplest cases you do the following:

1. Create or open an instance of the containing class, obtaining an OREF to that instance.
2. Use the SET command and dot syntax to assign a value as follows:

   ```
    Set oref.PropertyName = value
   ```

   Where `oref` is the OREF returned in the first step, `PropertyName` is the name of a property of this object, and `value` is a suitable value for this property:

   - If this is a literal property, `value` should evaluate to a literal value.
   - If the property is an object (including serial objects), then `value` should be an OREF that refers to the applicable object class.

The details are somewhat different for collection properties, relationships, and stream properties. These are all object classes that provide an API to access data. . The following shows an example in which we set the first list item of the `FavoriteColors` property of a `Sample.Person` object.

```
 set oref.FavoriteColors.GetAt(1)="red"
```

For another example, the following writes data into a stream property:

```
  Do person.Memo.Write("This is some text. ")
  Do person.Memo.Write("This is some more text.")
```

9.4 Referring to Property Values

To refer to a property value, in the simplest cases you do the following:

1. Create or open an instance of the containing class, obtaining an OREF to that instance.

2. Use dot syntax to refer to a property:

   ```
   Set newvariable = oref.PropertyName
   ```

   Where `oref` is the OREF returned in the first step, `PropertyName` is the name of a property of this object.

The details are somewhat different for collection properties, relationships, and stream properties. These are all object classes that provide an API to access data. The following shows an example in which we obtain the first list item from the `FavoriteColors` property of a `Sample.Person` object.

```
Set newvariable=oref.FavoriteColors.GetAt(1)
```

Similarly, the following writes data from a stream property to the console:

```
 Write person.Memo.Read()
```

9.5 Using Property Methods

Every property adds a set of generated methods to the class. These methods include **propnameIsValid()**, **propnameLogicalToDisplay()**, **propnameDisplayToLogical()**, **propnameLogicalToOdbc()**, **propnameOdbcToLogical()**, and others, where *propname* is the property name. For details and a list of the methods, see Using Property Methods.

InterSystems IRIS uses these methods internally, and you can call them directly as well. In each case, the argument is a property value. For example, suppose that `Sample.Person` had a property named `Gender` with logical values `M` and `F` (and display values `Male` and `Female`). You could display the logical and display values for this property for a given record, as follows;

```
MYNAMESPACE>set p=##class(Sample.Person).%OpenId(1)

MYNAMESPACE>w p.Gender
M
MYNAMESPACE>w ##class(Sample.Person).GenderLogicalToDisplay(p.Gender)
Male
```

9.6 See Also

- Defining and Using Literal Properties
- Defining and Using Object-Valued Properties (includes serial properties)
- Defining and Using Relationships
- Working with Streams
- Working with Collections

10

Using Property Methods

This topic describes and provides reference information for property methods, which are the actual methods that InterSystems IRIS® data platform uses when you use OREFs to work with the properties of objects. You can use these methods directly.

For information on how these methods are defined, see Overriding Property Methods.

10.1 Introduction

When a class has properties, the class compiler automatically generates a number of methods for each property and uses those internally. The names of the methods are based on the property names. In most cases, you cannot see these methods; access for simple properties is implemented directly within the virtual machine for optimal performance. The last section provides sample code you can use to list these methods.

For example, if we define a class Person with three properties:

Class Definition

```
Class MyApp.Person Extends %Persistent
{

Property Name As %String;

Property Age As %Integer;

Property DOB As %Date;
}
```

When compiled, this class includes the following methods:

- NameDisplayToLogical(), NameGet(), NameGetStored(), NameIsValid(), NameLogicalToDisplay(), NameLogicalToOdbc(), NameNormalize(), NameSet()
- AgeDisplayToLogical(), AgeGet(), AgeGetStored(), AgeIsValid(), AgeLogicalToDisplay(), AgeLogicalToOdbc(), AgeNormalize(), AgeSet()
- DOBDisplayToLogical(), DOBGet(), DOBGetStored(), DOBIsValid(), DOBLogicalToDisplay(), DOBLogicalToOdbc(), DOBNormalize(), DOBSet()

Most of these are instance methods that take one argument: the value of the property to be used. For example:

ObjectScript

```
 Set x = person.DOBIsValid(person.DOB)
 Write person.DOBLogicalToDisplay(person.DOB)
```

The InterSystems IRIS dot syntax for referring to object properties is an interface for a set of accessor methods to retrieve and set values. For each non-calculated property, whenever the code refers to **oref.Prop** (where *oref* is an object and Prop is a property), it is executed as if a system-supplied **PropGet()** or **PropSet()** method were invoked. For example:

ObjectScript

```
Set person.DOB = x
```

acts as if the following method was called:

ObjectScript

```
Do person.DOBSet(x)
```

while:

ObjectScript

```
Write person.Name
```

acts like:

ObjectScript

```
Write person.NameGet()
```

Accessing the properties of an object by using the **PropGet()** and **PropSet()** methods requires that the object be loaded into memory. On the other hand, the **PropGetStored()** class method allows you to retrieve the property value of a stored object directly from disk, without having to load the entire object into memory. For example, to write the name of the person with ID 44, you could use:

ObjectScript

```
Write ##class(MyApp.Person).NameGetStored(44)
```

10.2 Data Formats

Many of the property methods translate data from one format to another, for example when displaying data in a human-readable format or when accessing data via ODBC. For reference, the formats are:

- *Display* — The format in which data can be input and displayed. For instance, a date in the form of “April 3, 1998” or “23 November, 1977.”
- *Logical* — The in-memory format of data, which is the format upon which operations are performed. While dates have the display format described above, their logical format is integer; for the sample dates above, their values in logical format are 57436 and 50000, respectively.
- *Storage* — The on-disk format of data — the format in which data is stored to the database. Typically this is identical to the logical format.
- *ODBC* — The format in which data can be presented via ODBC or JDBC. This format is used when data is exposed to ODBC/SQL. The available formats correspond to those defined by ODBC.
- *XSD* — SOAP-encoded format. This format is used when you export to or import from XML. This applies only to classes that are XML-enabled.

10.3 Reference

All method signatures are examples that use a property named `sampleprop`.

BuildValueArray()

```
Method samplepropBuildValueArray(serializedcollection, byRef array) as %Status
```

Example name: **samplepropBuildValueArray()**

Where available: list and array properties.

Given the serialized form of a collection object, this method returns (by reference) a multidimensional array containing that collection. To obtain the serialized form of a collection property, call the **Serialize()** method of the collection object. For example, suppose that `Sample.Person` has a property defined as follows:

Class Member

```
Property FavoriteColors As list Of %String;
```

The following example demonstrates serializing this property and obtaining a multidimensional array:

```
USER>set p=##class(Sample.Person).%OpenId(1)

USER>set serialized=p.FavoriteColors.Serialize()

USER>set status=p.FavoriteColorsBuildValueArray(serialized,.array)

USER>write status
1
USER>zw array
array(1)="red"
array(2)="pink"
array(3)="purple"
```

CollectionToDisplay()

```
Method samplepropCollectionToDisplay(serializedcollection) as ReturnValue
```

Where available: list and array properties.

Given the serialized form of a collection object, this method returns the value in display format. To obtain the serialized form of a collection property, call the **Serialize()** method of the collection object. For example, suppose that `Sample.Person` has a property defined as follows:

Class Member

```
Property FavoriteColors As list Of %String;
```

The following example demonstrates serializing this property and obtaining the display value:

```
USER>set p=##class(Sample.Person).%OpenId(1)

USER>w p.FavoriteColorsCollectionToDisplay(p.FavoriteColors.Serialize())
red
pink
purple
```

CollectionToOdbc()

```
Method samplepropCollectionToOdbc(serializedcollection) as ReturnValue
```

Where available: list and array properties.

Given the serialized form of a collection object, this method returns the value in ODBC format. To obtain the serialized form of a collection property, call the **Serialize()** method of the collection object. For example, suppose that `Sample.Person` has a property defined as follows:

Class Member

```
Property FavoriteColors As list Of %String;
```

The following example demonstrates serializing this property and obtaining the ODBC value:

```
USER>set p=##class(Sample.Person).%OpenId(1)

USER>w p.FavoriteColorsCollectionToDisplay(p.FavoriteColors.Serialize())
red,pink,purple
```

DisplayToCollection()

```
Method samplepropDisplayToCollection(delimitedstring) as ReturnValue
```

Where available: list and array properties.

This method accepts a delimited string and converts it to a serialized collection value.

DisplayToLogical()

```
Method samplepropDisplayToLogical(InputValue) as ReturnValue
```

Where available: literal properties and list and array properties containing literal values, if the data type class defines **DisplayToLogical()**.

Given a display value, this method converts it to a logical value, as appropriate for the property definition.

For example, consider the following property definition:

Class Member

```
Property Test(DISPLAYLIST = ",one,two,three", VALUELIST = ",1,2,3");
```

Given an instance of the class that contains this property, we can get the logical value corresponding to `two`, as follows:

```
USER>write x.TestDisplayToLogical("two")
2
```

Get()

```
Method samplepropGet() as ReturnValue
```

Where available: literal properties and object-valued properties.

This method returns the in-memory value of the property. This method is not used when you access the data via SQL.

For example (showing a literal property):

Terminal

```
USER>set test=##class(Sample.Person).%OpenId(1)

USER>write test.NameGet()
Sample Name
```

For a object-valued property, the in-memory value is an OREF:

Terminal

```
USER>set oref=test.AddressGet()

USER>write
oref=<OBJECT REFERENCE>[539@Sample.Address]
test=<OBJECT REFERENCE>[533@Sample.Person]
USER>w oref
539@Sample.Address
USER>w oref.Street
47 Pinkerton Way
```

GetObject()

```
Method samplepropGetObject() as OID
```

Where available: object-valued properties.

This method gets the OID associated with the property. This method is not used when you access the data via SQL. This is similar to **GetObjectId()** except that an OID is returned.

GetObjectId()

```
Method samplepropGetObjectId() as ID
```

Where available: object-valued properties.

This method gets the ID associated with the property. This method is not used when you access the data via SQL.

```
USER>set test=##class(Sample.Person).%OpenId(1000)

USER>write test.AddressGetObjectId()
5
```

In this case, the `Address` property of `Sample.Person` is the `Sample.Address` object whose ID is 5.

GetStored()

```
ClassMethod samplepropGetStored(id) as ReturnValue
```

Where available: literal properties and object-valued properties.

Given the ID of a saved object, this method returns the value of the given property directly from disk, without having to load the entire object into memory. This method is not used when you access the data via SQL. For example, to write the name of the person with ID `44`, you could use:

Terminal

```
USER>write ##class(Sample.Person).NameGetStored(44)
Sample Name
```

GetSwizzled()

```
Method samplepropGetSwizzled(onlyCheck as %Boolean) as OREF
```

Where available: object-valued properties.

Opens the given object instance, loading it into memory. This is known as swizzling.

If *onlyCheck* is omitted, this method swizzles the given object and returns an OREF for the object. If *onlyCheck* is 1, the method returns an OREF for the object, but only if it is already swizzled. Otherwise, it returns a null OREF.

For example:

```
USER>set emp = ##class(Sample.Employee).%OpenId(102)

USER>set co = emp.CompanyGetSwizzled(1)

USER>if (co = "") {write "Company is not swizzled"} else {write co.Name}
Company is not swizzled
USER>set co = emp.CarGetSwizzled()

USER>if (co = "") {write "Company is not swizzled"} else {write co.Name}
Acme Company, Inc.
```

IsEmpty()

```
Method samplepropIsEmpty() as %Boolean
```

Where available: serial properties. For example, suppose that `Sample.Person` has a property named `SerialAddress` of type `Sample.SerialAddress`, which is a serial class.

```
USER>set new=##class(Sample.Person).%New()

USER>write new.SerialAddressIsEmpty()
1
USER>set new.SerialAddress=##class(Sample.SerialAddress).%New()

USER>write new.SerialAddressIsEmpty()
1
USER>set new.SerialAddress.Street="17 Turnip Ave"

USER>write new.SerialAddressIsEmpty()
0
```

IsValid()

```
Method samplepropIsValid(InputValue) as %Status
```

Where available: literal properties (including %Library.List, which corresponds to the native list format), serial properties, and list and array properties.

Given an input value, this method performs validation as appropriate for the property definition.

For example, consider the following property definition:

Class Member

```
Property Test(DISPLAYLIST = ",one,two,three", VALUELIST = ",1,2,3");
```

The value 3 is valid for this property, but the value 4 is not:

```
USER>set x=##class(Sample.Person).%New()

USER>set status=x.TestIsValid(3)

USER>write status
1
USER>set status=x.TestIsValid(4)

USER>write status
0 M%,1,2,36USER-$e^zTestIsValid+1^Sample.Person.1^1e^^
USER>do $system.Status.DisplayError(status)

ERROR #7205: Datatype value '4' not in VALUELIST ',1,2,3'
```

LogicalToDisplay()

```
Method samplepropLogicalToDisplay(InputValue) as ReturnValue
```

Where available: literal properties and list and array properties containing literal values, if the data type class defines **LogicalToDisplay()**.

Given a logical value, this method converts it to a display value, as appropriate for the property definition.

For example, consider the following property definition:

Class Member

```
Property Test(DISPLAYLIST = ",one,two,three", VALUELIST = ",1,2,3");
```

Given an instance of the class that contains this property, we can get the display value corresponding to 2, as follows:

Terminal

```
USER>write x.TestLogicalToDisplay(2)
two
```

LogicalToOdbc()

```
Method samplepropLogicalToOdbc(InputValue) as ReturnValue
```

Where available: literal properties and list and array properties containing literal values, if the data type class defines **LogicalToOdbc()**.

Given a logical input value, this method converts it to an ODBC value, as appropriate for the property definition.

For example, suppose that `Integer` is a property of type integer. Given an instance `x` of the class that contains this property, we can obtain the ODBC value for this property as follows:

Terminal

```
USER>write x.IntegerLogicalToOdbc("000056")
56
```

LogicalToXSD()

```
Method samplepropLogicalToXSD(InputValue) as ReturnValue
```

Where available: literal properties and list and array properties containing literal values, if used in a class that extends %XML.Adaptor.

Given a logical input value, this method converts it to an SOAP-encoded value, as appropriate for the property definition.

For example, suppose that `Sample.Person` has a property named Boolean, which is a %Boolean value. Given an instance `p` of the class that contains this property, we can convert 1 to the appropriate SOAP-encoded value, as follows:

Terminal

```
USER>set p=##class(Sample.Person).%OpenId(1)
USER>write p.BooleanLogicalToXSD(1)
true
```

OdbcToCollection()

```
Method samplepropOdbcToCollection(delimitedstring) as ReturnValue
```

Where available: list and array properties.

This method accepts a delimited string of ODBC values and converts it to a serialized collection value.

NewObject()

```
Method samplepropNewObject() as OREF
```

Where available: object-valued properties (except for list and array properties).

Returns an OREF that refers to a new instance of the given class and sets the property equal to this instance. For example:

Terminal

```
USER>set p=##class(Sample.Person).%New()

USER>set address=p.AddressNewObject()

USER>write address
604@Sample.Address
```

The previous is equivalent to the following:

Terminal

```
USER>set p=##class(Sample.Person).%New()

USER>set address=p.Address

USER>set address=##class(Sample.Address).%New()

USER>write address
604@Sample.Address
```

Normalize()

```
Method samplepropNormalize(InputValue) as ReturnValue
```

Where available: literal properties and list and array properties containing literal values.

Given an input value, this method normalizes the value as appropriate for the property definition.

For example, suppose that `Integer` is a property of type integer. Given an instance `x` of the class that contains this property, we can obtain a normalized value for this property as follows:

Terminal

```
USER>write x.IntegerNormalize(000078.0000)
78
```

Set()

```
Method samplepropSet(InputValue) as %Status
```

Where available: literal properties and object-valued properties.

Given an input value, this method sets the property equal to this value. If the property is object-valued, the input value should be an OREF. This method returns a %Status. This method is not used when you access the data via SQL. For example:

```
USER>set test=##class(Sample.Person).%New()

USER>set status=test.NameSet("Sample Name")

USER>write status
1
```

For another example, suppose that `Sample.Person` has a property named `ObjectProp` of type `Sample.OtherClass`:

```
USER>set test=##class(Sample.Person).%New()

USER>set other=##class(Sample.OtherClass).%New()

USER>set status=test.ObjectPropSet(other)

USER>write status
1
```

SetObject()

```
Method samplepropSetObject(Oid) as %Status
```

Where available: object-valued properties.

Given the OID of an object, this method sets the property using that OID. This is similar to **SetObjectId()** except for using the OID instead of the ID. This method is not used when you access the data via SQL.

SetObjectId()

```
Method samplepropSetObjectId(Id) as %Status
```

Where available: object-valued properties.

Given the ID of an object, this method sets the property using that ID. This method is not used when you access the data via SQL. For example, suppose `Sample.Person` has a `Address` property of type `Sample.Address`.

Terminal

```
USER>set test=##class(Sample.Person).%New()

USER>set status=test.AddressSetObjectId(14)
```

This code sets the `Address` property equal to the `Sample.Address` instance whose ID is 14.

XSDToLogical()

```
Method samplepropXSDToLogical(InputValue) as ReturnValue
```

Where available: literal properties and list and array properties containing literal values, if used in a class that extends %XML.Adaptor.

Given a SOAP-encoded input value, this method converts it to an logical value, as appropriate for the property definition.

For example, suppose that `Sample.Person` has a property named Boolean, which is a %Boolean value. Given an instance `p` of the class that contains this property, we can convert `true` to the appropriate logical value, as follows:

Terminal

```
USER>set p=##class(Sample.Person).%OpenId(1)

USER>write p.BooleanXSDToLogical("true")
1
```

10.4 Listing Property Methods

The following sample code uses the %Dictionary API and displays the property methods generated for a given method of a given class:

Class Member

```
ClassMethod DisplayPropertyMethods(className As %String = "", propertyName As %String = "")
{
    Set cdef=##class(%Dictionary.CompiledClass).%OpenId(className)
    Set propertynamelength=$LENGTH(propertyName)
    Set methodlist=cdef.Methods
    #dim item as %Dictionary.CompiledMethod
    For i=1:1:methodlist.Count() {
        Set item=methodlist.GetAt(i)
        Set methodname=item.Name
        Set extract=$extract(methodname,0,propertynamelength)
        // This checks that the method name starts with the property name.
        // There will be false positives when one property name
        // starts with the same characters as another
        If (extract=propertyName) {
            Write !, methodname
        }
    }
}
```

For example:

Terminal

```
USER>do ##class(Tools.Tools).DisplayPropertyMethods("Sample.Person","Test")

TestDisplayToLogical
TestGet
TestGetStored
TestIsValid
TestLogicalToDisplay
TestLogicalToOdbc
TestNormalize
TestSet
```

10.5 See Also

- Defining and Using Properties
- Overriding Property Methods
- Defining Data Type Classes

11

Introduction to Persistent Objects

This topic presents the concepts that are useful to understand when working with persistent classes.

Some of the examples shown here are from the Samples-Data sample (https://github.com/intersystems/Samples-Data). InterSystems recommends that you create a dedicated namespace called SAMPLES (for example) and load samples into that namespace. For the general process, see *Downloading Samples for Use with InterSystems IRIS® data platform*.

11.1 Persistent Classes

A *persistent class* is any class that inherits from %Persistent. A *persistent object* is an instance of such a class.

The %Persistent class is a subclass of %RegisteredObject and thus is an object class. In addition to providing the object interface methods, the %Persistent class defines the *persistence interface*, a set of methods. Among other things, these methods enable you to save objects to the database, load objects from the database, delete objects, and test for existence.

11.2 Features Specific to Persistent Classes

Persistent classes can include several kinds of class members that are not meaningful in other types of classes:

- Storage definitions, which govern access for the data for the class.
- Indices. An index can define the unique identifier for objects in the class; see The Object ID.

  An index can also add a constraint that ensures uniqueness of a given field or combination of fields. For information on such indexes, see Defining Persistent Classes.

  Another purpose of an index is to define a specific sorted subset of commonly requested data associated with a class, so that queries can run more quickly. For example, as a general rule, if a query that includes a WHERE clause using a given field, the query runs more rapidly if that field is indexed. In contrast, if there is no index on that field, the engine must perform a *full table scan*, checking every row to see if it matches the given criteria — an expensive operation if the table is large. See Other Options for Persistent Classes.
- Foreign keys, which establish referential integrity constraints between tables used when data is added or changed. If you use relationship properties, the system automatically treats these as foreign keys. But you can add foreign keys if you do not want to use relationships or if you have other reasons to add them.

  For more information on foreign keys, see Other Options for Persistent Classes.

- Triggers, which define code to be executed automatically when specific events occur, specifically when a record is inserted, modified, or deleted.

  For more information on triggers, see Other Options for Persistent Classes.

Also note that a class method or a class query can be defined so that it can be invoked as a stored procedure, which enables you to invoke it from SQL.

11.3 Introduction to the Default SQL Projection

For any persistent class, the compiler generates an SQL table definition, so that the stored data can be accessed via SQL in addition to via the object interface.

The table contains one record for each saved object, and the table can be queried via InterSystems SQL. The following shows the results of a query of the Sample.Person table:

ID	Age	DOB	FavoriteColors	Name	SSN	Spouse	Home_City	Home_State	Home_Street	Home_Zip	Office_City
1	1	09/19/2013	Green Green	Humby,Michael G.	732-63-4007		Boston	MD	4033 Second Avenue	87184	Vail
2	17	08/26/1997		Jenkins,Usha V.	921-79-1526		Jackson	KS	1156 Washington Place	73402	Tampa
3	16	06/02/1998	White Black	Diavolo,Emma T.	842-30-8977		Larchmont	AL	5794 Elm Street	13434	Miami
4	26	07/09/1988	Yellow Purple	Anderson,Alvin T.	235-56-7318		Albany	ND	4145 Ash Place	57771	Vail
5	27	07/23/1987	Blue	Zemaitis,Christen D.	958-46-8312		Oak Creek	VT	6581 Ash Drive	89034	Gansevoort
6	89	08/09/1925	Red	Adams,Frances Q.	297-20-9962		Chicago	MT	6141 Washington Blvd	42799	Vail
7	78	04/18/1936		Press,Laura F.	586-94-4188		Larchmont	OH	5772 Washington Drive	38850	Islip

The following table summarizes the default projection:

Table 11–1: The Object-SQL Projection

From (Object Concept) ...	To (Relational Concept) ...
Package	Schema
Class	Table
OID	Identity field
Data type property	Field
Reference property	Reference field
Embedded object	Set of fields
List property	List field
Array property	Child table
Stream property	BLOB
Index	Index
Class method	Stored procedure

Other topics provide details and describe any changes you can make:

- For information on the table name and the name of the schema to which it belongs, see Defining Persistent Classes.

  The same topic also has information on how you can control the projection of subclasses.
- For information on the projection of literal properties, see Defining and Using Literal Properties.
- For information on the projection of collection properties, see Storage and SQL Projection of Collection Properties.
- For information on the projection of stream properties, see Working with Streams.
- For information on the projection of object-valued properties, see Defining and Using Object-Valued Properties.
- For information on the projection of relationships, see Defining and Using Relationships.

11.4 The Object ID

When you save an object for the first time, the system creates two permanent identifiers for it: the ID (discussed here) and the OID (discussed in the next section). You can use these identifiers later to access or remove the saved objects.

The object ID is the more commonly used identifier, and you use this ID as the first argument when you call methods such as **%OpenId()**, **%ExistsId()**, and **%DeleteId()**, available in all persistent classes.

11.4.1 Determining the ID

The system can automatically generate an integer to use as the ID. However, it is often useful to have an ID that is more meaningful to your application, so a common practice is to add an IdKey index definition to the persistent class, where this index refers to the property or properties that will provide the ID value. The following example includes an IdKey index that refers to the SSN property:

Class Definition

```
Class MyApp.Person Extends %Persistent
{
Index MainIDX On SSN [ Idkey ];

Property SSN As %String;

// other properties
}
```

In this example, the SSN for a person serves as the ID (and can be used with methods such as **%OpenId()**, **%ExistsId()**, and **%DeleteId()**). For example:

ObjectScript

```
 set person=##class(MyApp.Person).%OpenId("123-45-6789")
```

When you add an IdKey index to a class, the compiler generates additional methods (known as *index methods*) for that class, which you can use to access or remove objects. The index methods include *indexName***Open()**, *indexName***Exists()**, and *indexName***Delete()**, where *indexName* is the name of the IdKey index. When the IdKey index refers to only a single property, these methods accept the value of that property as the first argument. For example:

ObjectScript

```
 set person=##class(MyApp.Person).MainIDXOpen("123-45-6789")
```

You can have an IdKey index that refers to multiple properties, as follows:

Class Definition

```
Class MyApp.Account Extends %Persistent
{
Index MainIDX On (CountryCode,RegionalID) [ Idkey ];

Property CountryCode As %String;

Property RegionalID As %String;

// other properties
}
```

For such a class, the ID has a more complex form; it consists of the property values used by the IdKey index, concatenated with pairs of pipe `||` characters. In this case, it is simpler to use the generated index methods. The expected arguments are the property values used by the IdKey index, in the order used by that index. For example:

ObjectScript

```
 set account=##class(MyApp.Account).MainIDXOpen("US","1234567")
```

11.4.2 Projection of Object IDs to SQL

The ID of an object is available in the corresponding SQL table. If possible, InterSystems IRIS uses the field name `ID`. InterSystems IRIS also provides a way to access the ID if you are not sure what field name to use. The system is as follows:

- An object ID is not a property of the object and is treated differently from the properties.
- If the class does not contain a property named `ID` (in any case variation), then the table also contains the field `ID`, and that field contains the object ID. For an example, see the previous section.
- If the class contains a property that is projected to SQL with the name `ID` (in any case variation), then the table also contains the field `ID1`, and this field holds the value of the object ID.

Similarly, if the class contains properties that are projected as `ID` and `ID1`, then the table also contains the field `ID2`, and this field holds the value of the object ID.

- In all cases, the table also provides the pseudo-field `%ID`, which also holds the value of the object ID.

11.4.3 Object IDs in SQL

InterSystems IRIS enforces uniqueness for the ID field (whatever its actual name might be). InterSystems IRIS also prevents this field from being changed. This means that you cannot perform SQL **UPDATE** or **INSERT** operations on this field. For instance, the following shows the SQL needed to add a new record to a table:

SQL

```
INSERT INTO PERSON (FNAME, LNAME) VALUES (:fname, :lname)
```

Notice that this SQL does not refer to the ID field. InterSystems IRIS generates a value for the ID field and inserts that when it creates the requested record.

11.5 The Object OID

In addition to its ID, each saved object has an additional permanent identifier: its OID. The OID also includes the class name and is unique in the database.

The %Persistent class provides additional methods that use the OID; these include **%Open()**, **%Exists()**, and **%Delete()**. The methods that use OID as the argument do not include `Id` in their names; these methods are used much less often.

When a persistent object is stored in the database, the values of any of its reference attributes (that is, references to other persistent objects) are stored as OID values. For object attributes that do not have OIDs, the literal value of the object is stored along with the rest of the state of the object.

The OID is not available via SQL.

11.6 See Also

- Storage Definitions
- Storage Globals
- Persistent Objects and Extents
- Working with Persistent Objects
- Defining Persistent Classes
- Other Options for Persistent Classes

12
Storage Definitions

In most cases, a persistent class includes a *storage definition*, which describes the global structure that InterSystems IRIS® data platform uses when it saves data for the class or reads saved data for the class.

This page explains the role of the storage definition and of the *storage class* used by the storage definition and explains what happens if the class definition changes.

12.1 Introduction to Storage Definitions

When you define and compile a persistent class, InterSystems IRIS generates a storage definition for it, adding the storage definition to the end of the class definition. The following shows an example storage definition:

```
Storage Default
{
<Data name="PresidentDefaultData">
<Value name="1">
<Value>%%CLASSNAME</Value>
</Value>
<Value name="2">
<Value>Name</Value>
</Value>
<Value name="3">
<Value>BirthYear</Value>
</Value>
<Value name="4">
<Value>Bio</Value>
</Value>
</Data>
<DataLocation>^GlobalsTest.PresidentD</DataLocation>
<DefaultData>PresidentDefaultData</DefaultData>
<IdLocation>^GlobalsTest.PresidentD</IdLocation>
<IndexLocation>^GlobalsTest.PresidentI</IndexLocation>
<StreamLocation>^GlobalsTest.PresidentS</StreamLocation>
<Type>%Storage.Persistent</Type>
}
```

Notice the `<Type>` element, which specifies the storage class (%Storage.Persistent in this example).

Another page more closely examines the globals referred to by the storage definition; for now, simply notice that the storage definition describes how the globals are organized (for example, which subscript contains data for each property).

Note: A persistent class can actually contain multiple storage definitions but only one can be active at a time. The active storage definition is specified using the StorageStrategy keyword of the class, not described here. By default, a persistent class has a single storage definition called Default.

12.2 Storage Classes

The %Persistent class provides the high-level interface for storing and retrieving objects in the database. The actual work of storing and loading objects is governed by a *storage class*. As shown in the previous example, a storage definition (part of the class definition) specifies the storage class to use.

The storage class generates the actual internal methods used to store, load, and delete objects in a database. These internal methods are the *storage interface*, which includes internal methods such as **%LoadData()**, **%SaveData()**, and **%DeleteData()**. Applications never call these methods directly; instead they are called at the appropriate time by the methods of the persistence interface (such as **%OpenId()** and **%Save()**).

There are two storage classes:

- %Storage.Persistent, which is the default storage class used by persistent objects. It automatically creates and maintains a default storage structure for a persistent class.

  New persistent classes automatically use the %Storage.Persistent storage class. The %Storage.Persistent class lets you control certain aspects of the storage structure used for a class by means of the various keywords in the storage definition.

  Refer to the Class Definition Reference for details on the various storage keywords.

  Also see Extent Definitions for information on the *MANAGEDEXTENT* class parameter.

- %Storage.SQL, which is a special storage class that uses generated SQL SELECT, INSERT, UPDATE, and DELETE statements to provide object persistence. %Storage.SQL is typically used for:

  – Mapping objects to preexisting global structures used by older applications.

  – Storing objects within an external relational database using the SQL Gateway.

  %Storage.SQL does not automatically support schema evolution or multi-class extents.

12.3 Schema Evolution

The storage definition for a class is generated when the class is first compiled. When you compile a persistent (or serial) class that uses the default %Storage.Persistent storage class, the class compiler analyzes the properties defined by the class and automatically uses them in the storage definition.

Similarly, when you add properties, the class compiler updates the storage definition to provide access to these properties. When you remove properties, their descriptions within the storage definition remain unchanged.

This automatic process is *schema evolution*, and the goal is to safely preserve access to your data. Schema evolution does not apply to classes that use %Storage.SQL.

12.4 Resetting a Storage Definition

During the development process, you are likely to need to reset the storage definition of your classes, for reasons such as these:

- You may make many modifications to your persistent classes: adding, modifying, and deleting properties. As a result, you may end up with a fairly convoluted storage definition as the class compiler attempts to maintain a compatible structure.
- Also, class projection, such as for SQL, occurs after compilation. If a class compiles properly and then projection fails, InterSystems IRIS does not remove the storage definition.

To reset a storage definition, delete the storage definition from the class and then recompile the class.

Important: Also delete all sample data for the class. Or if there is existing data you need to keep, you must migrate it to the new global structure.

12.5 Redefining a Persistent Class That Has Stored Data

During the development process, it is common to redefine your classes. If you have already created sample data for the class, note the following points:

- The compiler has no effect on the globals that store the data for the class.

  (In fact, when you delete a class definition, its data globals are untouched. If you no longer need these globals, delete them manually.)
- If you add or remove properties of a class but do not modify the storage definition of the class, then all code that accesses data for that class continues to work as before. See Schema Evolution.
- If you do modify the storage definition of the class, then code that accesses the data may or may not continue to work as before, depending on the nature of the change.
- If you change the class from non-sharded to sharded or vice-versa, your existing data may become inaccessible.
- If you modify a property definition in a way that causes the property validation to be more restrictive, then you will receive errors when you work with objects (or records) that no longer pass validation. For example, if you decrease the *MAXLEN* parameter for a property, then you will receive validation errors when you work with an object that has a value for that property that is now too long.

12.6 See Also

- Introduction to Persistent Objects
- Persistent Objects and Storage Globals

13

Persistent Objects and Storage Globals

Persistent classes allow you to save objects to the database and retrieve them as objects or via SQL. Regardless of how they are accessed, these objects are stored in globals, which can be thought of as persistent multidimensional arrays.

This page describes how the global names are determined and how you can control them. It also looks at the contents of the globals, for illustration purposes. The best way to ensure data consistency is to access the data via objects or SQL.

So that you can be aware of the variations in storage globals, this page also introduces columnar storage, which has performance benefits in some scenarios; column storage is also discussed more fully in Choose an SQL Table Storage Layout.

13.1 Standard Global Names

For a persistent class, when you create and compile a class definition, global names for the class are generated based on the class name.

For example, let’s define the following class, GlobalsTest.President:

Class Definition

```
Class GlobalsTest.President Extends %Persistent
{

/// President's name (last,first)
Property Name As %String(PATTERN="1U.L1"",""1U.L");

/// Year of birth
Property BirthYear As %Integer;

/// Short biography
Property Bio As %Stream.GlobalCharacter;

/// Index for Name
Index NameIndex On Name;

/// Index for BirthYear
Index DOBIndex On BirthYear;

}
```

After compiling the class, we can see the following storage definition generated at the bottom of the class:

Class Member

```
Storage Default
{
<Data name="PresidentDefaultData">
<Value name="1">
<Value>%%CLASSNAME</Value>
```

```
</Value>
<Value name="2">
<Value>Name</Value>
</Value>
<Value name="3">
<Value>BirthYear</Value>
</Value>
<Value name="4">
<Value>Bio</Value>
</Value>
</Data>
<DataLocation>^GlobalsTest.PresidentD</DataLocation>
<DefaultData>PresidentDefaultData</DefaultData>
<IdLocation>^GlobalsTest.PresidentD</IdLocation>
<IndexLocation>^GlobalsTest.PresidentI</IndexLocation>
<StreamLocation>^GlobalsTest.PresidentS</StreamLocation>
<Type>%Storage.Persistent</Type>
}
```

Notice, in particular, the following storage keywords:

- The `DataLocation` is the global where class data will be stored. The name of the global is the complete class name (including the package name) with a “D” appended to the name, in this case, `^GlobalsTest.PresidentD`.
- The `IdLocation` (often the same as the `DataLocation`) is the global where the ID counter will be stored, at its root.
- The `IndexLocation` is the global where the indexes for the class will be stored. The name of the global is the complete class name with an “I” appended to the name, or, `^GlobalsTest.PresidentI`.
- The `StreamLocation` is the global where any stream properties will be stored. The name of the global is the complete class name with an “S” appended to the name, or, `^GlobalsTest.PresidentS`.

After creating and storing a few objects of our class, we can view the contents of these globals in the Terminal:

```
USER>zwrite ^GlobalsTest.PresidentD
^GlobalsTest.PresidentD=3
^GlobalsTest.PresidentD(1)=$lb("",1732,"1","Washington,George")
^GlobalsTest.PresidentD(2)=$lb("",1735,"2","Adams,John")
^GlobalsTest.PresidentD(3)=$lb("",1743,"3","Jefferson,Thomas")

USER>zwrite ^GlobalsTest.PresidentI
^GlobalsTest.PresidentI("DOBIndex",1732,1)=""
^GlobalsTest.PresidentI("DOBIndex",1735,2)=""
^GlobalsTest.PresidentI("DOBIndex",1743,3)=""
^GlobalsTest.PresidentI("NameIndex"," ADAMS,JOHN",2)=""
^GlobalsTest.PresidentI("NameIndex"," JEFFERSON,THOMAS",3)=""
^GlobalsTest.PresidentI("NameIndex"," WASHINGTON,GEORGE",1)=""

USER>zwrite ^GlobalsTest.PresidentS
^GlobalsTest.PresidentS=3
^GlobalsTest.PresidentS(1)="1,239"
^GlobalsTest.PresidentS(1,1)="George Washington was born to a moderately prosperous family of planters
 in colonial ..."
^GlobalsTest.PresidentS(2)="1,195"
^GlobalsTest.PresidentS(2,1)="John Adams was born in Braintree, Massachusetts, and entered Harvard
College at age 1..."
^GlobalsTest.PresidentS(3)="1,202"
^GlobalsTest.PresidentS(3,1)="Thomas Jefferson was born in the colony of Virginia and attended the
College of Willi..."
```

The subscript of `^GlobalsTest.PresidentD` is the IDKey. Since we did not define one of our indexes as the IDKey, the ID is used as the IDKey. For more information on IDs, see Controlling How IDs Are Generated.

The first subscript of `^GlobalsTest.PresidentI` is the name of the index.

The first subscript of `^GlobalsTest.PresidentS` is the ID of the bio entry, not the ID of the president.

You can also view these globals in the Management Portal (**System Explorer** > **Globals**).

Important: Only the first 31 characters in a global name are significant, so if a complete class name is very long, you might see global names like `^package1.pC347.VeryLongCla4F4AD`. The system generates names such as these to ensure that all of the global names for your class are unique. If you plan to work directly with the globals of a class, make sure to examine the storage definition so that you know the actual name of the global. Alternatively, you can control the global names by using the *DEFAULTGLOBAL* parameter in your class definition. See User-Defined Global Names.

13.2 Hashed Global Names

The system will generate shorter global names if you set the *USEEXTENTSET* parameter to the value 1. (The default value for this parameter is 0, meaning use the standard global names.) These shorter global names are created from a hash of the package name and a hash of the class name, followed by a suffix. While the standard names are more readable, the shorter names can contribute to better performance.

When you set *USEEXTENTSET* to 1, each index is also assigned to a separate global, instead of using a single index global with different first subscripts. Again, this is done for increased performance.

To use hashed global names for the GlobalsTest.President class we defined earlier, we would add the following to the class definition:

```
/// Use hashed global names
Parameter USEEXTENTSET = 1;
```

After deleting the data, deleting the storage definition, and recompiling the class, we can see the new storage definition with hashed global names:

```
Storage Default
{
...
<DataLocation>^Ebnm.EKUy.1</DataLocation>
<DefaultData>PresidentDefaultData</DefaultData>
<ExtentLocation>^Ebnm.EKUy</ExtentLocation>
<IdLocation>^Ebnm.EKUy.1</IdLocation>
<Index name="DOBIndex">
<Location>^Ebnm.EKUy.2</Location>
</Index>
<Index name="IDKEY">
<Location>^Ebnm.EKUy.1</Location>
</Index>
<Index name="NameIndex">
<Location>^Ebnm.EKUy.3</Location>
</Index>
<IndexLocation>^Ebnm.EKUy.I</IndexLocation>
<StreamLocation>^Ebnm.EKUy.S</StreamLocation>
<Type>%Storage.Persistent</Type>
}
```

Notice, in particular, the following storage keywords:

- The `ExtentLocation` is the hashed value that will be used to calculate global names for this class, in this case, `^Ebnm.EKUy`.
- The `DataLocation` (equivalent to the `IDKEY` index), where class data will be stored, is now the hashed value with a ".1" appended to the name, in this case, `^Ebnm.EKUy.1`.
- Each index now has its own `Location` and thus its own separate global. The name of the IdKey index global is equivalent to the hashed value with a ".1" appended to the name, in this example, `^Ebnm.EKUy.1`. The globals for the remaining indexes have ".2" to ".N" appended to the name. Here, the DOBIndex is stored in global `^Ebnm.EKUy.2` and the NameIndex is stored in `^Ebnm.EKUy.3`.

- The `IndexLocation` is the hashed value with “.I” appended to the name, or `^Ebnm.EKUy.I`, however, this global is often not used.
- The `StreamLocation` is the hashed value with “.S” appended to the name, or `^Ebnm.EKUy.S`.

After creating and storing a few objects, the contents of these globals might look as follows, again using the Terminal:

```
USER>zwrite ^Ebnm.EKUy.1
^Ebnm.EKUy.1=3
^Ebnm.EKUy.1(1)=$lb("","Washington,George",1732,"1")
^Ebnm.EKUy.1(2)=$lb("","Adams,John",1735,"2")
^Ebnm.EKUy.1(3)=$lb("","Jefferson,Thomas",1743,"3")

USER>zwrite ^Ebnm.EKUy.2
^Ebnm.EKUy.2(1732,1)=""
^Ebnm.EKUy.2(1735,2)=""
^Ebnm.EKUy.2(1743,3)=""

USER>zwrite ^Ebnm.EKUy.3
^Ebnm.EKUy.3(" ADAMS,JOHN",2)=""
^Ebnm.EKUy.3(" JEFFERSON,THOMAS",3)=""
^Ebnm.EKUy.3(" WASHINGTON,GEORGE",1)=""

USER>zwrite ^Ebnm.EKUy.S
^Ebnm.EKUy.S=3
^Ebnm.EKUy.S(1)="1,239"
^Ebnm.EKUy.S(1,1)="George Washington was born to a moderately prosperous family of planters in colonial
 ..."
^Ebnm.EKUy.S(2)="1,195"
^Ebnm.EKUy.S(2,1)="John Adams was born in Braintree, Massachusetts, and entered Harvard College at age
 1..."
^Ebnm.EKUy.S(3)="1,202"
^Ebnm.EKUy.S(3,1)="Thomas Jefferson was born in the colony of Virginia and attended the College of
Willi..."
```

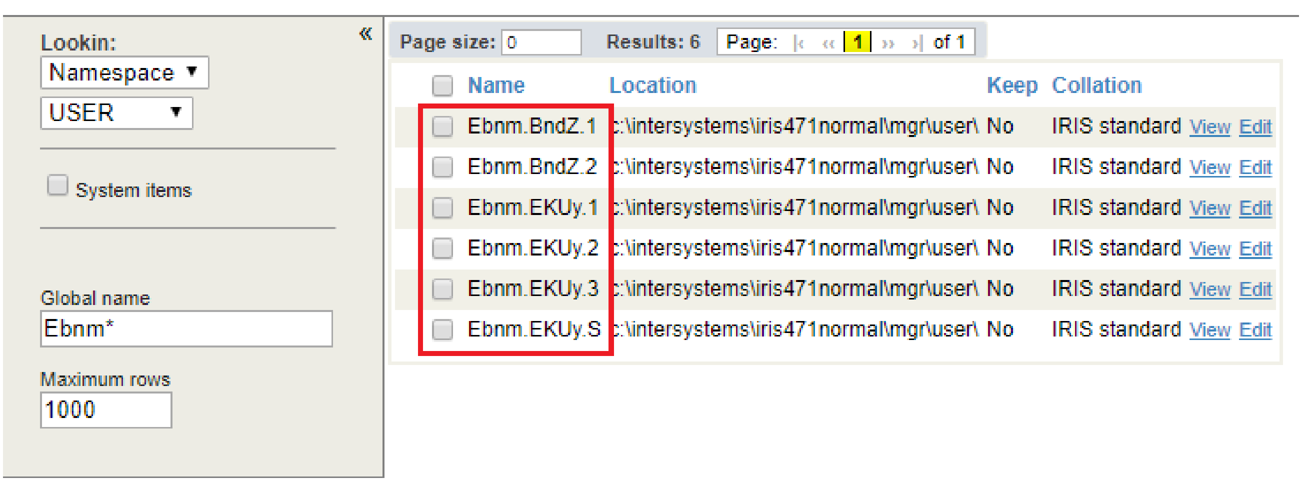
For classes defined using an SQL **CREATE TABLE** statement, the default for the `USEEXTENTSET` parameter is 1. For more information on creating tables, see Defining Tables.

For example, let’s create a table using the Management Portal (**System Explorer** > **SQL** > **Execute Query**):

SQL

```
CREATE TABLE GlobalsTest.State (NAME CHAR (30) NOT NULL, ADMITYEAR INT)
```

After populating the table with some data, we see the globals `^Ebnm.BndZ.1` and `^Ebnm.BndZ.2` in the Management Portal (**System Explorer** > **Globals**). Notice that the package name is still GlobalsTest, so the first segment of the global names for GlobalsTest.State is the same as for GlobalsTest.President.



Using the Terminal, the contents of the globals might look like:

```
USER>zwrite ^Ebnm.BndZ.1
^Ebnm.BndZ.1=3
^Ebnm.BndZ.1(1)=$lb("Delaware",1787)
^Ebnm.BndZ.1(2)=$lb("Pennsylvania",1787)
^Ebnm.BndZ.1(3)=$lb("New Jersey",1787)

USER>zwrite ^Ebnm.BndZ.2
^Ebnm.BndZ.2(1)=$zwc(412,1,0)/*$bit(2..4)*/
```

The global `^Ebnm.BndZ.1` contains the States data and `^Ebnm.BndZ.2` is a bitmap extent index. See Bitmap Extent Index.

If we wanted to use standard global names with a class created via SQL, we could set the `USEEXTENTSET` parameter to the value 0:

SQL

```
CREATE TABLE GlobalsTest.State (%CLASSPARAMETER USEEXTENTSET 0, NAME CHAR (30) NOT NULL, ADMITYEAR INT)
```

This would generate the standard global names `^GlobalsTest.StateD` and `^GlobalsTest.StateI`.

Note: You can change the default value used for the `USEEXTENTSET` parameter to 0 for classes created via a **CREATE TABLE** statement by executing the command `do $SYSTEM.SQL.SetDDLUseExtentSet(0, .oldval)`. The previous default value is returned in *oldval*.

For classes created via XEP, the default for the `USEEXTENTSET` parameter is 1 and may not be changed. You can read more about XEP in *Persisting Java Objects with InterSystems XEP*.

13.3 User-Defined Global Names

For finer control of the global names for a class, use the *DEFAULTGLOBAL* parameter. This parameter works in conjunction with the *USEEXTENTSET* parameter to determine the global naming scheme.

For example, let’s add the *DEFAULTGLOBAL* parameter to set the root of the global names for the GlobalsTest.President class to `^GT.Pres`:

```
/// Use hashed global names
Parameter USEEXTENTSET = 1;

/// Set the root of the global names
Parameter DEFAULTGLOBAL = "^GT.Pres";
```

After deleting the data, deleting the storage definition, and recompiling the class, we can see the following global names:

```
Storage Default
{
...
<DataLocation>^GT.Pres.1</DataLocation>
<DefaultData>PresidentDefaultData</DefaultData>
<ExtentLocation>^GT.Pres</ExtentLocation>
<IdLocation>^GT.Pres.1</IdLocation>
<Index name="DOBIndex">
<Location>^GT.Pres.2</Location>
</Index>
<Index name="IDKEY">
<Location>^GT.Pres.1</Location>
</Index>
<Index name="NameIndex">
<Location>^GT.Pres.3</Location>
</Index>
<IndexLocation>^GT.Pres.I</IndexLocation>
<StreamLocation>^GT.Pres.S</StreamLocation>
<Type>%Storage.Persistent</Type>
}
```

Likewise, we can use the *DEFAULTGLOBAL* parameter when defining a class using SQL:

SQL

```
CREATE TABLE GlobalsTest.State (%CLASSPARAMETER USEEXTENTSET 0, %CLASSPARAMETER DEFAULTGLOBAL =
'^GT.State',
NAME CHAR (30) NOT NULL, ADMITYEAR INT)
```

This would generate the global names `^GT.StateD` and `^GT.StateI`.

13.4 Redefining Global Names

If you edit a class definition in a way that redefines the previously existing global names, for example, by changing the values of the *USEEXTENTSET* or *DEFAULTGLOBAL* parameters, you must delete the existing storage definition to allow the compiler to generate a new storage definition. Note that any data in the existing globals is preserved. Any data to be retained must be migrated to the new global structure.

For more information, see Redefining a Persistent Class That Has Stored Data.

13.5 Using Columnar Storage

When you use persistent classes to store data in InterSystems IRIS, the data is typically stored in rows. This storage layout is appropriate in cases where you are doing online transaction processing, with frequent inserts, updates, and deletes. However, storing data in columns can be more appropriate for online analytical processing, where you are aggregating data in specific columns across the database, and real-time inserts, updates, and deletes are less common. For example, using when row storage, calculating the average of all values for a given property requires you to load all of the rows in the database. Columnar storage makes that computation faster by letting you load only the column that contains the values for that property.

To use columnar storage for a class:

- Specify the *STORAGEDEFAULT* class parameter as `"columnar"`:

  ### Class Member

  ```
  Parameter STORAGEDEFAULT = "columnar";
  ```

- For the same class, specify either the `Final` class keyword or the `NoExtent` class keyword, with any immediate subclasses defined explicitly as `Final`.

If we make these changes to the example class, delete the data for that class, delete the storage definition for the class and then recompile, the resulting storage definition is as follows:

When you are using transaction-based processing, but you have a few properties you frequently perform analytical queries on, you can use columnar storage for just those properties. To use columnar storage for a single property in a class, specify the *STORAGEDEFAULT* property parameter as `"columnar"`:

Class Member

```
Property Amount As %Numeric(STORAGEDEFAULT = "columnar");
```

You can use also use row storage for a class and create a columnar index for a property that is frequently queried.

To create a columnar index on a property in a class, use the keyword `type = columnar` on that index:

```
Class Sample.BankTransaction Extends %Persistent [ DdlAllowed ]
{

/// Line below is optional
Parameter STORAGEDEFAULT = "row";

Property Amount As %Numeric(SCALE = 2);

Index AmountIndex On Amount [ type = columnar ];

//other class members...

}
```

For more information on indexes, see Adding Indexes.

For more information on columnar storage, see Choose an SQL Table Storage Layout.

13.6 See Also

- Introduction to Persistent Objects
- Storage Definitions
- Using Multidimensional Storage (Globals)
- Choose an SQL Table Storage Layout
- Redefining a Persistent Class That Has Stored Data

14

Persistent Objects and Extents

The term *extent* refers to all the records, on disk, for a given persistent class. The %Persistent class provides several methods that operate on the extent of class.

14.1 Introduction to Extents

InterSystems IRIS® data platform uses an unconventional and powerful interpretation of the object-table mapping. By default, the extent of a given persistent class includes the extents of any subclasses. Therefore:

- If the persistent class Person has the subclass Employee, the Person extent includes all instances of Person and all instances of Employee.
- For any given instance of class Employee, that instance is included in the Person extent and in the Employee extent.

Indices automatically span the entire extent of the class in which they are defined. The indexes defined in Person contain both Person instances and Employee instances. Indexes defined in the Employee extent contain only Employee instances.

The subclass can define additional properties not defined in its superclass. These are available in the extent of the subclass, but not in the extent of the superclass. For example, the Employee extent might include the Department field, which is not included in the Person extent.

The preceding points mean that it is comparatively easy in InterSystems IRIS to write a query that retrieves all records of the same type. For example, if you want to count people of all types, you can run a query against the Person table. If you want to count only employees, run the same query against the Employee table.

Similarly, the methods that use the ID all behave polymorphically. That is, they can operate on different types of objects depending on the ID value it is passed.

For example, the extent for Sample.Person objects includes instances of Sample.Person as well as instances of Sample.Employee. When you call **%OpenId()** for the Sample.Person class, the resulting OREF could be an instance of either Sample.Person or Sample.Employee, depending on what is stored within the database:

ObjectScript

```
// Open person "10"
Set obj = ##class(Sample.Person).%OpenId(10)

Write $ClassName(obj),!    // Sample.Person


// Open person "110"
Set obj = ##class(Sample.Person).%OpenId(110)

Write $ClassName(obj),!    // Sample.Employee
```

Note that the **%OpenId()** method for the Sample.Employee class will *not* return an object if we try to open ID `10`, because the ID 10 is not the Sample.Employee extent:

ObjectScript

```
// Open employee "10"
Set obj = ##class(Sample.Employee).%OpenId(10)

Write $IsObject(obj),!  // 0

// Open employee "110"
Set obj = ##class(Sample.Employee).%OpenId(110)

Write $IsObject(obj),!  // 1
```

14.2 Extent Definitions

For classes that use the default storage class (%Storage.Persistent), InterSystems IRIS maintains extent definitions and globals that those extents have registered for use with its Extent Manager. The interface to the Extent Manager is through the %ExtentMgr.Util class. This registration process occurs during class compilation.

If there are any errors or name conflicts, these cause the compile to fail. For example:

```
ERROR #5564: Storage reference: '^This.App.Global used in 'User.ClassA.cls'
is already registered for use by 'User.ClassB.cls'
```

For compilation to succeed, resolve the conflicts; this usually involves either changing the name of the index or adding explicit storage locations for the data.

Note: If an application has multiple classes intentionally sharing a global reference, specify that *MANAGEDEXTENT* equals `0` for all the relevant classes, if they use default storage.

The *MANAGEDEXTENT* class parameter has a default value of `1`; this value causes global name registration and a conflicting use check. A value of `0` specifies that there is neither registration nor conflict checking.

Another possibility is that InterSystems IRIS contains obsolete extent definitions. For example, deleting a class does not delete the extent definition by default.

Continuing with the example above, perhaps User.ClassA.cls was deleted, but its extent definition remains. Here, the solution is to delete the obsolete extent definition with the following command:

```
set st = ##class(%ExtentMgr.Util).DeleteExtentDefinition("User.ClassA", "cls")
```

Now, User.ClassB.cls can be compiled without a conflict.

To delete a class and its extent definition, use one of the following calls:

- `$SYSTEM.OBJ.Delete(`*`classname`*`,`*`qspec`*`)` where *classname* is the class to delete and *qspec* includes the flag `e` or the qualifier `/deleteextent`.
- `$SYSTEM.OBJ.DeletePackage(`*`packagename`*`,`*`qspec`*`)` where *packagename* is the package to delete and *qspec* includes the flag `e` or the qualifier `/deleteextent`.
- `$SYSTEM.OBJ.DeleteAll(`*`qspec`*`)` where *qspec* includes the flag `e` or the qualifier `/deleteextent`. This deletes all classes in the namespace.

These calls are methods of the %SYSTEM.OBJ class.

14.3 Extent Indexes

An extent index is an index of all extents and subextents defined in the current namespace. The class compiler maintains this index for locally compiled classes that use the default storage class (%Storage.Persistent). For these classes, when package mappings, routine mappings, or global mappings are changed for the namespace, InterSystems IRIS automatically rebuilds this index.

However, classes mapped from other namespaces are not automatically added to or removed from the index when changes to the class runtime occur in the original namespace. Instead, such changes must be updated in the local namespace's extent index by using one of following calls:

- `$SYSTEM.OBJ.RebuildExtentIndex(`*`updatemode`*`, `*`lockmode`*`)` rebuilds the entire extent index for a given namespace and returns a status value indicating success or failure. If *updatemode* is true then the index is not purged and only detected differences are updated in the index. If *updatemode* is false then the index is purged and rebuilt entirely. A *lockmode* of `1` causes an exclusive lock to be taken out on the entire index structure, a value of `2` causes individual locks to be taken out on class nodes only for the duration of indexing that class, and a value of `3` causes a shared lock to be taken out on the entire index structure. If the requested locks cannot be obtained then an error is reported to the caller. A *lockmode* of `0` indicates no locking.
- `$SYSTEM.OBJ.RebuildExtentIndexOne(`*`classname`*`, `*`lockmode`*`)` rebuilds the extent index for a single class and returns a status value indicating success or failure. This method updates the index nodes in the extent index for the class indicated by *classname*. The values for *lockmode* are the same as for **RebuildExtentIndex()**, but values of `1`, `2`, and `3` are interpreted as locks on the individual class nodes.

These calls are methods of the %SYSTEM.OBJ class.

14.4 Extent Queries

Every persistent class automatically includes a class query called `"Extent"` that provides a set of all the IDs in the extent.

For general information on using class queries, see Defining and Using Class Queries. The following example uses a class query to display all the IDs for the Sample.Person class:

ObjectScript

```
 set query = ##class(%SQL.Statement).%New()
 set qStatus = query.%PrepareClassQuery("Sample.Person","Extent")
 if $$$ISERR(qStatus) {write "%Prepare failed:" do $SYSTEM.Status.DisplayError(qStatus) quit}

 set rset=query.%Execute()
 if (rset.%SQLCODE '= 0) {write "%Execute failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message
quit}

 while rset.%Next()
 {
   write rset.%Get("ID"),!
 }
 if (rset.%SQLCODE < 0) {write "%Next failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message
quit}
```

The Sample.Person extent includes all instances of Sample.Person as well as its subclasses. For an explanation of this, see Defining Persistent Classes.

The `"Extent"` query is equivalent to the following SQL query:

SQL

```
SELECT %ID FROM Sample.Person
```

Note that you cannot rely on the order in which ID values are returned using either of these methods: InterSystems IRIS may determine that it is more efficient to use an index that is ordered using some other property value to satisfy this request. You can add an ORDER BY %ID clause to the SQL query if you need to.

14.5 See Also

- Defining Classes
- Introduction to Persistent Objects
- Working with Persistent Objects

15
Working with Persistent Objects

The %Persistent class is the API for objects that can be saved (written to disk). This topic describes how to use this API. Information in this topic applies to all subclasses of %Persistent.

Most of the samples shown in this topic are from the Samples-Data sample (https://github.com/intersystems/Samples-Data). InterSystems recommends that you create a dedicated namespace called SAMPLES (for example) and load samples into that namespace. For the general process, see Downloading Samples for Use with InterSystems IRIS® Data Platform.

15.1 Saving Objects

To save an object to the database, invoke its **%Save()** method. For example:

ObjectScript

```
Set obj = ##class(MyApp.MyClass).%New()
Set obj.MyValue = 10

Set sc = obj.%Save()
```

The **%Save()** method returns a %Status value that indicates whether the save operation succeeded or failed. Failure could occur, for example, if an object has invalid property values or violates a uniqueness constraint; see Validating Objects.

Calling **%Save()** on an object automatically saves all modified objects that can be reached from the object being saved: that is, all embedded objects, collections, streams, referenced objects, and relationships involving the object are automatically saved if needed. The entire save operation is carried out as one transaction: if any object fails to save, the entire transaction fails and rolls back (no changes are made to disk; all in-memory object values are what they were before calling **%Save()**).

When an object is saved for the first time, the default behavior is for the **%Save()** method to automatically assign it an object ID value that is used to later find the object within the database. In the default case, the ID is generated using the $Increment function; alternately, the class can use a user-provided object ID based on property values that have an idkey index (and, in this case, the property values cannot include the string ||) . Once assigned, you cannot alter the object ID value for a specific object instance (even if it is a user-provided ID).

You can find the object ID value for an object that has been saved using the **%Id()** method:

ObjectScript

```
// Open person "22"
Set person = ##class(Sample.Person).%OpenId(22)
Write "Object ID: ",person.%Id(),!
```

In more detail, the **%Save()** method does the following:

1. First it constructs a temporary structure known as a SaveSet. The SaveSet is simply a graph containing references to every object that is reachable from the object being saved. (Generally, when an object class A has a property whose value is another object class B, an instance of A can reach an instance of B.) The purpose of the SaveSet is to make sure that save operations involving complex sets of related objects are handled as efficiently as possible. The SaveSet also resolves any save order dependencies among objects.

   As each object is added to the SaveSet, its **%OnAddToSaveSet()** callback method is called, if present.

2. It then visits each object in the SaveSet in order and checks if they are modified (that is, if any of their property values have been modified since the object was opened or last saved). If an object has been modified, it will then be saved.

3. Before being saved, each modified object is validated (its property values are tested; its **%OnValidateObject()** method, if present, is called; and then its uniqueness constraints are tested); if the object is valid, the save proceeds. If any object is invalid, then the call to **%Save()** fails and the current transaction is rolled back.

4. Before and after saving each object, the **%OnBeforeSave()** and **%OnAfterSave()** callback methods are called, if present. Similarly, any appropriately defined triggers are called as well.

   These callbacks and any triggers are passed an *Insert* argument, which indicates whether an object is being inserted (saved for the first time) or updated.

   If one of these callback methods or triggers fails (returns an error code) then the call to **%Save()** fails and the current transaction is rolled back.

5. Lastly, the **%OnSaveFinally()** callback method is called, if present.

   This callback method is called after the transaction has completed and the status of the save has been finalized.

If the current object is not modified, then **%Save()** does not write it to disk; it returns success because the object did not need to be saved and, therefore, there is no way that there could have been a failure to save it. In fact, the return value of **%Save()** indicates that the save operation either did all that it was asked or it was unable to do as it was asked (and not specifically whether or not anything was written to disk).

Important: In a multi-process environment, be sure to use proper concurrency controls; see Specifying Concurrency Options.

15.1.1 Rollback

The **%Save()** method automatically saves all the objects in its SaveSet as a single transaction. If any of these objects fail to save, then the entire transaction is rolled back. In this rollback, InterSystems IRIS does the following:

1. It reverts assigned IDs.
2. It recovers removed IDs.
3. It recovers modified bits.
4. It invokes the **%OnRollBack()** callback method, if implemented, for any object that had been successfully serialized.

   InterSystems IRIS does not invoke this method for an object that has not been successfully serialized, that is, an object that is not valid.

15.1.2 Saving Objects and Transactions

As noted previously, the **%Save()** method automatically saves all the objects in its SaveSet as a single transaction. If any of these objects fail to save, then the entire transaction is rolled back.

If you wish to save two or more *unrelated* objects as a single transaction, however, you must enclose the calls to **%Save()** within an explicit transaction: that is, you must start the transaction using the TSTART command and end it with the TCOMMIT command.

For example:

ObjectScript

```
// start a transaction
TSTART

// save first object
Set sc = obj1.%Save()

// save second object (if first was save)
If ($$$ISOK(sc)) {
    Set sc = obj2.%Save()
}

// if both saves are ok, commit the transaction
If ($$$ISOK(sc)) {
    TCOMMIT
}
```

There are two things to note about this example:

1. The **%Save()** method knows if it is being called within an enclosing transaction (because the system variable, $TLEVEL, will be greater than 0).
2. If any of the **%Save()** methods within the transaction fails, the *entire* transaction is rolled back (the TROLLBACK command is invoked). This means that an application must test every call to **%Save()** within a explicit transaction and if one fails, skip calling **%Save()** on the other objects and skip invoking the final **TCOMMIT** command.

15.2 Testing the Existence of Saved Objects

There are two basic ways to test if a specific object instance is stored within the database:

- Using ObjectScript
- Using SQL

In these simple examples, the ID is an integer, which is how InterSystems IRIS generates IDs by default; also see The Object ID for information on other options.

15.2.1 Testing for Object Existence with ObjectScript

The **%ExistsId()** class method checks a specified ID; it returns a true value (1) if the specified object is present in the database and false (0) otherwise. It is available to all classes that inherit from %Persistent. For example:

ObjectScript

```
Write ##class(Sample.Person).%ExistsId(1),!   // should be 1
Write ##class(Sample.Person).%ExistsId(-1),!  // should be 0
```

Here, the first line should return 1 because Sample.Person inherits from %Persistent and the SAMPLES database provides data for this class.

You can also use the **%Exists()** method, which requires an OID rather than an ID. An OID is a permanent identifier unique to the database that includes both the ID and the class name of the object. For more details, see Identifiers for Saved Objects: ID and OID.

15.2.2 Testing for Object Existence with SQL

To test for the existence of a saved object with SQL, use a SELECT statement that selects a row whose %ID field matches the given ID. (The identity property of a saved object is projected as the %ID field.)

For example, using embedded SQL:

ObjectScript

```
 &sql(SELECT %ID FROM Sample.Person WHERE %ID = '1')
 Write SQLCODE,!  // should be 0: success

 &sql(SELECT %ID FROM Sample.Person WHERE %ID = '-1')
 Write SQLCODE,!  // should be 100: not found
```

Here, the first case should result in an SQLCODE of 0 (meaning success) because Sample.Person inherits from %Persistent and the SAMPLES database provides data for this class.

The second case should result in an *SQLCODE* of 100, which means that the statement successfully executed but returned no data. This is expected because the system never automatically generates an ID value less than zero.

For more information on embedded SQL, see Embedded SQL. For more information on *SQLCODE*, see SQLCODE Values and Error Messages in the *InterSystems IRIS Error Reference*.

15.3 Opening Saved Objects

To open an object (load an object instance from disk into memory), use the **%OpenId()** method, which is as follows:

```
classmethod %OpenId(id As %String,
                    concurrency As %Integer = -1,
                    ByRef sc As %Status = $$$OK) as %ObjectHandle
```

Where:

- *id* is the ID of the object to open.

  In these examples, the ID is an integer. See The Object ID for information on other options
- *concurrency* is the concurrency level (locking) used to open the object.
- *sc*, which is passed by reference, is a %Status value that indicates whether the call succeeded or failed.

The method returns an OREF if it can open the given object. It returns a null value ("") if it cannot find or otherwise open the object.

For example:

ObjectScript

```
 // Open person "10"
 Set person = ##class(Sample.Person).%OpenId(10)

 Write "Person: ",person,!    // should be an object reference

 // Open person "-10"
 Set person = ##class(Sample.Person).%OpenId(-10)

 Write "Person: ",person,!    // should be a null string
```

You can also use the **%Open()** method, which requires an OID rather than an ID. An OID is a permanent identifier unique to the database that includes both the ID and the class name of the object. For more details, see Identifiers for Saved Objects: ID and OID.

To perform additional processing that executes when an object opens, you can define the **%OnOpen()** and **%OnOpenFinally()** callback methods. For more details, see Defining Callback Methods.

15.3.1 Multiple Calls to %OpenId()

If **%OpenId()** is called multiple times within an InterSystems IRIS process for the same ID and the same object, only one object instance is created in memory. All subsequent calls to **%OpenId()** return a reference to the object already loaded into memory.

For example, consider this class featuring multiple **%OpenId()** calls to the same object:

Class Definition

```
Class User.TestOpenId Extends %RegisteredObject
{

ClassMethod Main()
{
   set A = ##class(Sample.Person).%OpenId(1)
   write "A: ", A.Name

   set A.Name = "David"
   write !, "A after set: ", A.Name

   set B = ##class(Sample.Person).%OpenId(1)
   write !, "B: ", B.Name

   do ..Sub()

   job ..JobSub()
   hang 1
   write !, "D in JobSub: ", ^JobSubD
   kill ^JobSubD

   kill A, B
   set E = ##class(Sample.Person).%OpenId(1)
   write !, "E:", E.Name
}

ClassMethod Sub()
{
   set C = ##class(Sample.Person).%OpenId(1)
   write !, "C in Sub: ", C.Name
}

ClassMethod JobSub()
{
   set D = ##class(Sample.Person).%OpenId(1)
   set ^JobSubD = D.Name
}

}
```

Calling the **Main()** method produces these results:

1. The initial **%OpenId()** call loads the instance object into memory. The *A* variable references this object and you can modify the loaded object through this variable.

   ### ObjectScript

   ```
   set A = ##class(Sample.Person).%OpenId(1)
   write "A: ", A.Name
   ```

   ```
   A: Uhles,Norbert F.
   ```

ObjectScript

```
   set A.Name = "David"
   write !, "A after set: ", A.Name
```

```
A after set: David
```

2. The second **%OpenId()** call does *not* reload the object from the database and does not overwrite the changes made using variable *A*. Instead, variable *B* references the same object as variable *A*.

   ObjectScript

```
   set B = ##class(Sample.Person).%OpenId(1)
   write !, "B: ", B.Name
```

```
B: David
```

3. The third **%OpenId()** call, in the **Sub()** method, also references the previously loaded object. This method is part of the same InterSystems IRIS process as the **Main()** method. Variable *C* references the same object as variables *A* and *B*, even though those variables are not available within the scope of this method. At the end of the **Sub()** method, the process returns back to the **Main()** method, the variable *C* is destroyed, and variables *A* and *B* are back in scope.

   ObjectScript

```
   do ..Sub()
```

   Class Member

```
ClassMethod Sub()
{
   set C = ##class(Sample.Person).%OpenId(1)
   write !, "C in Sub: ", C.Name
}
```

```
C in Sub: David
```

4. The fourth **%OpenId()** call, in the **JobSub()** method, is run in a separate InterSystems IRIS process by using the JOB command. This new process loads a new instance object into memory, and variable *D* references this new object.

   ObjectScript

```
   job ..JobSub()
   hang 1
   write !, "D in JobSub: ", ^JobSubD
   kill ^JobSubD
```

   Class Member

```
ClassMethod JobSub()
{
   set D = ##class(Sample.Person).%OpenId(1)
   set ^JobSubD = D.Name
}
```

```
D in JobSub: Uhles,Norbert F.
```

5. The fifth **%OpenId()** call occurs after deleting all variables (*A* and *B*) that reference the loaded object in the original process, thereby removing that object from memory. As with the previous **%OpenId()** call, this call also loads a new instance object into memory, and variable *E* references this object.

   ObjectScript

```
   kill A, B
   set E = ##class(Sample.Person).%OpenId(1)
   write !, "E:", E.Name
```

```
E: Uhles,Norbert F.
```

To force a reload of an object being loaded multiple times, you can use the **%Reload()** method. Reloading the object reverts any changes made to the object that were not saved. Any previously assigned variables reference the reloaded version. For example:

ObjectScript

```
set A = ##class(Sample.Person).%OpenId(1)
write "A: ", A.Name // A: Uhles,Norbert F.

set A.Name = "David"
write !, "A after set: ", A.Name // David

set B = ##class(Sample.Person).%OpenId(1)
write !, "B before reload: ", B.Name // David

do B.%Reload()
write !, "B after reload: ", B.Name // Uhles,Norbert F.
write !, "A after reload: ", A.Name // Uhles,Norbert F.
```

15.3.1.1 Concurrency

The **%OpenId()** method takes an optional *concurrency* argument as input. This argument specifies the concurrency level (type of locks) that should be used to open the object instance.

If the **%OpenId()** method is unable to acquire a lock on an object, it fails.

To raise or lower the current concurrency setting for an object, reopen it with **%OpenId()** and specify a different concurrency level. For example,

ObjectScript

```
set person = ##class(Sample.Person).%OpenId(6,0)
```

opens *person* with a concurrency of 0 and the following effectively upgrades the concurrency to 4:

ObjectScript

```
set person = ##class(Sample.Person).%OpenId(6,4)
```

Specifying a concurrency level of 3 (shared, retained lock) or 4 (shared, exclusive lock) forces a reload of the object when opening it. For more information on the possible object concurrency levels, see Specifying Concurrency Options.

15.4 The Internal Reference Count

Assigning an object value to a variable (or object property) has the side effect of incrementing the object's internal reference count, as shown in the following example:

ObjectScript

```
SET x = ##class(Sample.Person).%New()
WRITE x,!
SET y = ##class(Sample.Person).%New()
WRITE y,!
SET z = ##class(Sample.Person).%New()
WRITE z,!
```

When the number of references to an object reaches 0, the system automatically destroys the object (invoke its %OnClose() callback method and remove it from memory).

15.5 Swizzling

If you open (load into memory) an instance of a persistent object, and use an object that it references, then this referenced object is automatically opened. This process is referred to as *swizzling*; it is also sometimes known as lazy loading.

For example, the following code opens an instance of Sample.Employee object and automatically swizzles (opens) its related Sample.Company object by referring to it using dot syntax:

ObjectScript

```
// Open employee "101"
Set emp = ##class(Sample.Employee).%OpenId(101)

// Automatically open Sample.Company by referring to it:
Write "Company: ",emp.Company.Name,!
```

When an object is swizzled, it is opened using the default concurrency value of the class it is a member of, not the concurrency value of the object that swizzles it. See Specifying Concurrency Options.

A swizzled object is removed from memory as soon as no objects or variables refer to it.

Sometimes you might want to manually swizzle an object or check to see if an object is swizzled. For both of these tasks, you can use the property method **<prop>GetSwizzled()**. In the case of the Company property in the above example, calling `CompanyGetSwizzled()` swizzles the Company object and returns an OREF for the object. If you pass in a 1 as an argument, as in `CompanyGetSwizzled(1)`, the method returns an OREF for the object, but only if it is already swizzled. Otherwise, it returns a null OREF.

As an example, in Terminal:

```
USER>set emp = ##class(Sample.Employee).%OpenId(102)

USER>set co = emp.CompanyGetSwizzled(1)

USER>if (co = "") {write "Company is not swizzled"} else {write co.Name}
Company is not swizzled
USER>set co = emp.CarGetSwizzled()

USER>if (co = "") {write "Company is not swizzled"} else {write co.Name}
Acme Company, Inc.
```

15.6 Reading Stored Values

Suppose you have opened an instance of a persistent object, modified its properties, and then wish to view the original value stored in the database before saving the object. The easiest way to do this is to use one of the generated property methods, specifically *propertyname***GetStored()**. For example:

ObjectScript

```
// Open person "1"
Set person = ##class(Sample.Person).%OpenId(1)
Write "Original value: ",person.Name,!

// modify the object
Set person.Name = "Black,Jimmy Carl"
Write "Modified value: ",person.Name,!

// Now see what value is on disk
Set id = person.%Id()
Set name = ##class(Sample.Person).NameGetStored(id)
Write "Disk value: ",name,!
```

Another easy option is to use an embedded SQL statement (SQL is always executed against the database; not against objects in memory). Instead of calling `NameGetStored()`, use the following:

ObjectScript

```
// Now see what value is on disk
&sql(SELECT Name INTO :name
        FROM Sample.Person WHERE %ID = :id)

If (SQLCODE = 0) {Write "Disk value: ",name,!}
```

15.7 Deleting Saved Objects

The persistence interface includes methods for deleting objects from the database.

15.7.1 The %DeleteId() Method

The **%DeleteId()** method deletes an object that is stored within a database, including any stream data associated with the object. The signature of this method is as follows:

```
classmethod %DeleteId(id As %String, concurrency As %Integer = -1) as %Status
```

Where:

- *id* is the ID of the object to open.
- *concurrency* is the concurrency level (locking) used when deleting the object.

For example:

```
Set sc = ##class(MyApp.MyClass).%DeleteId(id)
```

%DeleteId() returns a %Status value indicating whether the object was deleted or not.

%DeleteId() calls the **%OnDelete()** callback method (if present) before deleting the object. **%OnDelete()** returns a %Status value; if **%OnDelete()** returns an error value, then the object will not be deleted, the current transaction is rolled back, and **%DeleteId()** returns an error value.

%DeleteId() calls the **%OnAfterDelete()** callback method (if present) after deleting the object. This call occurs immediately after **%DeleteData()** is called, provided that **%DeleteData()** does not return an error. If **%OnAfterDelete()** returns an error, then the current transaction is rolled back, and **%DeleteId()** returns an error value.

Lastly, **%DeleteId()** calls the **%OnDeleteFinally()** callback method, if present. This callback method is called after the transaction has completed and the status of the deletion has been finalized.

Note that the **%DeleteId()** method has no effect on any object instances in memory.

You can also use the **%Delete()** method, which requires an OID rather than an ID. An OID is a permanent identifier unique to the database that includes both the ID and the class name of the object. For more details, see Identifiers for Saved Objects: ID and OID.

15.7.2 The %DeleteExtent() Method

The **%DeleteExtent()** method deletes every object (and subclass of object) within its extent. Specifically it iterates through the entire extent and invokes the **%Delete()** method on each instance of the object. By default, it calls the **%KillExtent()** method when all instances are successfully deleted.

The signature of this method is as follows:

```
classmethod %DeleteExtent(concurrency As %Integer = -1, ByRef deletecount, ByRef instancecount,
pInitializeExtent As %Integer = 1, Output errorLog As %Status) as %Status
```

Where:

- *concurrency* is the concurrency level (locking) used when deleting the object.
- *deletecount* is the number of instances deleted.
- *instancecount* is the original number of instances.
- *pInitializeExtent* determines whether to call **%KillExtent()** when the extent is empty.
  - A value of 0 indicates that **%KillExtent()** will not be called. Some empty globals may remain, as well as the ID counter, if it exists.
  - A value of 1 indicates that **%KillExtent()** will be called, but the default stream global will not be deleted. This is the default.
  - A value of 2 indicates that **%KillExtent()** will be called, and the default stream global will be deleted.

  Note: If the globals used by the extent are not exclusively used by the extent, some globals may remain even if **%KillExtent()** is called.

- *errorLog* is used to return any applicable errors.

For example:

```
Set sc = ##class(MyApp.MyClass).%DeleteExtent(-1, .deletecount, .instancecount, 2, .errorLog)
```

15.7.3 The %KillExtent() Method

The **%KillExtent()** method directly deletes the globals that store an extent of a class, its subextents (subclasses), and its child extents (extents of classes that have a child relationship of the class). The default stream storage global is optionally deleted. **%KillExtent()** does not invoke the **%Delete()** method and performs no referential integrity actions.

The signature of this method is as follows:

```
classmethod %KillExtent(pDirect As %Integer = 1, killstreams As %Boolean = 0) as %Status
```

Where:

- *pDirect* determines whether the filing timestamps are also deleted from the ^OBJ.DSTIME global. This is used in InterSystems IRIS Business Intelligence. See Keeping the Cubes Current. The default is 1.
- *killstreams* determines whether the default stream global is deleted. The default is 0.

For example:

```
Set sc = ##class(MyApp.MyClass).%KillExtent(1, 0)
```

CAUTION: **%KillExtent()** should only be called directly in a development environment and should not be used in a live application. **%KillExtent()** bypasses constraints and user-implemented callbacks, potentially causing data integrity problems. If you need your application to delete every object in an extent, use **%DeleteExtent()** instead.

15.8 Accessing Object Identifiers

If an object has been saved, it has an ID and an OID, the permanent identifiers that are used on disk. If you have an OREF for the object, you can use that to obtain these identifiers.

To find the ID associated with an OREF, call the **%Id()** method of the object. For example:

ObjectScript

```
write oref.%Id()
```

To find the OID associated with an OREF, you have two options:

1. Call the **%Oid()** method of the object. For example:

   ObjectScript

   ```
   write oref.%Oid()
   ```

2. Access the %%OID property of the object. Because this property name contains % characters, you must enclose the name in double quotes. For example:

   ObjectScript

   ```
   write oref."%%OID"
   ```

15.9 See Also

- Introduction to Persistent Objects
- Specifying Concurrency Options
- Defining Persistent Classes
- Other Options for Persistent Classes
- Using Triggers

16
Object Concurrency Options

It is important to specify concurrency appropriately when you open or delete persistent objects.

16.1 Why Specify Concurrency?

The following scenario demonstrates why it is important to control concurrency appropriately when you read or write objects. Consider the following scenario:

1. Process A opens an object without specifying the concurrency.

```
SAMPLES>set person = ##class(Sample.Person).%OpenId(5)

SAMPLES>write person
1@Sample.Person
```

2. Process B opens the same object with the concurrency value of 4.

```
SAMPLES>set person = ##class(Sample.Person).%OpenId(5, 4)

SAMPLES>write person
1@Sample.Person
```

3. Process A modifies a property of the object and attempts to save it using **%Save()** and receives an error status.

```
SAMPLES>do person.FavoriteColors.Insert("Green")

SAMPLES>set status = person.%Save()

SAMPLES>do $system.Status.DisplayError(status)

ERROR #5803: Failed to acquire exclusive lock on instance of 'Sample.Person'
```

This is an example of concurrent operations without adequate concurrency control. For example, if process A *could* possibly save the object back to the disk, it *must* open the object with concurrency 4 to ensure it can save the object without conflict with other processes. (These values are discussed next.) In this case, Process B would then be denied access (failed with a concurrency violation) or would have to wait until Process A releases the object.

16.2 Options

You can specify concurrency at several different levels:

1. You can specify the *concurrency* argument for the method that you are using.

   Many of the methods of the %Persistent class allow you to specify this argument, an integer. This argument determines how locks are used for concurrency control. There are multiple allowed concurrency values.

   If you do not specify the *concurrency* argument in a call to the method, the method sets the argument to -1, which means that InterSystems IRIS uses the value of the *DEFAULTCONCURRENCY* class parameter for the class you are working with; see the next item.

2. You can specify the *DEFAULTCONCURRENCY* class parameter for the associated class. All persistent classes inherit this parameter from %Persistent, which defines the parameter as an expression that obtains the default concurrency for the process; see the next item.

   You could override this parameter in your class and specify a hardcoded value or an expression that determines the concurrency via your own rules. In either case, the value of the parameter must be one of the allowed concurrency values.

3. You can set the default concurrency mode for a process. To do so, use the **$system.OBJ.SetConcurrencyMode()** method (which is the **SetConcurrencyMode()** method of the %SYSTEM.OBJ class).

   As in the other cases, you must use one of the allowed concurrency values. If you do not set the concurrency mode for a process explicitly, the default value is 1.

   The **$system.OBJ.SetConcurrencyMode()** method has no effect on any classes that specify an explicit value for the *DEFAULTCONCURRENCY* class parameter.

To take a simple example, if you open a persistent object using its **%OpenId()** method without specifying a concurrency value (or you specify a value of -1 explicitly), the class does not specify the *DEFAULTCONCURRENCY* class parameter, and you do not set the concurrency mode in code, the default concurrency value for the object is set to 1.

Note: Some system classes have the *DEFAULTCONCURRENCY* class parameter set to a value of 0. Sharded classes always use a concurrency value of 0. To check the concurrency value of an object, examine its %Concurrency property.

16.3 Concurrency Values

This section describes the possible concurrency values. First, note the following details:

- Atomic writes are guaranteed when concurrency is greater than 0.
- InterSystems IRIS acquires and releases locks during operations such as saving and deleting objects; the details depend upon the concurrency value, what constraints are present, lock escalation status, and the storage structure.
- In all cases, when an object is removed from memory, any locks for it are removed.

The possible concurrency values are as follows; each value has a name, also shown in the list.

Concurrency Value 0 (No locking)

No locks are used.

Concurrency Value 1 (Atomic read)

Locks are acquired and released as needed to guarantee that an object read will be executed as an atomic operation.

InterSystems IRIS does not acquire any lock when creating a new object.

While opening an object, InterSystems IRIS acquires a shared lock for the object, *if that is necessary to guarantee an atomic read*. InterSystems IRIS releases the lock after completing the read operation.

The following table lists the locks that are present in each scenario:

	When object is created	While object is being opened	After object has been opened	After save operation is complete
New object	no lock	N/A	N/A	no lock
Existing object	N/A	shared lock, if that is necessary to guarantee an atomic read	no lock	no lock

Concurrency Value 2 (Shared locks)

The same as 1 (atomic read) except that opening an object *always* acquires a shared lock (even if the lock is not needed to guarantee an atomic read). The following table lists the locks that are present in each scenario:

	When object is created	While object is being opened	After object has been opened	After save operation is complete
New object	no lock	N/A	N/A	no lock
Existing object	N/A	shared lock	no lock	no lock

Concurrency Value 3 (Shared/retained locks)

InterSystems IRIS does not acquire any lock when creating a new object.

While opening an existing object, InterSystems IRIS acquires a shared lock for the object.

After saving a new object, InterSystems IRIS has a shared lock for the object.

The following table lists the scenarios:

	When object is created	While object is being opened	After object has been opened	After save operation is complete
New object	no lock	N/A	N/A	shared lock
Existing object	N/A	shared lock	shared lock	shared lock

Concurrency Value 4 (Exclusive/retained locks)

When an existing object is opened or when a new object is first saved, InterSystems IRIS acquires an exclusive lock.

The following table lists the scenarios:

	When object is created	While object is being opened	After object has been opened	After save operation is complete
New object	no lock	N/A	N/A	exclusive lock
Existing object	N/A	exclusive lock	exclusive lock	exclusive lock

Concurrency Value -1 (Use default concurrency value)

If you do not specify a concurrency value explicitly, the value is set to -1, meaning that the default concurrency value is used. See Object Concurrency Options for information on how the default concurrency value is determined.

16.4 Concurrency and Swizzled Objects

An object referenced by a property is swizzled on access using the default concurrency defined by the swizzled object's class. If the default is not defined for the class, the object is swizzled using the default concurrency mode of the process. (See Object Concurrency Options for details.) The swizzled object does not use the concurrency value of the object that swizzles it.

If the object being swizzled is already in memory, then swizzling does not actually open the object — it simply references the existing in-memory object; in that case, the current state of the object is maintained and the concurrency is unchanged.

There are two ways to override this default behavior:

- Upgrade the concurrency on the swizzled object with a call to the **%OpenId()** method that specifies the new concurrency. For example:

  ObjectScript

  ```
  Do person.Spouse.%OpenId(person.Spouse.%Id(),4,.status)
  ```

  where the first argument to **%OpenId()** specifies the ID, the second specifies the new concurrency, and the third (passed by reference) receives the status of the method.

- Set the default concurrency mode for the process before swizzling the object. For example:

  ObjectScript

  ```
  Set olddefault = $system.OBJ.SetConcurrencyMode(4)
  ```

  This method takes the new concurrency mode as its argument and returns the previous concurrency mode.

  When you no longer need a different concurrency mode, reset the default concurrency mode as follows:

  ObjectScript

  ```
  Do $system.OBJ.SetConcurrencyMode(olddefault)
  ```

16.5 Version Checking (Alternative to Concurrency Argument)

Rather than specifying the concurrency argument when you open or delete an object, you can implement *version checking*. To do so, you specify a class parameter called *VERSIONPROPERTY*. All persistent classes have this parameter. When defining a persistent class, the procedure for enabling version checking is:

1. Create a property of type %Integer that holds the updateable version number for each instance of the class.
2. For that property, set the value of the InitialExpression keyword to 0.
3. For the class, set the value of the *VERSIONPROPERTY* class parameter equal to the name of that property. The value of *VERSIONPROPERTY* cannot be changed to a different property by a subclass.

This incorporates version checking into updates to instances of the class.

When version checking is implemented, the property specified by *VERSIONPROPERTY* is automatically incremented each time an instance of the class is updated (either by objects or SQL). Prior to incrementing the property, InterSystems IRIS compares its in-memory value to its stored value. If they are different, then a concurrency conflict is indicated and an error is returned; if they are the same, then the property is incremented and saved.

Note: You can use this set of features to implement optimistic concurrency.

To implement a concurrency check in an SQL update statement for a class where *VERSIONPROPERTY* refers to a property called InstanceVersion, the code would be something like:

```
SELECT InstanceVersion,Name,SpecialRelevantField,%ID
    FROM my_table
    WHERE %ID = :myid

    // Application performs operations on the selected row

UPDATE my_table
    SET SpecialRelevantField = :newMoreSpecialValue
    WHERE %ID = :myid AND InstanceVersion = :myversion
```

where *myversion* is the value of the version property selected with the original data.

16.6 See Also

- Introduction to Persistent Objects
- Working with Persistent Objects
- Locking and Concurrency Control

17
Defining Persistent Classes

A persistent class is a class that defines persistent objects. This topic describes how to create such classes.

The samples shown on this page are from Samples-Data (https://github.com/intersystems/Samples-Data). InterSystems recommends that you create a dedicated namespace called SAMPLES (for example) and load samples into that namespace. For the general process, see *Downloading Samples for Use with InterSystems IRIS® data platform*.

17.1 Defining a Persistent Class

To define a class that defines persistent objects, ensure that the *primary* (first) superclass of your class is either %Persistent or some other persistent class.

For example:

Class Definition

```
Class MyApp.MyClass Extends %Persistent
{
}
```

17.2 Projection of Packages to Schemas

For persistent classes, the package is represented in SQL as an SQL schema. For instance, if a class is called Team.Player (the Player class in the Team package), the corresponding table is Team.Player (the Player table in the Team schema).

The default package is User, which is represented in SQL as the SQLUser schema. Hence, a class called User.Person corresponds to a table called SQLUser.Person.

If a package name contains periods, the corresponding table name uses an underscore in the place of each. For example, the class MyTest.Test.MyClass (the MyClass class in the MyTest.Test package) becomes the table MyTest_Test.MyClass (the MyClass table in the MyTest_Test schema).

If an SQL table name is referenced without the schema name, the default schema name (SQLUser) is used. For instance, the command:

SQL

```
Select ID, Name from Person
```

is the same as:

SQL

```
Select ID, Name from SQLUser.Person
```

17.3 Specifying the Table Name for a Persistent Class

For a persistent class, by default, the *short class name* (the part of the name after the last period) becomes the table name.

To specify a different table name, use the SqlTableName class keyword. For example:

```
Class App.Products Extends %Persistent [ SqlTableName = NewTableName ]
```

Although InterSystems IRIS places no restrictions on class names, SQL tables cannot have names that are SQL reserved words. Thus if you create a persistent class with a name that is an SQL reserved word, you will not be able to query the associated table. In this case, you must either rename the class or specify a table name for the projection that differs from the class name, using the technique described here.

17.4 Controlling How IDs Are Generated

When you save an object for the first time, the system generates an ID for the object. IDs are permanent.

By default, InterSystems IRIS uses an integer for the ID, incremented by 1 from the last saved object.

You can define a given persistent class so that it generates IDs in either of the following ways:

- The ID can be based on a specific property of the class, if that property is unique per instance. For example, you could use a drug code as the ID. To define a class this way, add an index like the following to the class:

  ```
  Index IndexName On PropertyName [ IdKey ];
  ```

  Or (equivalently):

  ```
  Index IndexName On PropertyName [ IdKey, Unique ];
  ```

  Where *IndexName* is the name of the index, and *PropertyName* is the name of the property.

  If you define a class this way, when InterSystems IRIS saves an object for the first time, it uses the value of that property as the ID. Furthermore, InterSystems IRIS requires a value for the property and enforces uniqueness of that property. If you create another object with the same value for the designated property and then attempt to save the new object, InterSystems IRIS issues this error:

  ```
  ERROR #5805: ID key not unique for extent
  ```

  Also, InterSystems IRIS prevents you from changing that property in the future. That is, if you open a saved object, change the property value, and try attempt to save the changed object, InterSystems IRIS issues this error:

  ```
  ERROR #5814: Oid previously assigned
  ```

  This message refers to the OID rather than the ID, because the underlying logic prevents the OID from being changed; the OID is based on the ID.

- The ID can be based on multiple properties. To define a class this way, add an index like the following to the class:

  ```
  Index IndexName On (PropertyName1,PropertyName2,...) [ IdKey, Unique ];
  ```

  Or (equivalently):

  ```
  Index IndexName On (PropertyName1,PropertyName2,...) [ IdKey ];
  ```

  Where *IndexName* is the name of the index, and *PropertyName1*, *PropertyName2*, and so on are the property names.

  If you define a class this way, when InterSystems IRIS saves an object for the first time, it generates an ID as follows:

  ```
  PropertyName1||PropertyName2||...
  ```

  Furthermore, InterSystems IRIS requires values for the properties and enforces uniqueness of the given combination of properties. It also prevents you from changing *any* of those properties.

Important: If a literal property (that is, an attribute) contains a sequential pair of vertical bars (`||`), do not add an IdKey index that uses that property. This restriction is imposed by the way in which the InterSystems SQL mechanism works. The use of `||` in IdKey properties can result in unpredictable behavior.

The system generates an OID as well. In all cases, the OID has the following form:

```
$LISTBUILD(ID,Classname)
```

Where *ID* is the generated ID, and *Classname* is the name of the class.

17.5 Controlling the SQL Projection of Subclasses

When several persistent classes are in superclass/subclass hierarchy, there are two ways in which InterSystems IRIS can store their data. The default scenario is by far the most common.

17.5.1 Default SQL Projection of Subclasses

The class compiler projects a flattened representation of a persistent class, such that the projected table contains all the appropriate fields for the class, including those that are inherited. Hence, for a subclass, the SQL projection is a table composed of:

- All the columns in the extent of the superclass
- Additional columns based on properties only in the subclass
- Rows that represent the saved instances of the subclass

Furthermore, in the default scenario, the extent of the superclass contains one record for each saved object of the superclass *and all its subclasses*. The extent of each subclass is a subset of the extent of the superclass.

For example, consider the persistent classes Sample.Person and Sample.Employee in SAMPLES. The Sample.Employee class inherits from Sample.Person and adds some additional properties. In the SAMPLES, both classes have saved data.

- The SQL projection of Sample.Person is a table that contains all the suitable properties of the Sample.Person class. The Sample.Person table contains one record for each saved instance of the Sample.Person class *and* each saved instance of the Sample.Employee class.

- The Sample.Employee table includes the same columns as Sample.Person and also includes columns that are specific to the Sample.Employee class. The Sample.Employee table contains one record for each saved instance of the Sample.Employee class.

To see this, use the following SQL queries. The first lists all instances of Sample.Person and shows their properties:

SQL

```
SELECT * FROM Sample.Person
```

The second query lists all instances of Sample.Employee and their properties:

SQL

```
SELECT * FROM Sample.Employee
```

Notice that the Sample.Person table contains records with IDs in the range 1 to 200. The records with IDs in the range 101 to 200 are employees, and the Sample.Employee table shows the same employees (with the same IDs and with additional columns). The Sample.Person table is arranged in two apparent groups *only* because of the artificial way that the SAMPLES database is built. The Sample.Person table is populated and then the Sample.Employee table is populated.

Typically, the table of a subclass has more columns and fewer rows than its parent. There are more columns in the subclass because it usually adds additional properties when it extends the parent class; there are often fewer rows because there are often fewer instances of the subclass than the parent.

17.5.2 Alternative SQL Projection of Subclasses

The default projection is the most convenient, but on occasion, you might find it necessary to use the alternative SQL projection. In this scenario, each class has its own extent. To cause this form of projection, include the following in the definition of the superclass:

```
[ NoExtent ]
```

For example:

Class Definition

```
Class MyApp.MyNoExtentClass [ NoExtent ]
{
//class implementation
}
```

Each subclass of this class then receives its own extent.

If you create classes in this way and use them as properties of other classes, see Variation: CLASSNAME Parameter.

17.6 See Also

- Defining Classes
- Introduction to Persistent Objects
- Working with Persistent Objects
- Other Options for Persistent Classes

18

Defining and Using Literal Properties

You can define literal properties in any object class. A literal property holds a literal value and is based on a data type class. This topic describes how to define and use such properties.

Where noted, some topics also apply to properties of other kinds.

18.1 Defining Literal Properties

To add a literal property to a class definition, add an element like one of the following to the class:

Class Member

```
Property PropName as Classname;

Property PropName as Classname [ Keywords ] ;

Property PropName as Classname(PARAM1=value,PARAM2=value) [ Keywords ] ;

Property PropName as Classname(PARAM1=value,PARAM2=value) ;
```

Where:

- *PropName* is the name of the property.
- *Classname* is the class on which this property is based. If you omit this, *Classname* is assumed to be %String, which has a default maximum length of 50 characters. To define this property as a literal property other than %String, specify *Classname* as the name of another data type class, which can be a custom data type class that you create.
- *Keywords* represents any property keywords. See Compiler Keywords.
- *PARAM1*, *PARAM2*, and so on are property parameters. The available parameter depend on the class used by the property.

Notice that the property parameters, if included, are enclosed in parentheses and precede any property keywords. The property keywords, if included, are enclosed in square brackets.

18.1.1 Examples

For example, a class can define a Count property using the %Integer data type class:

Class Member

```
Property Count As %Integer;
```

Because %Integer is a data type class, Count is a data type property.

You can use data type parameters to place constraints on the allowed values of data type properties. For example, for a property of type %String, you can specify the *MAXLEN* parameter:

Class Member

```
Property Description As %String(MAXLEN = 500);
```

To set a data type property value equal to the empty string, use code of the form:

ObjectScript

```
 Set object.Property = $C(0)
```

Every property has a collation type, which determines how values are ordered (such as whether capitalization has effects or not). The default collation type for strings is SQLUPPER. For more details on collations, see Data Collation.

18.2 Defining an Initial Expression for a Property

By default, when a new object is created, each property equals null. You can specify an ObjectScript expression to use as the initial value for a given property instead. The expression is evaluated when the object is created.

To specify an initial expression for a property, include the InitialExpression keyword in the property definition:

```
Property PropName as Classname [ InitialExpression=expression ] ;
```

Where *expression* is an ObjectScript expression (note that you cannot use any other language for this expression). For details, see InitialExpression.

18.3 Defining a Property As Required

This option applies only to properties in persistent classes. (Note that this option applies to any kind of property, not just literal ones.)

By default, when a new object is saved, InterSystems IRIS does not require it to have non-null values for its properties. You can define a property so that a non-null value is required. To do so, include the Required keyword in the property definition:

```
Property PropName as Classname [ Required ] ;
```

Then, if you attempt to save an object that has a null value for the property, InterSystems IRIS issues the following error:

```
ERROR #5659: Property required
```

The Required keyword also affects how you can insert or modify data for this class via SQL access. Specifically, an error occurs if the value is missing when you attempt to insert or update a record.

18.4 Defining a Computed Property

In InterSystems IRIS, you can define computed properties, whose values are computed via ObjectScript, possibly based on other properties. The generic phrase *computed properties* (or *computed fields*) includes both of the following variations:

- *Always computed* — The value of this property is calculated when it is accessed. It is not stored in the database unless you defined an index, in which case that index is stored.
- *Triggered computed* — The value of this property is recalculated when triggered (details given below).

  If the property is defined in a persistent class, its value *is* stored in the database.

In both cases, the recalculation is performed whether you use object access or an SQL query.

There are six property keywords (SqlComputed, SqlComputeCode, SqlComputeOnChange, Transient, Calculated, ComputeLocalOnly) that control if and how a property is computed. Instead of specifying SqlComputeCode, you can write compute code in a `*Property*Computation` method, where *Property* is the name of the computed property. For more details, see Computed Values.

The following table summarizes the possibilities:

Table 18–1: How to Tell if a Property Is Computed

		SqlComputed is true (and SqlComputeCode is defined)	SqlComputed is false
Calculated is true	Transient is true	Property is *always computed*	Property is not computed
Calculated is true	Transient is false	Property is *always computed*	Property is not computed
Calculated is false	Transient is true	Property is *always computed*	Property is not computed
Calculated is false	Transient is false	Property is *triggered computed* (SqlComputeOnChange can also be specified in this case)	Property is not computed

To define a computed property, do the following:

- Include the SqlComputed keyword in the property definition. (That is, specify the SqlComputed keyword as true.)
- Include the SqlComputeCode keyword in the property definition. For the value of this keyword, specify (in curly braces) a line of ObjectScript code that sets the value of the property, according to rules given in SqlComputeCode. For example:

  Class Definition

```
Class Sample.Population Extends %Persistent
{
Property FirstName As %String;

Property LastName As %String;

Property FullName As %String [ SqlComputeCode = {set {*}={FirstName}_" "_{LastName}}, SqlComputed
 ];
// ...

}
```

Alternatively, specify a *Property*`Computation` method using ObjectScript or another language, such as Python, to set the value of the property. For example:

Class/ObjectScript

```
Class Sample.Population Extends %Persistent
{
Property FirstName As %String;

Property LastName As %String;

Property FullName As %String [ SqlComputed ];
// ...

ClassMethod FullNameComputation(cols As %Library.PropertyHelper) As %String
{
    return cols.getfield("FirstName")_" "_cols.getfield("LastName")
}
// ...
}
```

Class/Python

```
Class Sample.Population Extends %Persistent
{
Property FirstName As %String;

Property LastName As %String;

Property FullName As %String [ SqlComputed ];
// ...

ClassMethod FullNameComputation(cols As %Library.PropertyHelper) As %String [ Language = python ]
{
    return cols.getfield("FirstName") + " " + cols.getfield("LastName")
}
// ...
}
```

- If you want to make the property always computed, specify the Calculated keyword as true in the property definition.

  Or, if you want to make the property triggered computed, do not include the Calculated and Transient keywords in the property definition. (That is, make sure both keywords are false.)

- If the property is triggered computed, optionally specify SqlComputeOnChange.

  This keyword can specify one or more properties. When any of these properties change in value, the triggered property is recomputed. Note that you must use the property names rather than the names given by SqlFieldName, which is discussed later). For example (with artificial line breaks):

  ```
  Property messageId As %Integer [
  SqlComputeCode = { set
  {*}=$Select({Status}="":0,1:$List($List($Extract({Status},3,$Length({Status}))))) },
  SqlComputed, SqlComputeOnChange = Status ];
  ```

  For another example (with artificial line breaks):

  ```
  Property Test2 As %String [ SqlComputeCode = { set {*}={Refprop1}_{Refprop2}}, SqlComputed,
  SqlComputeOnChange = (Refprop1, Refprop2) ];
  ```

  The value of SqlComputeOnChange can also include the values `%%INSERT` or `%%UPDATE`; for details, see SqlComputeOnChange.

- In a sharded or federated environment, if you want to return data stored in a computed field only from the server that the query was issued from, you can additionally specify the ComputeLocalOnly keyword.

If you intend to index this field, use deterministic code, rather than nondeterministic code. InterSystems IRIS cannot maintain an index on the results of nondeterministic code because it is not possible to reliably remove stale index key values.

(Deterministic code returns the same value every time when passed the same arguments. So for example, code that returns $h is nondeterministic, because $h is modified outside of the control of the function.)

Also see Calculated. And see Controlling the SQL Projection of Computed Properties, below.

18.5 Defining a Multidimensional Property

You can define a property to be multidimensional, which means that you intend the property to act as a multidimensional array. To do so, include the MultiDimensional keyword in the property definition:

```
Property PropName as Classname [ MultiDimensional ] ;
```

This property is different from other properties as follows:

- InterSystems IRIS does not provide property methods for it (see Using and Overriding Property Methods).
- It is ignored when the object is validated or saved.
- It is not saved to disk, unless your application includes code to save it specifically.
- It cannot be exposed to clients such as Java.
- It cannot be stored in or exposed through SQL tables.

Multidimensional properties are rare but provide a useful way to temporarily contain information about the state of an object.

Also see Specifying Values for a Multidimensional Property.

18.6 Defining Enumerated Properties

Many properties support the parameters *VALUELIST* and *DISPLAYLIST*. You use these to define enumerated properties.

To specify a list of valid values for a property, use its *VALUELIST* parameter. The form of *VALUELIST* is a delimiter-separated list of logical values, where the delimiter is the first character. For instance:

Class Member

```
Property Color As %String(VALUELIST = ",red,green,blue");
```

In this example, *VALUELIST* specifies that valid possible values are red, green, and blue, with a comma as its delimiter. Similarly,

Class Member

```
Property Color As %String(VALUELIST = " red green blue");
```

specifies the same list, but with a space as its delimiter.

The property is restricted to values in the list, and the data type validation code simply checks to see if the value is in the list. If no list is present, there are no special restrictions on values.

DISPLAYLIST is an additional list that, if present, represents the corresponding display values to be returned by the **LogicalToDisplay()** method of the property.

For an example that shows how to obtain the display values, see Using Property Methods.

18.7 Specifying Values for Literal Properties

To specify a value for a literal property, use the SET command, an OREF, and dot syntax as follows:

```
SET oref.MyProp=value
```

Where *oref* is an OREF, *MyProp* is a property of the corresponding object, and *value* is an ObjectScript expression that evaluates to a literal value. For example:

```
SET patient.LastName="Muggles"
SET patient.HomeAddress.City="Carver"
SET mrn=##class(MyApp.MyClass).GetNewMRN()
set patient.MRN=mrn
```

The literal value must be a valid logical value for the property type. For example, use 1 or 0 for a property based on %Boolean. For another example, for an enumerated property, the value must be one of the items specified by the *VALUELIST* parameter.

18.7.1 Specifying Values for a Multidimensional Property

For a multidimensional property, you can specify values for any subscripts of the property. For example:

```
set oref.MyStateProp("temp1")=value1
set oref.MyStateProp("temp2")=value2
set oref.MyStateProp("temp3")=value3
```

Multidimensional properties are useful for holding temporary information for use by the object. These properties are not saved to disk.

18.8 Controlling the SQL Projection of Literal Properties

A persistent class is projected as an SQL table. For that class, all properties are projected to SQL, aside from the following exceptions:

- Transient properties (but see Controlling the SQL Projection of Computed Properties)
- Calculated properties (but see Controlling the SQL Projection of Computed Properties)
- Private properties
- Multidimensional properties

This section discusses the details for literal properties.

18.8.1 Specifying the Field Names

By default, a property (if projected to SQL) is projected as an SQL field with the same name as the property. To specify a different field name, use the property keyword SqlFieldName. (You may want to use this keyword if the property name is an SQL reserved word). For instance, if there is a %String property called “Select,” you could define its projected name with the following syntax:

Class Member

```
Property Select As %String [ SqlFieldName = SelectField ];
```

Note: If you do use a property name that is an SQL reserved word, you can also escape the corresponding field name with double quotes when using it in a SQL statement, for example:

```
SELECT ID, Name, "Select" FROM SQLUser.Person
```

For more information, see Delimited Identifiers.

18.8.2 Specifying the Column Numbers

The system automatically assigns a unique column number for each field. To control column number assignments, specify the property keyword SqlColumnNumber, as in the following example:

Class Member

```
Property RGBValue As %String [ SqlColumnNumber = 3 ];
```

The value you specify for SqlColumnNumber must be an integer greater than 1. If you use the SqlColumnNumber keyword without an argument, InterSystems IRIS assigns a column number that is not preserved and that has no permanent position in the table.

If any property has an SQL column number specified, then InterSystems IRIS assigns column numbers for the other properties. The starting value for the assigned column numbers is the number following the highest SQL column number specified.

The value of the SqlColumnNumber keyword is inherited.

18.8.3 Effect of the Data Type Class and Property Parameters

The data type class used by a given property has an effect on the SQL projection. Specifically, the SQL category of the data type (defined with the SqlCategory keyword) control how the property is projected. Where applicable, the property parameters also have an effect:

For example, consider the following property definition:

Class Member

```
Property Name As %String(MAXLEN = 30);}
```

This property is projected as a string field with a maximum length of 30 characters.

18.8.4 Controlling the SQL Projection of Computed Properties

In InterSystems IRIS, you can define computed properties, whose values are computed via ObjectScript; see Defining Computed Properties. The following table indicates which variations are projected to SQL:

Property Definition	Projection to SQL
Property is not computed and is not transient (the default)	Property is projected to SQL
Property is not computed and *is* transient	Property is not projected to SQL
Property is *always computed*	Property is projected to SQL
Property is *triggered computed*	Property is projected to SQL

This table considers only the effects of the Calculated, Transient, SqlComputed, and SqlComputeCode keywords. It assumes that the property does not use other keywords that would prevent it from having an SQL projection. For example, it assumes that the property is not private.

18.9 See Also

- Defining and Using Properties
- Common Data Type Classes
- Common Property Parameters
- Working with Collections
- Working with Streams
- Defining and Using Object-Valued Properties
- Defining and Using Relationships
- Using Property Methods

19
Common Data Type Classes

This topic provides an overview of the InterSystems IRIS data type classes, which you typically use to define literal properties. Each data type class specifies several keywords that control how that data type is used in InterSystems SQL and how the data type is projected to client systems. You should choose a class that has the appropriate client projection, if applicable, for your needs. If there is no suitable class, you can create your own data type class, as described in Defining Data Type Classes.

Note that each data type class also uses one or more property parameters, whose default values you can override within your property definition.

19.1 Introduction

The most commonly used data type classes are as follows:

%BigInt

Holds a 64–bit integer. This class is similar to %Integer except for its OdbcType and ClientDataType.

%Binary

Holds binary data. The actual binary data is sent to and from the client without any Unicode (or other) translations.

%Boolean

Holds a boolean value. The possible logical values are 0 (false) and 1 (true).

%Char

Holds a fixed length character field.

%Counter

Holds an integer, meant for use as a unique counter. Any property whose type class is %Counter will be assigned a value when a new object is saved or a new record is inserted via SQL, if no value is specified for that property. For details, see %Counter in the InterSystems Class Reference.

%Currency

Holds a currency value. This class is defined only for migrating from Sybase or SQL Server to InterSystems IRIS.

%Date

Holds a date. The logical value is in InterSystems IRIS $HOROLOG format.

%DateTime

Holds a date and time. This class is used mainly for T-SQL migrations and maps datetime/smalldatetime behavior to the %TimeStamp datatype. In this class, the **DisplayToLogical()** and **OdbcToLogical()** methods provide logic to handle imprecise datetime values that are supported by T-SQL applications.

%Decimal

Holds a fixed point number. The logical value is a decimal format number. See Numeric Computing in InterSystems Applications.

%Double

Holds an IEEE floating-point number. The logical value is an IEEE floating-point number. See Numeric Computing in InterSystems Applications.

%EnumString

Holds a string. This is a specialized subclass of %String that allows you to define an enumerated set of *possible* values (using the *DISPLAYLIST* and *VALUELIST* parameters). Unlike %String, the display values for this property are used when columns of this type are queried via ODBC.

%ExactString

Holds a string. This is a subclass of %String with the EXACT default collation.

%Integer

Holds an integer.

%List

Holds data in $List format. The logical value is data in $List format.

%ListOfBinary

Holds data in $List format, with each list item as binary data. The logical value is data in $List format.

%Name

Holds a name in the form "Lastname,Firstname". The %Name data type has special indexing support when used in conjunction with the %Storage.Persistent class. For details, see %Name in the InterSystems Class Reference.

%Numeric

Holds a fixed-point number.

%PosixTime

Holds a value for a time and date. The logical value of this data type is the number of seconds since (or before) January 1, 1970 00:00:00, encoded as a 64-bit integer. %PosixTime uses less disk space and memory than %TimeStamp data type, and is better for performance than %TimeStamp.

%SmallInt

Holds a small integer value. This class is the same as %Integer except for its OdbcType.

%Status

Holds an error status code and status information. Many methods in the InterSystems IRIS Class Library return values of type %Status. For information on working with these values, see %Status.

%String

Holds a string. For %String, the default maximum length is 50 characters. You can override this by setting the MAXLEN parameter.

%Time

Holds a time value. The logical value is the number of seconds past midnight.

%TimeStamp

Holds a value for a time and date. The logical value of the %TimeStamp data type is in YYYY-MM-DD HH:MM:SS.nnnnnnnnn format. Note that if *h* is a date/time value in $H format, then you can use the $ZDATETIME as follows to obtain a valid logical value for a %TimeStamp property: `$ZDATETIME(h,3)`

Also see the comments for %PosixTime.

%TinyInt

Holds a very small integer value. This class is the same as %Integer except for its OdbcType and its maximum and minimum values.

%Vector

Holds a value in the native **$vector** format. See also the TO_VECTOR() SQL function to convert user input to this datatype. See Using Vector Search.

There are many additional data type classes, most of which are subclasses of these classes. See the InterSystems Class Reference for details.

19.2 Data Type Classes Grouped by SqlCategory

For a data type class, the SqlCategory class keyword specifies the SQL category that InterSystems SQL uses when operating on values of properties of that type. Operations controlled by SqlCategory include comparison operations (such as greater than, less than, or equal to); other operations may also use it. The following table shows the SqlCategory values of the data types listed in this topic.

Table 19–1: Data Type Classes Grouped by SqlCategory

Value	InterSystems IRIS Data Type
DATE	%Date
DOUBLE	%Double
INTEGER	%BigInt, %Boolean, %Counter, %Integer, %SmallInt, %TinyInt
NAME	%Name
NUMERIC	%Currency, %Decimal, %Numeric
POSIXTS	%PosixTime

Value	InterSystems IRIS Data Type
STRING	%Binary, %Char, %EnumString, %ExactString, %List, %ListOfBinary, %Status, %String
TIME	%Time
TIMESTAMP	%DateTime, %TimeStamp
VECTOR	%Vector

For further information on how literal properties are projected to SQL types, see SQL Data Types.

19.3 Data Type Classes Grouped by OdbcType

For a data type class, the OdbcType class keyword controls how InterSystems IRIS translates logical data values to and from values used by the InterSystems SQL ODBC interface. The following table shows the OdbcType values of the data types listed in this topic.

Table 19–2: Data Type Classes Grouped by OdbcType

Value	InterSystems IRIS Data Type
BIGINT	%BigInt
BIT	%Boolean
DATE	%Date
DOUBLE	%Double
INTEGER	%Counter, %Integer
NUMERIC	%Currency, %Decimal, %Numeric
TIME	%Time
TIMESTAMP	%DateTime, %PosixTime, %TimeStamp
VARBINARY	%Binary
VARCHAR	%Char, %EnumString, %ExactString, %List, %ListOfBinary, %Name, %Status, %String, %Vector

19.4 Data Type Classes Grouped by ClientDataType

For a data type class, the ClientDataType class keyword controls how InterSystems IRIS projects a property (of that type) to Java or Active X. The following table shows the ClientDataType values of the data types listed in this topic.

Table 19–3: Data Type Classes Grouped by ClientDataType

Value	Used for
BIGINT	%BigInt
BINARY	%Binary (or any property requiring that there is no Unicode conversion of data)
CURRENCY	%Currency

Value	Used for
DATE	%Date
DOUBLE	%Double
DECIMAL	%Decimal
INTEGER	%Counter, %Integer, %SmallInt, %TinyInt
LIST	%List, %ListOfBinary
NUMERIC	%Numeric
TIME	%Time
TIMESTAMP	%DateTime, %PosixTime, %TimeStamp
VARCHAR	%Char, %EnumString, %ExactString, %Name, %String, %Vector

19.5 See Also

- Defining and Using Literal Properties
- Common Property Parameters
- Using and Overriding Property Methods
- Defining Data Type Classes

20

Common Property Parameters

This page provides reference information on common property parameters available for use in literal properties.

20.1 Introduction

Depending on the property, you can specify parameters of that property, to affect its behavior. For example, parameters can specify minimum and maximum values, formatting for use in display, collation, delimiters for use in specific scenarios, and so on. You can specify parameters within the class definition. The following shows an example:

```
Property MyProperty as %String (MAXLEN=500);
```

Note that many of these parameters act as constraints; the term *constraint* refers to any parameter that constrains the values of a property. For example, *MAXVAL*, *MINVAL*, *DISPLAYLIST*, *VALUELIST*, and *PATTERN* are all constraints.

20.2 Available Property Parameters, Listed by Data Type Class

You can use *CALCSELECTIVITY*, *CAPTION*, *EXTERNALSQLNAME*, *EXTERNALSQLTYPE*, *JAVATYPE*, and *STORAGEDEFAULT* in all property definitions. You can use additional property parameters, depending on the data type class:

%BigInt

Common parameters: *DISPLAYLIST*, *FORMAT*, *MAXVAL*, *MINVAL*, *VALUELIST*

Parameters for XML and SOAP: *XSDTYPE*

%Binary

Common parameters: *MAXLEN*, *MINLEN*

Parameters for XML and SOAP: *CANONICALXML*, *MTOM*, *XSDTYPE*

%Boolean

Parameters for XML and SOAP: *XSDTYPE*

%Char

Common parameters: *COLLATION*, *CONTENT*, *DISPLAYLIST*, *ESCAPE*, *MAXLEN*, *MINLEN*, *PATTERN*, *TRUNCATE*, *VALUELIST*

Parameters for XML and SOAP: *XMLLISTPARAMETER*, *XSDTYPE*

%Counter

Common parameters: *DISPLAYLIST*, *FORMAT*, *MAXVAL*, *MINVAL*, *VALUELIST*

Parameters for XML and SOAP: *XSDTYPE*

%Currency

Common parameters: *DISPLAYLIST*, *FORMAT*, *MAXVAL*, *MINVAL*, *SCALE*, *VALUELIST*

Parameters for XML and SOAP: *XSDTYPE*

Note: this special-purpose class is only for use in migrations to InterSystems IRIS.

%Date

Common parameters: *DISPLAYLIST*, *FORMAT*, *MAXVAL*, *MINVAL*, *VALUELIST*

Parameters for XML and SOAP: *XSDTYPE*

%DateTime

Common parameters: *DISPLAYLIST*, *MAXVAL*, *MINVAL*, *VALUELIST*

Parameters for XML and SOAP *XMLDEFAULTVALUE*, *XMLTIMEZONE*, *XSDTYPE*

Other parameters: *DATEFORMAT*

%Decimal

Common parameters: *DISPLAYLIST*, *FORMAT*, *MAXVAL*, *MINVAL*, *SCALE*, *VALUELIST*

Parameters for XML and SOAP: *XSDTYPE*

%Double

Common parameters: *DISPLAYLIST*, *FORMAT*, *MAXVAL*, *MINVAL*, *SCALE*, *VALUELIST*

Parameters for XML and SOAP: *XSDTYPE*

%EnumString

Common parameters: *COLLATION*, *CONTENT*, *DISPLAYLIST*, *ESCAPE*, *MAXLEN*, *MINLEN*, *PATTERN*, *TRUNCATE*, *VALUELIST*

Parameters for XML and SOAP: *XSDTYPE*

%ExactString

Common parameters: *COLLATION*, *CONTENT*, *DISPLAYLIST*, *ESCAPE*, *MAXLEN*, *MINLEN*, *PATTERN*, *TRUNCATE*, *VALUELIST*

Parameters for XML and SOAP: *XSDLISTPARAMETER*, *XSDTYPE*

%Integer

Common parameters: *DISPLAYLIST*, *FORMAT*, *MAXVAL*, *MINVAL*, *VALUELIST*

Parameters for XML and SOAP: *XSDTYPE*

Other parameters: *STRICT*

%List

Common parameters: *ODBCDELIMITER*

Parameters for XML and SOAP: *XSDTYPE*

%ListOfBinary

Common parameters: *ODBCDELIMITER*

Parameters for XML and SOAP: *XSDTYPE*

%Name

Common parameters: *COLLATION*,*MAXLEN*

Parameters for XML and SOAP: *XSDTYPE*

Other parameters: *INDEXSUBSCRIPTS*

%Numeric

Common parameters: *DISPLAYLIST*, *FORMAT*, *MAXVAL*, *MINVAL*, *SCALE*, *VALUELIST*

Parameters for XML and SOAP: *XSDTYPE*

%PosixTime

Common parameters: *MAXVAL*, *MINVAL*

Parameters for XML and SOAP *XMLDEFAULTVALUE*, *XMLTIMEZONE*, *XSDTYPE*

Other parameters: *DATEFORMAT*

%SmallInt

Common parameters: *DISPLAYLIST*, *FORMAT*, *MAXVAL*, *MINVAL*, *VALUELIST*

Parameters for XML and SOAP: *XSDTYPE*

%Status

Parameters for XML and SOAP: *XSDTYPE*

%String

Common parameters: *COLLATION*, *CONTENT*, *DISPLAYLIST*, *ESCAPE*, *MAXLEN*, *MINLEN*, *PATTERN*, *TRUNCATE*, *VALUELIST*

Parameters for XML and SOAP: *XMLLISTPARAMETER*, *XSDTYPE*

%Time

Common parameters: *DISPLAYLIST*, *FORMAT*, *MAXVAL*, *MINVAL*, *VALUELIST*

Parameters for XML and SOAP: *XMLTIMEZONE*, *XSDTYPE*

Other parameters: *PRECISION*

%TimeStamp

Common parameters: *DISPLAYLIST*, *MAXVAL*, *MINVAL*, *VALUELIST*

Parameters for XML and SOAP: *XMLDEFAULTVALUE*, *XMLTIMEZONE*, *XSDTYPE*

%TinyInt

Common parameters: *DISPLAYLIST*, *FORMAT*, *MAXVAL*, *MINVAL*, *VALUELIST*

Parameters for XML and SOAP: *XSDTYPE*

%Vector

Common parameters: *DATATYPE*, *LEN*, *STORAGEDEFAULT*

Many data type classes also support *INDEXNULLMARKER*, which should almost never be overridden; it is described with the rest of the parameters in the next section.

20.3 Property Parameter Reference

This section provides reference information on the property parameters used by the most common data type classes, other than the ones that apply to XML and SOAP contexts.

CALCSELECTIVITY

Available for all literal properties. Controls whether the automatic collected stats utility calculates the *selectivity* for a property. Usually it is best to leave this parameter as the default (1). For details, see Table Stats for Query Optimizer.

CAPTION

Available for all literal properties. Caption to use for this property in client applications.

COLLATION

Specifies the manner in which property values are transformed for indexing.

The allowable values for collation are discussed in SQL Introduction.

CONTENT

Specifies the contents of the string, when the string is used in a context where it might be interpreted as XML or HTML. Specify `"STRING"` (the default), `"ESCAPE"`, or `"MIXED"`.

For details, see Controlling the Projection to XML Schemas.

DATEFORMAT

- For %DateTime, this parameter specifies the order of the date parts when a numeric date format is specified for the display or ODBC input value. Valid parameters are mdy, dmy, ymd, ydm, myd, and dym. The default *DATEFORMAT* is mdy.
- For %PosixTime, this parameter specifies the format for the display value. For allowed values, see the *fformat* parameter of the **$zdatetime** and **$zdatetimeh** functions.

DISPLAYLIST

Used for enumerated (multiple-choice) properties, along with *VALUELIST*. For more information, see Defining Enumerated Properties.

ESCAPE

Specifies the type of escaping to be done, if the string is used in certain contexts. Use either "XML" (the default) or "HTML".

By default, the less than, greater than, and ampersand characters are interpreted as < > and & respectively. For further details on "XML", see Controlling the Projection to XML Schemas.

EXTERNALSQLNAME

Available for all literal properties. Used in linked tables, this parameter specifies the name of the field in the external table to which this property is linked. The Link Table wizard specifies this parameter for each property when it generates a class. The name of the SQL field on the remote database may need to differ from property name on the InterSystems IRIS server for various reason, such as because the remote database field name is a reserved word in InterSystems IRIS. For information on linked tables, see The Link Table Wizard.

Note that the property parameter *EXTERNALSQLNAME* has a different purpose than the SQLFieldName compiler keyword, and these items can have different values. SQLFieldName specifies the projected SQL field name in the InterSystems IRIS database, and *EXTERNALSQLNAME* is the field name in the remote database.

EXTERNALSQLTYPE

Available for all literal properties. Used in linked tables, this parameter specifies the SQL type of the field in the external table to which this property is linked. The Link Table wizard specifies this parameter for each property when it generates a class. See *EXTERNALSQLNAME*.

FORMAT

Specifies the format for the display value. For the value of *FORMAT*, use a format string as specified in the *format* argument of the $FNUMBER function. For properties of type %Numeric or %Decimal, you can also use the option "AUTO", which suppresses any trailing zeroes.

INDEXNULLMARKER

Specifies the default null marker to use when a property of this type is used in a subscript of an index map. For default values of this parameter, see %Library.DataType and %Library.PosixTime.

CAUTION: Do not override this parameter unless you have existing code setting these values into index globals directly and your application code is careful not to use those values in INSERT or SELECT statements.

INDEXSUBSCRIPTS

For %Name. Specifies the number of subscripts used by the property in indexes, using a comma as a delimiter in the property value; the %Storage.Persistent class uses this number. A value of 2 specifies that the first comma piece of the property value is stored as the first subscript and the second comma piece of the property value is stored as the second subscript.

JAVATYPE

Available for all literal properties. The Java data type to which this property is projected.

MAXLEN

Specifies the maximum number of characters the string can contain. The default is 50, and if this parameter is `""`, the length is unlimited. As with many other parameters, this parameter affects how InterSystems IRIS validates data. Note that it also affects how the field is projected over a database driver.

MAXVAL

Specifies the maximum allowed logical value for the data type.

MINLEN

Specifies the minimum number of characters the string can contain.

MINVAL

Specifies the minimum allowed logical value for the data type.

ODBCDELIMITER

Specifies the delimiter character used to construct a %List value when it is projected via ODBC.

PATTERN

Specifies a pattern that the string must match. The value of *PATTERN* must be a valid InterSystems IRIS pattern-matching expression. For an overview of pattern matching, see Pattern Matching.

PRECISION

For %Time. Specifies the number of decimal places to retain. If the value is `""` (the default), the system retains the number of decimal places that are provided in the source value. If the value is 0, InterSystems IRIS rounds the provided value to the nearest second.

SCALE

Specifies the number of digits following the decimal point.

STORAGEDEFAULT

Available for all literal properties. Specifies how the property values are to be stored, when this property is used in a persistent class. The allowed values for this parameter are `row` and `columnar`. See Choose an SQL Table Storage Layout.

Collection properties also use this property parameter, but in a different way. See Storage and SQL Projection of Collection Properties.

STRICT

For %Integer. Requires that value be an integer. By default, if a property is of type %Integer, and you specify a non-integer numeric value, InterSystems IRIS converts the value to an integer. If *STRICT* is 1 for a property, in such a case, InterSystems IRIS does not convert the value; instead validation fails.

TRUNCATE

Specifies whether to truncate the string to *MAXLEN* characters, where 1 is TRUE and 0 is FALSE. This parameter is used by the **Normalize()** and **IsValid()** methods but is not used by database drivers.

VALUELIST

Used for enumerated (multiple-choice) properties, along with *DISPLAYLIST*. For more information, see Defining Enumerated Properties.

Note: The parameters that available for all properties are not defined in data type classes. The rest are defined within the data type classes that use them.

20.4 See Also

- Defining and Using Literal Properties
- Common Data Type Classes
- Controlling the Projection to XML Schemas

21
Working with Collection Classes

This page describes how to use the collection APIs (lists and arrays) in InterSystems IRIS® data platform. You can use collections as properties of objects, and you can also use them standalone.

21.1 Introduction to Collections

A *collection* is an object class that contains multiple elements of the same type. These types can be literal values, such as strings or integers, or objects. Each element in a collection consists of a *key* (the position of that element within the collection) and a *value* (the data stored at that position). Using collection methods and properties, you can perform actions such as:

- Access elements based on their keys and modify their values.
- Insert or delete elements at specified key positions.
- Count the number of elements in a collection.

InterSystems IRIS provides APIs for two kinds of collections: list and arrays.

- In *list collections*, elements are ordered sequentially. The keys in a list are integers from 1 to *N*, where *N* is the last element in the list. This example sets the second element of a list (that is, the element with a key of 2) to the string value `red`.

  ObjectScript

  ```
  do myList.SetAt("red",2)
  ```

- In *array collections*, elements are accessed based on arbitrary key names that you specify when you create the element. You can specify string or numeric key names. This example sets the `color` key of an array to the string value `red`.

  ObjectScript

  ```
  do myArray.SetAt("red","color")
  ```

  Elements in an array are ordered by key, with numeric keys first, sorted from smallest to largest, and string keys next, sorted alphabetically with uppercase letters coming before lowercase letters. For example: -2, -1, 0, 1, 2, A, AA, AB, a, aa, ab.

See Using Property Methods for information on the *sampleprop***BuildValueArray()** method.

21.2 Defining Collection Properties

To define list or array properties, use these property definition syntaxes:

Class Member

```
Property MyProp as list of Type;
```

Class Member

```
Property MyProp as array of Type;
```

Here, *MyProp* is the property name, and *Type* is either a data type class or an object class.

This example shows how to define a property for a list of %String values.

Class Member

```
Property Colors as list of %String;
```

This code shows how to define a property for an array of Doctor objects.

Class Member

```
Property Doctors as array of Doctor;
```

InterSystems IRIS represents these collection properties as instances of classes in the %Collection package. These classes contain the methods and properties that you can use to work with these collections, as follows:

- If the property is a list of strings, integers, or other data type, it is an instance of %Collection.ListOfDT.
- If the property is a list of objects, it is an instance of %Collection.ListOfObj.
- If the property is an array of strings, integers, or other data type, it is an instance of %Collection.ArrayOfDT.
- If the property is an array of objects, it is an instance of %Collection.ArrayOfObj.

Note: Do not use the *%Collection* classes directly as the type of a property. For example, do not create a property definition like this:

```
Property MyProp as %Collection.ArrayOfDT;
```

21.3 Defining Standalone Collections

As an alternative to defining collection properties, you can define standalone collections, such as for use as a method argument or return value. To define a standalone collection, use classes in the %Library package.

To create a standalone collection, call the **%New()** method of the suitable class to obtain an instance of that class. Then use methods of that instance to perform collection operations. For example, this code creates a list of strings using the %Library.ListOfDataTypes class, inserts three elements into the list, and then displays the number of elements in the list:

ObjectScript

```
set mylist=##class(%ListOfDataTypes).%New()
do mylist.Insert("red")
do mylist.Insert("green")
do mylist.Insert("blue")
write mylist.Count()
```

This table summarizes the ways to define standalone collections and the %Library classes used to create them.

Collection Type	%Library Class	Sample Creation Syntax
List of strings, integers, or other data type	%ListOfDataTypes	`set myList = ##class(%ListOfDataTypes).%New()`
List of objects	%ListOfObjects	`set myList = ##class(%ListOfObjects).%New()`
Array of strings, integers, or other data type	%ArrayOfDataTypes	`set myArray = ##class(%ArrayOfDataTypes).%New()`
Array of objects	%ArrayOfObjects	`set myArray = ##class(%ArrayOfObjects).%New()`

21.4 Working with List Collections

The examples in these sections show how to work with list properties, but you can use similar syntaxes to work with lists that are in a standalone collection.

21.4.1 Insert List Elements

To insert an element at the end of a list, use the **Insert()** method. For example, suppose that `obj` is a reference to an object, and `Colors` is a list property of the associated object. This property is defined as follows:

Class Member

```
Property Colors as list of %String;
```

This code inserts three elements into the list. If the list was previously empty, then these elements are located at positions 1, 2, and 3, respectively.

ObjectScript

```
do obj.Colors.Insert("Red") // key = 1
do obj.Colors.Insert("Green") // key = 2
do obj.Colors.Insert("Blue") // key = 3
```

To insert an element at a specific position within a list, use the **InsertAt()** method. For example, this code inserts the string `Yellow` into the second position of the `Colors` property of `obj`.

ObjectScript

```
do obj.Colors.InsertAt("Yellow",2)
```

The list elements now have this order: `"Red", "Yellow", "Green", "Blue"`. The new element is at position 2 and the elements previously at position 2 and 3 (`"Green"` and `"Blue"`) move to positions 3 and 4 to make room for the new element.

The insertion of list elements works the same for objects. For example, suppose that `pat` is an object reference, and `Diagnoses` is a list property of the associated object. This property is defined as follows, where PatientDiagnosis is the name of a class:

Class Member

```
Property Diagnoses as list of PatientDiagnosis;
```

This code creates a new class instance of PatientDiagnosis, stored in object reference `patdiag`, and inserts this object at the end of the `Diagnoses` list.

ObjectScript

```
Set patdiag = ##class(PatientDiagnosis).%New()
Set patdiag.DiagnosisCode=code
Set patdiag.DiagnosedBy=diagdoc
Set status=pat.Diagnoses.Insert(patdiag)
```

21.4.2 Access List Elements

To access list elements, you can use these collection methods:

- **GetAt(key)** – Return the element value at the position specified by *key*.
- **GetPrevious(key)** – Return the element value at the position immediately before *key*.
- **GetNext(key)** – Return the element value at the position immediately after *key*.
- **Find(value, key)** – Starting after *key*, return the key of the next list element that equals *value*.

For example, this code iterates over a list and displays the elements of that list in order. The Count property of collections determines the number of elements to iterate over.

ObjectScript

```
set p = ##class(Sample.Person).%OpenId(1)
for i = 1:1:p.FavoriteColors.Count() {write !, p.FavoriteColors.GetAt(i)}
```

21.4.3 Modify List Elements

To modify a value at a specified key, you can use the **SetAt()** method, as shown by this syntax:

ObjectScript

```
do oref.PropertyName.SetAt(value,key)
```

Here, `oref` is an object reference and `PropertyName` is the name of a list property of that object. For example, suppose that `person.FavoriteColors` is a list property of favorite colors containing elements `red`, `blue`, and `green`. This code changes the second color in the list to `yellow`:

ObjectScript

```
do person.FavoriteColors.SetAt("yellow",2)
```

21.4.4 Remove List Elements

To remove a list element, use the **RemoveAt()** method. For example, suppose that `person.FavoriteColors` is a list property of favorite colors containing elements `red`, `blue`, and `green`. This code removes the element at position 2 (`blue`)

ObjectScript

```
do person.FavoriteColors.RemoveAt(2)
```

The list elements now have this order: `red`, `green`. The element previously at position 3 (`green`) moves to position 2 to fill the gap caused by the removed element.

21.5 Working with Array Collections

To add an element to an array, use the **SetAt()** method. For example, `myArray.SetAt(value,key)` sets the array element at position *key* to the specified *value*. This code inserts a new color into an array of RGB values:

ObjectScript

```
do palette.Colors.SetAt("255,0,0","red")
```

Here, `palette` is a reference to the object that contains the array, `Colors` is the name of the array property, and `"red"` is the key to access the value `"255,0,0"`.

Important: Do not include a sequential pair of vertical bars (`||`) within the value that you use as the array key. This restriction is imposed by the InterSystems SQL mechanism.

The **SetAt()** methods also modifies the values of existing elements. For example, this code changes the value stored at the `"red"` key to the hexadecimal format:

ObjectScript

```
do palette.Colors.SetAt("#FF0000","red")
```

To iterate over the contents in an array, pass the key by reference to the **GetNext()** method, which causes the loop to iterate over both keys and values. For example, this code writes the keys and values of an string array in order. The elements are ordered alphabetically by key.

ObjectScript

```
set arr=##class(%ArrayOfDataTypes).%New()
do arr.SetAt("red","color")
do arr.SetAt("large","size")
do arr.SetAt("expensive","price")

set key=""
for  {set value=arr.GetNext(.key)  quit:key=""  write !,key," = ",value}

color = red
price = expensive
size = large
```

21.6 Copying Collection Data

To copy the items in one collection into another collection, set the recipient collection equal to the source collection. This copies the contents of the source into the recipient (not the OREF of the collection itself). Some examples of such a command are:

ObjectScript

```
Set person2.Colors = person1.Colors
Set dealer7.Inventory = owner3.cars
```

where *person2*, *person1*, *dealer7*, and *owner3* are all instances of classes and *Colors*, *Inventory*, and *cars* are all collection properties. The first line of code looks as it might for copying data between two instances of a single class and the second line of code as it might for copying data from an instance of one class to an instance of a different class.

If the recipient collection is a list and the source collection is an array, InterSystems IRIS copies only the data of the array (not its key values). If the recipient collection is an array and the source collection is a list, the InterSystems IRIS generates key values for the recipient array; these key values are integers based on the position of the item in the source list.

Note: There is no way to copy the OREF from one collection to another. It is only possible to copy the data.

21.7 See Also

- Defining and Using Properties
- Storage and SQL Projection of Collection Properties
- Defining and Using Literal Properties
- Working with Streams
- Defining and Using Object-Valued Properties
- Defining and Using Relationships
- Using Property Methods

22

Storage and SQL Projection of Collection Properties

This page describes how list and array properties are stored by default and projected to SQL by default, and how you can modify these details.

22.1 Default Storage and Projection of List Properties

By default, a list property is stored in the same table as the object it belongs to, and is projected to SQL as a single column consisting of a %List structure that contains multiple values. To find an item in this column, use the **FOR SOME %ELEMENT** predicate. For example, this query returns the rows in which the `FavoriteColors` column, which is a projection of the `FavoriteColors` list property, contains the element `'Red'`.

SQL

```
SELECT * FROM Sample.Person
WHERE FOR SOME %ELEMENT (FavoriteColors) (%VALUE = 'Red')
```

Alternatively, you can use the **%INLIST** predicate to either find an element in a list or find a list that does not contain a certain element. For example, this query returns the rows in which the `FavoriteColors` column does not contain the list element `'Red'`.

SQL

```
SELECT * FROM Sample.Person
WHERE NOT ('Red' %INLIST (FavoriteColors))
```

If the list for a particular instance contains no elements, it is projected as an empty string (and not an SQL NULL value).

22.2 Default Storage and Projection of Array Properties

By default, an array property is stored in a child table and is projected as that child table. The child table is in the same package as the parent table. The name of this child table is as follows:

```
tablename_fieldname
```

where:

- *tablename* is the SqlTableName of the parent class (if specified) or the short name of the parent class (if SqlTableName is not specified).
- *fieldname* is the SqlFieldName of the array property (if specified) or the name of the array property (if SqlFieldName is not specified).

For example, consider a Person class with an array property called Siblings. The projection of the Siblings property is a child table called Person_Siblings.

The child table contains these columns:

1. A column containing the IDs of the corresponding instance of the parent class. This column acts as a foreign key to the parent class containing the array and is named after that class. In the projected Person_Child table, this column is named Person.
2. A column named element_key that contains the identifier for each array member.
3. A column containing the value of each array member. This column is named after the array property. In the projected Person_Child table, this column is named Siblings.

This table shows sample entries and the generated column names of the Siblings child table.

Table 22–1: Sample Projection of an Array Property

Person	element_key	Siblings
10	C	Claudia
10	T	Tom
12	B	Bobby
12	C	Cindy
12	G	Greg
12	M	Marsha
12	P	Peter

If an instance of the parent class holds an empty collection (one that contains no elements), the ID for that instance does not appear in the child table, such as the instance above where ID equals 11.

Notice that there is no Siblings column in the parent table.

For the column(s) containing the array members, the number and contents of the column(s) depend on the kind of array:

- The projection of an array of data type properties is a single column of data.
- The projection of an array of reference properties is a single column of object references.
- The projection of an array of embedded objects is as multiple columns in the child table. The structure of these columns is described in Embedded Object Properties.

Together, the ID of each instance and the identifier of each array member comprise a unique index for the child table. Also, if a parent instance has no array associated with it, it has no associated entries in the child table.

Note: A serial object property is projected to SQL in the same way, by default.

Important: When a collection property is projected as an array, there are specific requirements for any index you might add to the property. See Indexing Collections. For an introduction to indexes in InterSystems IRIS persistent classes, see Other Options for Persistent Classes.

Important: There is no support for SQL triggers on child tables projected by array collections. However, if you update the array property and then save the parent object using ObjectScript, any applicable triggers will fire.

22.3 Controlling Storage of Collection Properties

You can store a list property as a child table, and you can store an array property as a $LIST. In both cases, you specify the *STORAGEDEFAULT* parameter of the property:

- For a list property, *STORAGEDEFAULT* is `"list"` by default. If you specify *STORAGEDEFAULT* as `"array"`, then the property is stored and projected as a child table. For example:

  ```
  Property MyList as list of %String (STORAGEDEFAULT="array");
  ```

  For details on the resulting projection, see Default Storage and Projection of Array Properties.

- For an array property, *STORAGEDEFAULT* is `"array"` by default. If you specify *STORAGEDEFAULT* as `"list"`, then the property is stored and projected as a list. For example:

  ```
  Property MyArray as array of %String (STORAGEDEFAULT="list");
  ```

  For details on the resulting projection, see Default Storage and Projection of List Properties.

Important: The *STORAGEDEFAULT* property parameter affects how the compiler generates storage for the class. If the class definition already includes a storage definition for the given property, the compiler ignores this property parameter.

22.4 Controlling the SQL Projection

However a collection property is actually stored, the property can be projected as a column in the parent table, as a child table, or in both ways (as of release 2022.1, this is true for both list and array properties). To control this, you specify the *SQLPROJECTION* parameter of the property. This parameter can have any of the following values:

- `"column"` — project this property as a column.
- `"table"` — project this property as a child table.
- `"table/column"` — project this property as a column and a child table.

For example, consider the following class definition:

Class Definition

```
Class Sample.Sample Extends %Persistent
{

Property Property1 As %String;

Property Property2 as array of %String(SQLPROJECTION = "table/column");

}
```

In this case, the system generates two tables for this class: Sample.Sample and Sample.Sample_Property2

The table Sample.Sample_Property2 stores the data for the array property Property2, as in the default scenario. Unlike the default scenario, however, a query can refer to the Property2 field in the Sample.Sample table. For example:

```
MYNAMESPACE>>SELECT Property2 FROM Sample.Sample where ID=7
13.     SELECT Property2 FROM Sample.Sample where ID=7

Property2
"1     value 12       value 23       value 3"
```

The `SELECT *` query, however, does not return the Property2 field:

```
MYNAMESPACE>>SELECT * FROM Sample.Sample where ID=7
14.     SELECT * FROM Sample.Sample where ID=7

ID      Property1
7       abc
```

22.5 Controlling the Name of the Projected Child Table

If a collection property is projected as a child table, you can control the name of that table. To do so, specify the *SQLTABLENAME* parameter of the property. For example:

```
Property MyArray as array of %String(SQLTABLENAME = "MyArrayTable");

Property MyList as list of %Integer(SQLTABLENAME = "MyListTable", STORAGEDEFAULT = "array");
```

The *SQLTABLENAME* parameter has no effect unless the property is projected as a child table.

22.6 See Also

- Introduction to the Default SQL Projection
- Working with Collections

23
Working with Streams

Streams provide a way to store extremely large amounts of data (longer than the string length limit). You can define stream properties in any object class. You can also define standalone stream objects for other purposes, such as for use as an method argument or return value. This topic describes streams and stream properties.

23.1 Introduction to Stream Classes

InterSystems IRIS® data platform provides the following stream classes:

- %Stream.GlobalCharacter — use this to store character data in global nodes.
- %Stream.GlobalBinary — use this to store binary data in global nodes.
- %Stream.FileCharacter — use this to store character data in an external file.
- %Stream.FileBinary — use to store binary data in an external file.
- %Stream.TmpCharacter — use this when you need a stream to hold character data but do not need to save the data.
- %Stream.TmpBinary — use this when you need a stream to hold binary data but do not need to save the data.

These classes all inherit from %Stream.Object, which defines the common stream interface.

The %Library package also includes stream classes, but those are deprecated. The class library includes additional stream classes, but those are not intended for general use.

Note that stream classes are object classes. Thus a stream is an object.

Important: Many of the methods of these classes return status values. In all cases, consult the class reference for details. If a method returns a status value, your code should check that returned value and proceed appropriately. Similarly, for %Stream.FileCharacter and %Stream.FileBinary, if you set the Filename property, you should next check for an error by examining *%objlasterror*.

23.2 Defining Stream Properties

InterSystems IRIS supports both binary streams and character streams. *Binary streams* contain the same sort of data as type %Binary, and are intended for very large binary objects such as pictures. Similarly, *character streams* contain the same sort of data as type %String, and are intended for storing large amounts of text. Character streams, like strings, may undergo a Unicode translation within client applications.

Stream data may be stored in an external file or an InterSystems IRIS global (or not at all), depending on how the stream property is defined:

- The %Stream.FileCharacter and %Stream.FileBinary classes are used for streams stored as external files.
- The %Stream.GlobalCharacter and %Stream.GlobalBinary classes are used for streams stored as globals.
- The %Stream.TmpCharacter and %Stream.TmpBinary classes are used for streams that do not need to be saved.

The first four classes can use the optional *LOCATION* parameter to specify a default storage location.

In the following example, the JournalEntry class contains four stream properties (one for each of the first four stream classes), and specifies a default storage location for two of them:

Class Definition

```
Class testPkg.JournalEntry Extends %Persistent
{
Property DailyText As %Stream.FileCharacter;

Property DailyImage As %Stream.FileBinary(LOCATION = "C:/Images");

Property Text As %Stream.GlobalCharacter(LOCATION = "^MyText");

Property Picture As %Stream.GlobalBinary;
}
```

In this example, data for DailyImage is stored in a file (with an automatically generated name) in the C:/Images directory, while the data for the Text property is stored in a global named ^MyText.

23.3 Using the Stream Interface

All streams inherit a set of methods and properties used to manipulate the data they contain. The next section lists the commonly used methods and properties, and the following sections provide concrete examples using them:

- Commonly used stream methods and properties
- How to instantiate a stream
- How to read and write stream data
- How to copy between streams
- How to insert stream data
- How to find literal values in a stream
- How to save a stream
- How to use streams in object applications

Important: Many of the methods of these classes return status values. In all cases, consult the class reference for details. If a method returns a status value, your code should check that returned value and proceed appropriately. Similarly, for %Stream.FileCharacter and %Stream.FileBinary, if you set the Filename property, you should next check for an error by examining *%objlasterror*.

23.3.1 Commonly Used Stream Methods and Properties

Some commonly used methods include the following:

- **Read()** — Read a specified number of characters starting at the current position in the stream.
- **Write()** — Append data to the stream, starting at the current position. Overwrites existing data if the position is not set to the end of the stream.
- **Rewind()** — Move to the beginning of the stream.
- **MoveTo()** — Move to a given position in the stream.
- **MoveToEnd()** — Move to the end of the stream.
- **CopyFrom()** — Copy the contents of a source stream into this stream.
- **NewFileName()** — Specify a filename for a %Stream.FileCharacter or %Stream.FileBinary property.

Commonly used properties include the following:

- AtEnd — Set to `true` when a Read encounters the end of the data source.
- Id — The unique identifier for an instance of a stream within the extent specified by %Location.
- Size — The current size of the stream (in bytes or characters, depending on the type of stream).

For detailed information on individual stream methods and properties, see the InterSystems Class Reference entries for the classes listed at the beginning of this topic.

23.3.2 Instantiating a Stream

When you use a stream class as an object property, the stream is instantiated implicitly when you instantiate the containing object.

When you use a stream class as a standalone object, use the **%New()** method to instantiate the stream.

23.3.3 Reading and Writing Stream Data

At the core of the stream interface are the methods **Read()**, **Write()**, and **Rewind()** and the properties AtEnd and Size.

The following example reads data from the Person.Memo stream and writes it to the console, 100 characters at a time. The value of `len` is passed by reference, and is reset to 100 before each Read. The Read method attempts to read the number of characters specified by `len`, and then sets it to the actual number of characters read:

```
Do person.Memo.Rewind()
While (person.Memo.AtEnd = 0) {
  Set len = 100
  Write person.Memo.Read(.len)
}
```

Similarly, you can write data into the stream:

```
Do person.Memo.Write("This is some text. ")
Do person.Memo.Write("This is some more text.")
```

23.3.3.1 Specifying a Translation Table

If you are reading or writing a stream of type %Stream.FileCharacter in any character set other than the native character set of the locale, you must set the TranslateTable property of the stream. See the Translation Tables reference page.

23.3.4 Copying Data between Streams

All streams contain a **CopyFrom()** method which allows one stream to fill itself from another stream. This can be used, for example, to copy data from a file into a stream property. In this case, one uses the %Library.File class, which is a wrapper around operating system commands and allows you to open a file as a stream. In this case, the code is:

```
// open a text file using a %Library.File stream
Set file = ##class(%File).%New("\data\textfile.txt")
Do file.Open("RU") // same flags as the OPEN command

// Open a Person object containing a Memo stream
// and copy the file into Memo
Set person = ##class(Person).%New()
Do person.Memo.CopyFrom(file)

Do person.%Save() // save the person object
Set person = ""   // close the person object
Set file = ""     // close the file
```

You can also copy data into a stream with the **Set** command:

```
Set person2.Memo = person1.Memo
```

where the Memo property of the Person class holds an OREF for a stream and this command copies the contents of *person1.Memo* into *person2.Memo*.

23.3.5 Inserting Stream Data

Apart from the temporary stream classes (whose data cannot be saved), streams have both a temporary and a permanent storage location. All inserts go into the temporary storage area, which is only made permanent when you save the stream. If you start inserting into a stream, then decide that you want to abandon the insert, the data stored in the permanent location is not altered.

If you create a stream, start inserting, then do some reading, you can call **MoveToEnd()** and then continue appending to the temporary stream data. When you save the stream, the data is moved to the permanent storage location.

For example:

```
Set test = ##class(Test).%OpenId(5)
Do test.text.MoveToEnd()
Do test.text.Write("append text")
Do test.%Save()
```

Here, the stream is saved to permanent storage when the *test* object is saved.

23.3.6 Finding Literal Values in a Stream

The stream interface includes the **FindAt()** method, which you can use to find the location of a given literal value. This method has the following arguments:

```
method FindAt(position As %Integer, target, ByRef tmpstr, caseinsensitive As %Boolean = 0) as %Integer
```

Where:

- *position* is the position at which to start searching.
- *target* is the literal value to search for.
- *tmpstr*, which is passed by reference, returns information that can be used in the next call to **FindAt()**. Use this when you want to search the same stream repeatedly, starting from the last position where the target was found. In this scenario,

specify *position* as –1 and pass *tmpstr* by reference in every call. Then each successive call to **FindAt()** will start where the last call left off.

- *caseinsensitive* specifies whether to perform a case-insensitive search. By default, the method does not consider case.

The method returns the position at this match starting at the beginning of the stream. If it does not find a match, it returns -1.

23.3.7 Saving a Stream

When you use a stream class as an object property, the stream data is saved when you save the containing object.

When you use a stream class as a standalone object, use the **%Save()** method to save the stream data. (Note that for the temporary stream classes — %Stream.TmpCharacter and %Stream.TmpBinary — this method returns immediately and does not save any data.)

23.3.8 Using Streams in Object Applications

Stream properties are manipulated via a transient object that is created by the object that owns the stream property. Streams act as literal values (think of them as large strings). Two object instances cannot refer to the same stream.

In the following class definition, the Person class has a Memo property that is a stream property:

```
Class testPkg.Person Extends %Persistent
{
Property Name As %String;

Property Memo As %Stream.GlobalCharacter;
}
```

The following ObjectScript fragment creates a new person object, implicitly instantiating the Memo stream. Then it writes some text to the stream.

```
    // create object and stream
    Set p = ##class(testPkg.Person).%New()
    Set p.Name = "Mo"
    Do p.Memo.Write("This is part one of a long memo. ")
    Do p.Memo.Write("This is part two of a long memo. ")
    Do p.Memo.Write("This is part three of a long memo. ")
    Do p.Memo.Write("This is part four of a long memo. ")
    Do p.%Save()
    Set id = p.%Id() // remember ID for later
    Set p = ""
```

The following code fragment opens the person object and then writes the contents of the stream. Note that when you open an object, the current position of any stream properties is set to the beginning of the stream. This code uses the **Rewind()** method for illustrative purposes.

```
    // read object and stream
    Set p = ##class(testPkg.Person).%OpenId(id)
    Do p.Memo.Rewind() // not required first time

    // write contents of stream to console, 100 characters at a time
    While (p.Memo.AtEnd = 0) {
        Set len = 100
        Write p.Memo.Read(.len)
    }
    Set p = ""
```

Note: If you want to replace the contents of a stream property, rewind the stream (if the current position of the stream is not already at the beginning), and then use the **Write()** method to write the new data to the stream. Do *not* use the **%New()** method to instantiate a new stream object and assign it to the stream property, for example, `set p.Memo = ##class(%Stream.GlobalCharacter).%New()`, as this leaves the old stream object orphaned in the database.

23.4 Stream Classes for Use with gzip Files

The %Stream package also defines the specialized stream classes %Stream.FileBinaryGzip and %Stream.FileCharacterGzip, which you can use to read and write to gzip files. These use the same interface described earlier. Note the following points:

- For these classes, the Size property returns the uncompressed size. When you access the Size property, InterSystems IRIS reads the data in order to calculate the size of the file and this can be an expensive operation
- When you access the Size property, InterSystems IRIS rewinds the stream and leaves you at its start.

23.5 Projection of Stream Properties to SQL and ODBC

A persistent class is projected as an SQL table. For such classes, character stream properties and binary stream properties are projected to SQL (and to ODBC clients) as BLOBs (binary large objects).

Stream properties are projected with the ODBC type LONG VARCHAR (or LONG VARBINARY). The ODBC driver/server uses a special protocol to read/write BLOBs. Typically you have to write BLOB applications by hand, because the standard reporting tools do not support them.

The following subsections describes how to use stream properties with SQL. It includes the following topics:

- How to read a stream via embedded SQL
- How to write a stream via embedded SQL

Stream fields have the following restrictions within SQL:

- You cannot use a stream value in a WHERE clause, with a few specific exceptions.
- You cannot UPDATE/INSERT multiple rows containing a stream; you must do it row by row.
- The only kind of index you can add is an SQL Search bitmap index. See Indexing Sources for SQL Search.

Also see Stream Data Types in the *InterSystems SQL Reference* for additional comments on using streams in SQL.

23.5.1 Reading a Stream via Embedded SQL

You can use embedded SQL to read a stream as follows:

1. Use embedded SQL to select the OID (Object ID) of the stream:

   ```
   &sql(SELECT Memo INTO :memo FROM Person WHERE Person.ID = 12345)
   ```

   This fetches the OID of the stream and places it into the `memo` host variable.

2. Then open the stream and process it as usual.

23.5.2 Writing a Stream via Embedded SQL

To write a stream via embedded SQL, you have several options. For the value to insert, you can use an object reference (OREF) of a stream, the string version of such an OREF, or a string literal.

The following examples show all these techniques. For these examples, assume that you have a table named `Test.ClassWStream` which has a column named `Prop1`, which expects a stream value.

The following example uses an object reference:

Class Member

```
///use an OREF
ClassMethod Insert1()
{
    set oref=##class(%Stream.GlobalCharacter).%New()
    do oref.Write("Technique 1")

    //do the insert; this time use an actual OREF
    &sql(INSERT INTO Test.ClassWStreams (Prop1) VALUES (:oref))
}
```

The next example uses a string version of an object reference:

Class Member

```
///use a string version of an OREF
ClassMethod Insert2()
{
    set oref=##class(%Stream.GlobalCharacter).%New()
    do oref.Write("Technique 2")

    //next line converts OREF to a string OREF
    set string=oref_""

    //do the insert
    &sql(INSERT INTO Test.ClassWStreams (Prop1) VALUES (:string))
}
```

The last example inserts a string literal into the stream `Prop1`:

Class Member

```
///insert a string literal into the stream column
ClassMethod Insert3()
{
    set literal="Technique 3"

    //do the insert; use a string
    &sql(INSERT INTO Test.ClassWStreams (Prop1) VALUES (:literal))
}
```

Note: The first character of the string literal cannot be a number. If it is a number, then SQL interprets this as an OREF and attempts to file it as such. Because the stream is not an OREF, this results in an SQL -415 error.

23.6 Stream Compression

InterSystems IRIS can automatically compress stream data that is stored in a global, saving database space and reducing the size of journal files.

Compression is controlled in stream classes by a class parameter, `COMPRESS`.

In the classes %Stream.TmpCharacter and %Stream.TmpBinary, `COMPRESS` is set to 0, meaning that the stream data is not compressed.

In the classes %Stream.GlobalCharacter and %Stream.GlobalBinary, `COMPRESS` is set to 1, meaning that new stream data is automatically compressed, except for the following cases, where the data is not suitable for compression:

- The stream can be stored in a single chunk less than 1024 characters long.
- The stream is already compressed, that is, the first chunk of data matches a typical compressed file format, such as JPEG, MP3, or ZIP.

- The first 4KB of the stream cannot be compressed by at least 20 percent.

23.7 See Also

- Defining and Using Properties
- Defining and Using Literal Properties
- Defining and Using Object-Valued Properties
- Defining and Using Relationships
- Working with Collections

24

Defining and Using Object-Valued Properties

This topic describes how to define and use object-valued properties, including serial object properties.

Relationships provide another way to associate different persistent classes; see Relationships.

24.1 Defining Object-Valued Properties

The phrase *object-valued property* generally refers to a property that is defined as follows:

```
Property PropName as Classname;
```

Where *Classname* is the name of an object class other than a collection or a stream. (Collection properties and stream properties are special cases discussed elsewhere.) In general, *Classname* is either a registered object class, a persistent class, or a serial class.

To define such a property, define the class to which the property refers and then add the property.

24.1.1 Variation: CLASSNAME Parameter

If a property is based on a persistent class, and that class uses the alternative projection of subclasses described in Defining Persistent Classes, an additional step is necessary. In this case, it is necessary to specify the *CLASSNAME* property parameter as 1 for that property. This step affects how InterSystems IRIS® data platform stores this property and enables InterSystems IRIS to retrieve the object to which it points.

For example, suppose that MyApp.Payment specifies NoExtent, and MyApp.CreditCard is a subclass of MyApp.Payment. Suppose that MyApp.CurrencyOrder contains a property of type MyApp.CreditCard. That property should specify *CLASSNAME* as 1:

Class Definition

```
Class MyApp.CurrencyOrder [ NoExtent ]
{
Property Payment as MyApp.CreditCard (CLASSNAME=1);

//other class members
}
```

Note that SQL arrow syntax does not work in this scenario. (You can instead use a suitable JOIN.)

Important: Do not specify *CLASSNAME* =1 for a property whose type is a serial class. This usage is not supported.

24.2 Introduction to Serial Objects

To define a serial class, ensure that the primary (first) superclass of your class is %SerialObject or another serial class. The purpose of such classes is to serve as a property in another object class. The values in a serial object are serialized into the parent object. Serial objects are also called embedded (or embeddable) objects. InterSystems IRIS handles serial object properties differently from non-serial object properties. Two of the differences are as follows:

- It is not necessary to call **%New()** to create the serial object before assigning values to properties in it.
- If the serial object property is contained in a persistent class, the properties of the serial object are stored within the extent of the persistent class.

To define a serial class, simply define a class that extends %SerialObject, and add properties and other class members as needed. The following shows an example:

Class Definition

```
Class Sample.Address Extends %SerialObject
{

/// The street address.
Property Street As %String(MAXLEN = 80);

/// The city name.
Property City As %String(MAXLEN = 80);

/// The 2-letter state abbreviation.
Property State As %String(MAXLEN = 2);

/// The 5-digit U.S. Zone Improvement Plan (ZIP) code.
Property Zip As %String(MAXLEN = 5);

}
```

24.3 Possible Combinations of Objects

The following table shows the possible combinations of a parent class and an object-valued property in that class:

	Property is a registered object class	Property is a persistent class	Property is a serial class
Parent class is a registered object class	Supported	Supported but not common	Supported
Parent class is a persistent class	Supported but not common	Supported	Supported
Parent class is a serial class	Not supported	Not supported	Supported

24.3.1 Terms for Object-Valued Properties

Within a persistent class, there are two terms for object-valued properties:

- *Reference properties* (properties based on other persistent objects)
- *Embedded object properties* (properties based on serial objects)

Relationships are another kind of property that associates different persistent classes; see Relationships. Relationships are bidirectional, unlike the properties described in this topic.

24.4 Specifying the Value of an Object Property

To set an object-valued property, set that property equal to an OREF of an instance of a suitable class.

Consider the scenario where ClassA contains a property PropB that is based on ClassB, where ClassB is an object class:

Class Definition

```
Class MyApp.ClassA
{

Property PropB as MyApp.ClassB;

//additional class members
}
```

And ClassB has a non-serial class with its own set of properties Prop1, Prop2, and Prop3.

Suppose that *MyClassAInstance* is an OREF for an instance of ClassA. To set the value of the PropB property for this instance, do the following:

1. If ClassB is not a serial class, first:

    a. Obtain an OREF for an instance of ClassB.

    b. Optionally set properties of this instance. You can also set them later.

    c. Set *MyClassAInstance*.PropB equal to that OREF.

    You can skip this step if ClassB is a serial class.

2. Optionally use cascading dot syntax to set properties of the property (that is, to set properties of *MyClassAInstance*.PropB).

For example:

ObjectScript

```
 set myclassBInstance=##class(MyApp.ClassB).%New()
 set myClassBInstance.Prop1="abc"
 set myClassBInstance.Prop2="def"
 set myClassAInstance.PropB=myclassBInstance
 set myClassAInstance.PropB.Prop3="ghi"
```

Notice that this example sets properties of the ClassB instance directly, right after the instance is created, and later more indirectly via cascading dot syntax.

The following steps accomplish the same goal:

ObjectScript

```
 set myClassAInstance.PropB=##class(MyApp.ClassB).%New()
 set myClassAInstance.PropB.Prop1="abc"
 set myClassAInstance.PropB.Prop2="def"
 set myClassAInstance.PropB.Prop3="ghi"
```

In contrast, if ClassB is a serial class, you can do the following, without ever calling **%New()** for ClassB:

ObjectScript

```
 set myClassAInstance.PropB.Prop1="abc"
 set myClassAInstance.PropB.Prop2="def"
 set myClassAInstance.PropB.Prop3="ghi"
```

24.5 Saving Changes

In the case where you are using persistent classes, save the containing object (that is, the instance that contains the object property). There is no need to save the object property directly, because that is saved automatically when the containing object is saved.

The following examples demonstrate these principles. Consider the following persistent classes:

Class Definition

```
Class MyApp.Customers Extends %Persistent
{

Property Name As %String;

Property HomeStreet As %String(MAXLEN = 80);

Property HomeCity As MyApp.Cities;

}
```

And:

Class Definition

```
Class MyApp.Cities Extends %Persistent
{

Property City As %String(MAXLEN = 80);

Property State As %String;

Property ZIP As %String;

}
```

In this case, we could create an instance of MyApp.Customers and set its properties as follows:

ObjectScript

```
    set customer=##class(MyApp.Customers).%New()
    set customer.Name="O'Greavy,N."
    set customer.HomeStreet="1234 Main Street"
    set customer.HomeCity=##class(MyApp.Cities).%New()
    set customer.HomeCity.City="Overton"
    set customer.HomeCity.State="Any State"
    set customer.HomeCity.ZIP="00000"
    set status=customer.%Save()
    if $$$ISERR(status) {
        do $system.Status.DisplayError(status)
    }
```

These steps add one new record to MyApp.Customers and one new record to MyApp.Cities.

Instead of calling **%New()** for MyApp.Cities, we could open an existing record:

ObjectScript

```
set customer=##class(MyApp.Customers).%New()
set customer.Name="Burton,J.K."
set customer.HomeStreet="17 Milk Street"
set customer.HomeCity=##class(MyApp.Cities).%OpenId(3)
set status=customer.%Save()
if $$$ISERR(status) {
    do $system.Status.DisplayError(status)
}
```

In the following variation, we open an existing city and modify it, in the process of adding the new customer:

ObjectScript

```
set customer=##class(MyApp.Customers).%New()
set customer.Name="Emerson,S."
set customer.HomeStreet="295 School Lane"
set customer.HomeCity=##class(MyApp.Cities).%OpenId(2)
set customer.HomeCity.ZIP="11111"
set status=customer.%Save()
if $$$ISERR(status) {
    do $system.Status.DisplayError(status)
}
```

This change would of course be visible to any other customers with this home city.

24.6 SQL Projection of Object-Valued Properties

A persistent class is projected as an SQL table. This section describes how reference properties and embedded object properties of such a class are projected to SQL.

24.6.1 Reference Properties

A reference property is projected as a field that contains the ID portion of the OID of the referenced object. For instance, suppose a customer object has a Rep property that refers to a SalesRep object. If a particular customer has a sales representative with an ID of 12, then the entry in the Rep column for that customer is also 12. Because this value matches that of the particular row of the ID column of the referenced object, you can use this value when performing any joins or other processing.

Note that within InterSystems SQL, you can use a special reference syntax to easily use such references, as an alternative to using a JOIN. For example:

SQL

```
SELECT Company->Name FROM Sample.Employee ORDER BY Company->Name
```

24.6.2 Embedded Object Properties

An embedded object property is projected as multiple columns in the table of the parent class. One column in the projection contains the entire object in serialized form (including all delimiters and control characters). The rest of the columns are each for one property of the object.

The name of the column for the object property is the same as that of the object property itself. The other column names are made up of the name of the object property, an underscore, and the property within the embedded object. For instance, suppose a class has a Home property containing an embedded object of type Address; Home itself has properties that include Street and Country. The projection of the embedded object then includes the columns named Home_Street and Home_Country. (Note that the column names are derived from the property, Home, and not the type, Address.)

For example, the sample class Sample.Person, includes a Home property which is an embedded object of type Sample.Address. You can use the component fields of Home via SQL as follows:

SQL

```
SELECT Name, Home_City, Home_State FROM Sample.Person
WHERE Home_City %STARTSWITH 'B'
ORDER BY Home_City
```

Embedded objects can also include other complex forms of data:

- The projection of a reference property includes a read-only field that includes the object reference as described in Reference Properties.
- The projection of an array is as a single, non-editable column that is part of the table.
- The projection of a list is as a list field as one of its projected fields; the list field is as described in Default Projection of List Properties.

24.7 See Also

- Defining and Using Properties
- Relationships
- Defining and Using Literal Properties
- Working with Collections
- Working with Streams
- Using and Overriding Property Methods

25

Defining and Using Relationships

This topic describes relationships, which are a special kind of property that you can define only in persistent classes. You cannot access relationships from Python methods.

25.1 Overview of Relationships

A *relationship* is an association between two persistent objects, each of a specific type. To create a relationship between two objects, each must have a relationship property, which defines its half of the relationship. InterSystems IRIS® data platform directly supports two kinds of relationships: one-to-many and parent-child.

InterSystems IRIS relationships have the following characteristics:

- Relationships are *binary* — a relationship is defined either between two, and only two, classes or between a class and itself.
- Relationships can only be defined for *persistent* classes.
- Relationships are *bidirectional* — both sides of a relationship must be defined.
- Relationships automatically provide referential integrity. They are visible to SQL as foreign keys. See SQL Projection of Relationships.
- Relationships automatically manage their in-memory and on-disk behavior.
- Relationships provide superior scaling and concurrency over object collections.

  On the other hand, in an object collection, there is an inherent order of the objects; the same is not true for relationships. If you insert objects A, B, and C, in that order, into a list of objects, that order is retained. If you insert objects A, B, and C, in that order, into a relationship property, that order is not retained.

Note: It is also possible to define foreign keys between persistent classes, rather than adding relationships. With a foreign key, you have a greater degree of control over what happens when an object in one class is added, updated, or deleted. See Using Triggers.

Note: Relationships are not supported for sharded classes.

25.1.1 One-to-Many Relationships

In a one-to-many relationship between class A and class B, one instance of class A is associated with zero or more instances of class B.

For example, a company class may define a one-to-many relationship with an employee class. In this case, there may be zero or more employee objects associated with each company object.

These classes are independent of each other as follows:

- When an instance of either class is created, it may or may not be associated with an instance of the other class.
- If an instance of class B is associated with a given instance of class A, this association can be removed or changed. The instance of class B can be associated with a different instance of class A. The instance of class B does not have to have any association with an instance of class A (and vice versa).

There can be a one-to-many relationship within a single class. One instance of that class can be associated with zero or more other instances of that class. For example, the Employee class might define a relationship between an employee and any employees who directly report that employee.

25.1.2 Parent-Child Relationships

In a parent-child relationship between class A and class B, one instance of class A is associated with zero or more instances of class B. Also, the child table is dependent on the parent table, as follows:

- When an instance of the class B is saved, it must be associated with an instance of class A. If you attempt to save the instance, and that association is not defined, the save action fails.
- The association cannot be changed. That is, you cannot associate the instance of class B with a different instance of class A.
- If the instance of class A is deleted, all associated instances of class B are deleted as well.
- You can delete an instance of class B. Class A is not required to have associated instances of class B.

For an example, an invoice class may define a parent-child relationship with a line item class. In this case, an invoice consists of zero or more line items. Those line items cannot be moved to a different invoice. Nor do they have meaning on their own.

Important: Also, in the child table (class B), the IDs are not purely numeric. As a consequence, it is not possible to add a bitmap index to the relationship property in this class, although other forms of index are permitted (and are useful, as shown later in this topic).

25.1.2.1 Parent-Child Relationships and Storage

If you define a parent-child relationship before compiling the classes, the data for both classes is stored in the same global. The data for the children is subordinate to that of the parent, in a structure similar to the following:

```
^Inv(1)
^Inv(1, "invoice", 1)
^Inv(1, "invoice", 2)
^Inv(1, "invoice", 3)
...
```

As a result, InterSystems IRIS can read and write these related objects more quickly.

25.1.3 Common Relationship Terminology

This section explains, by example, phrases that are in common use for convenience when discussing relationships.

Consider a one-to-many relationship between a company and its employees; that is, one company has multiple employees. In this scenario, the company is called the *one side* and the employee is called the *many side*.

Similarly, consider a parent-child relationship between a company and its products; that is, the company is the parent, and the products are the children. In this scenario, the company is called the *parent side* and the product is called the *children side* or the *child side*.

25.2 Defining a Relationship

To create a relationship between the records of two classes, you create a pair of complementary relationship properties, one in each class. To create a relationship between records of the same class, you create a pair of complementary relationship properties in that class.

The following subsections describe the general syntax and then discuss how to define one-to-many relationships and parent-child relationships.

25.2.1 General Syntax

The syntax for a relationship property is as follows:

```
Relationship Name As classname [ Cardinality = cardinality_type, Inverse = inverseProp ];
```

Where:

- *classname* is the class to which this relationship refers. This must be a persistent class.
- *cardinality_type* (required) defines how the relationship appears from this side as well as whether it is an independent relationship (one-to-many) or a dependent relationship (parent-child). *cardinality_type* can be `one`, `many`, `parent`, or `children`.
- *inverseProp* (required) is the name of the complementary relationship property, which is defined in the other class.

In the complementary relationship property, the *cardinality_type* keyword must be the complement of the *cardinality_type* keyword here. The values `one` and `many` are complements of each other. Similarly, the values `parent` and `children` are complements of each other.

Because a relationship is a kind of property, other property keywords are available for use in them, including Final, Required, SqlFieldName, and Private. Some property keywords, such as MultiDimensional, do not apply. See the Class Definition Reference for more information.

25.2.2 Defining a One-to-Many Relationship

This section describes how define a one-to-many relationship between *classA* and *classB*, where one instance of *classA* is associated with zero or more instances of *classB*.

Note: It is possible to have a one-to-many relationship between records of a single class. That is, in the following discussion, *classA* and *classB* can be the same class.

Class A must have a relationship property of the following form:

Class Member

```
Relationship manyProp As classB [ Cardinality = many, Inverse = oneProp ];
```

Where *oneProp* is the name of the complementary relationship property, which is defined in *classB*.

Class B must have a relationship property of the following form:

Class Member

```
Relationship oneProp As classA [ Cardinality = one, Inverse = manyProp ];
```

Where *manyProp* is the name of the complementary relationship property, which is defined in *classA*.

Important: On the one side (class A), the relationship uses a query to populate the relationship object. You can improve the performance of this query in almost all cases by adding an index on the complementary relationship property (that is, adding an index on the many side, class B).

25.2.3 Defining a Parent-Child Relationship

This section describes how define a parent-child relationship between *classA* and *classB*, where one instance of *classA* is the parent of zero or more instances of *classB*. These cannot be the same class.

Class A must have a relationship property of the following form:

Class Member

```
Relationship childProp As classB [ Cardinality = children, Inverse = parentProp ];
```

Where *parentProp* is the name of the complementary relationship property, which is defined in *classB*.

Class B must have a relationship property of the following form:

Class Member

```
Relationship parentProp As classA [ Cardinality = parent, Inverse = childProp ];
```

Where *childProp* is the name of the complementary relationship property, which is defined in *classA*.

Important: On the parent side (class A), the relationship uses a query to populate the relationship object. You can improve the performance of this query in almost all cases by adding an index on the complementary relationship property (that is, adding an index on the child side, class B).

25.2.3.1 Parent-Child Relationships and Compilation

For a parent-child relationship, InterSystems IRIS can generate a storage definition that stores the data for the parent and child objects within a single global, as shown earlier. Such a storage definition improves the speed with which you can access these related objects.

If you add a relationship after compiling the classes, InterSystems IRIS does not generate this optimized storage definition. In such a case, you can delete any test data you might have, delete the storage definitions of the two classes, and then recompile.

25.3 Examples

This section presents examples of a one-to-many relationship and a parent-child relationship.

25.3.1 Example One-to-Many Relationship

This example represents a one-to-many relationship between a company and its employees. The company class is as follows:

Class Definition

```
Class MyApp.Company Extends %Persistent
{

Property Name As %String;

Property Location As %String;

Relationship Employees As MyApp.Employee [ Cardinality = many, Inverse = Employer ];

}
```

And the employee class is as follows:

Class Definition

```
Class MyApp.Employee Extends (%Persistent, %Populate)
{

Property FirstName As %String;

Property LastName As %String;

Relationship Employer As MyApp.Company [ Cardinality = one, Inverse = Employees ];

Index EmployerIndex On Employer;

}
```

25.3.2 Example Parent-Child Relationship

This example represents a parent-child relationship between an invoice and its line items. The invoice class is as follows:

Class Definition

```
Class MyApp.Invoice Extends %Persistent
{

Property Buyer As %String;

Property InvoiceDate As %TimeStamp;

Relationship LineItems As MyApp.LineItem [ Cardinality = children, Inverse = Invoice ];

}
```

And the line item class is as follows:

Class Definition

```
Class MyApp.LineItem Extends %Persistent
{

Property ProductSKU As %String;

Property UnitPrice As %Numeric;

Relationship Invoice As MyApp.Invoice [ Cardinality = parent, Inverse = LineItems ];

Index InvoiceIndex On Invoice;

}
```

25.4 Connecting Objects

A relationship is bidirectional. Specifically, if you update the value of the relationship property in one object, that immediately affects the value of the corresponding relationship property in the related object. As a consequence, you can specify the value for a relationship property in one object, and the effect appears in both objects.

Because the nature of the relationship property is different in the two classes, there are two general scenarios for updating any relationship:

- Scenario 1: The relationship property is a simple reference property. Set the property equal to the appropriate object.
- Scenario 2: The relationship property is an instance of %RelationshipObject, which has an array-like interface. Use methods of that interface to insert objects into the relationship. Note that the objects in the relationship are *not* ordered; the relationship does not retain the order in which you inserted objects into it.

The following subsections give the details. The third subsection describes a variation of Scenario 1 that is especially suitable when you have a large number of objects in the relationship.

The information here describes how to add objects to relationships. The process of modifying objects is similar, with an important exception (by design) in the case of parent-child relationships: Once associated with a particular parent object (and then saved), a child object can never be associated with a different parent.

25.4.1 Scenario 1: Updating the Many or Child Side

On the many side or the child side (*ObjA*), the relationship property is a simple reference property that points to *ObjB*. To connect the objects from this side:

1. Obtain an OREF (*ObjB*) for an instance of the other class. (Either create a new object or open an existing object, as appropriate.)
2. Set the relationship property of *ObjA* equal to *ObjB*.

For an example, consider the example parent-child classes shown earlier. The following steps would update the relationship from the MyApp.LineItem side:

ObjectScript

```
 //obtain an OREF to the invoice class
set invoice=##class(MyApp.Invoice).%New()
//...specify invoice date and so on

set item=##class(MyApp.LineItem).%New()
//...set some properties of this object such as the product name and sale price...

//connect the objects
set item.Invoice=invoice
```

When you call the **%Save()** method for the *item* object, the system saves both objects (*item* and *invoice*).

Also see the last subsection for a variation of this technique.

25.4.2 Scenario 2: Updating the One or Parent Side

On the one side or the parent side, the relationship property is an instance of %RelationshipObject. On this side, you can do the following to connect the objects:

1. Obtain an OREF for an instance of the other object. (Either create a new object or open an existing object, as appropriate.)
2. Call the **Insert()** method of the relationship property on this side and pass that OREF as the argument.

Consider the example parent-child classes shown earlier. For those classes, the following steps would update the relationship from the MyApp.Invoice side:

ObjectScript

```
set invoice=##class(MyApp.Invoice).%OpenId(100034)
//set some properties such as the customer name and invoice date

set item=##class(MyApp.LineItem).%New()
//...set some properties of this object such as the product name and sale price...

//connect the objects
do invoice.LineItems.Insert(item)
```

When you call the **%Save()** method for the *invoice* object, the system saves both objects (*item* and *invoice*).

Important: InterSystems IRIS does not maintain information about the order in which objects are added into the relationship. That is, if you open a previously saved object and use **GetNext()** or similar methods to iterate through a relationship, the order of objects in that relationship is different from when the objects were created.

25.4.3 Fastest Way to Connect Objects

When you need to add a comparatively large number of objects to a relationship, use a variation of the technique given in Scenario 1. In this variation:

1. Obtain an OREF (*ObjA*) for Class A.
2. Obtain the ID for an instance of ClassB.
3. Use the property setter method of the relationship property of *ObjA*, passing the ID as the argument.

   If the relationship property is named *MyRel*, the property setter method is named **MyRelSetObjectId()**.

   (For details on property setter methods, see Using and Overriding Property Methods.

Consider the example classes described in Scenario 1. For those classes, the following steps would insert a large number of invoice items into an invoice (and would do so more rapidly than the technique given in that section):

ObjectScript

```
set invoice=##class(MyApp.Invoice).%New()
//set some properties such as the customer name and invoice date
do invoice.%Save()
set id=invoice.%Id()
kill invoice  //OREF is no longer needed

for index = 1:1:(1000)
 {
   set Item=##class(MyApp.LineItem).%New()
   //set properties of the invoice item

   //connect to the invoice
   do Item.InvoiceSetObjectId(id)
   do Item.%Save()
 }
```

25.5 Removing a Relationship

In the case of a one-to-many relationship, it is possible to remove a relationship between two objects. One way to do so is as follows:

1. Open the instance of the child object (or the object on the many side).
2. Set the applicable property of this object equal to null.

For example, there is a one-to-many relationship between Sample.Company and Sample.Employee. The following demonstrates that the employee whose ID is 101 works for the company whose ID is 5. Notice that this company has four employees:

```
MYNAMESPACE>set e=##class(Sample.Employee).%OpenId(101)

MYNAMESPACE>w e.Company.%Id()
5
MYNAMESPACE>set c=##class(Sample.Company).%OpenId(5)

MYNAMESPACE>w c.Employees.Count()
4
```

Next for this employee, we set the Company property equal to null. Notice that this company now has three employees:

```
MYNAMESPACE>set e.Company=""

MYNAMESPACE>w c.Employees.Count()
3
```

It is also possible to remove the relationship by modifying the other object. In this case, we use the **RemoveAt()** method of the collection property. For example, the following demonstrates that for the company whose ID is 17, the first employee is employee ID 102:

```
MYNAMESPACE>set e=##class(Sample.Employee).%OpenId(102)

MYNAMESPACE>w e.Company.%Id()
17
MYNAMESPACE>set c=##class(Sample.Company).%OpenId(17)

MYNAMESPACE>w c.Employees.Count()
4
MYNAMESPACE>w c.Employees.GetAt(1).%Id()
102
```

To remove the relationship between this company and this employee, we use the **RemoveAt()** method, passing the value 1 as the argument, to remove the first collection item. Notice that after we do so, this company has three employees:

```
MYNAMESPACE>do c.Employees.RemoveAt(1)

MYNAMESPACE>w c.Employees.Count()
3
```

In the case of a parent-child relationship, it is *not* possible to remove a relationship between two objects. You can, however, delete a child object.

25.6 Deleting Objects from Relationships

For a one-to-many relationship, the following rules govern what occurs when you attempt to delete objects:

- The relationship prevents you from deleting an object on the one side, if there are any objects on the many side that reference this object. For example, if you try to delete a company, and the employee table has records that point to that company, the delete operation fails.

  Thus it is necessary to first delete the records on the many side.
- The relationship does not prevent you from deleting an object on the many side (the employee table).

For a parent-child relationship, the rules are different:

- The relationship causes a deletion on the parent side to affect the child side. Specifically, if you delete an object on the parent side, the associated objects on the child side are automatically deleted.

  For example, if there is a parent-child relationship between invoices and line items, if you delete an invoice, its line items are deleted.
- The relationship does not prevent you from deleting an object on the child side (the line item table).

For both one-to-many and parent-child relationships, you can change the default behavior of deleting an object on the one side or the parent side by using the OnDelete property keyword.

25.7 Working with Relationships

Relationships are properties. Relationships with a cardinality of one or parent behave like atomic (non-collection) reference properties. Relationships with a cardinality of many or children are instances of the %RelationshipObject class, which has an array-like interface.

For example, you could use the Company and Employee objects defined above in the following way:

ObjectScript

```
// create a new instance of Company
Set company = ##class(MyApp.Company).%New()
Set company.Name = "Chiaroscuro LLC"

// create a new instance of Employee
Set emp = ##class(MyApp.Employee).%New()
Set emp.LastName = "Weiss"
Set emp.FirstName = "Melanie"

// Now associate Employee with Company
Set emp.Employer = company

// Save the Company (this will save emp as well)
Do company.%Save()

// Close the newly created objects
Set company = ""
Set emp = ""
```

Relationships are fully bidirectional in memory; any operation on either side is immediately visible on the other side. Hence, the code above is equivalent to the following, which instead operates on the company:

ObjectScript

```
Do company.Employees.Insert(emp)

Write emp.Employer.Name
// this will print out "Chiaroscuro LLC"
```

You can load relationships from disk and use them as you would any other property. When you refer to the object on the one side of a relationship from any object on the many side, the object on the one side is automatically swizzled into memory in the same way as a reference (object-valued) property. When you refer to any of the objects on the many side of a relationship from the object on the one side, the objects on the many side are not swizzled immediately; instead a transient %RelationshipObject collection object is created. As soon as any methods are called on this collection, it builds a list containing the ID values of the objects within the relationship. It is only when you refer to one of the objects within this collection that the actual related object is swizzled into memory.

Here is an example that displays all Employee objects related to a specific Company:

ObjectScript

```
// open an instance of Company
Set company = ##class(Company).%OpenId(id)

// iterate over the employees; print their names
Set key = ""

Do {
   Set employee = company.Employees.GetNext(.key)
   If (employee '= "") {
       Write employee.Name,!
   }
} While (key '= "")
```

In this example, closing *company* removes the Company object and all of its related Employee objects from memory. Note, however, that every Employee object contained in the relationship will be swizzled into memory by the time the loop completes. To reduce the amount of memory that this operation uses—perhaps there are thousands of Employee objects—then modify the loop to unswizzle the Employee object after displaying the name, by calling the **%UnSwizzleAt()** method:

ObjectScript

```
Do {
   Set employee = company.Employees.GetNext(.key)
   If (employee '= "") {
       Write employee.Name,!
       // remove employee from memory
       Do company.Employees.%UnSwizzleAt(key)
   }
} While (key '= "")
```

Important: Relationships do not support the list interface. That means you cannot get the count of related objects and iterate over the relationship by incrementing a pointer from one (1) by one (1) up to the number of related objects; instead, you must use array-collection style iteration. For more information on iterating through objects in a relationship, see the reference page for %Library.RelationshipObject.

25.8 SQL Projection of Relationships

A persistent class is projected as an SQL table. This section describes how relationships of such a class are projected to SQL.

Note: Although you can modify the projection of the other properties of the classes involved, it is not possible to modify the SQL projection of relationships per se. For example, it is not supported to specify the *CLASSNAME* property parameter for the relationship. This parameter is mentioned in Defining Object-Valued Properties.

25.8.1 SQL Projection of One-to-Many Relationships

This section describes the SQL projection of a one-to-many relationship. As an example, consider the example one-to-many classes shown earlier. In this case, the classes are projected as follows:

- On the one side (that is, in the company class), there is no field that represents the relationship. The company table has fields for other properties, but there is no field that holds the employees.
- On the many side (that is, in the employee class), the relationship is a simple reference property, and that is projected to SQL in the same way as other reference properties. The employee table has a field named `Employer`, which points to the company table.

To query these tables together, you can query the employee table and use arrow syntax, as in the following example:

SQL

```
SELECT Employer->Name, LastName,FirstName FROM MyApp.Employee
```

Or you can perform an explicit join, as in the following example:

SQL

```
SELECT c.Name, e.LastName, e.FirstName FROM MyApp.Company c, MyApp.Employee e WHERE e.Employer = c.ID
```

Also, this pair of relationship properties implicitly adds a foreign key to the employee table; the foreign key has UPDATE and DELETE both specified as NOACTION.

25.8.2 SQL Projection of Parent-Child Relationships

Similarly, consider the example parent-child classes shown earlier, which have a parent-child relationship between an invoice and its line items. In this case, the classes are projected as follows:

- On the parent side (that is, in the invoice class), there is no field that represents the relationship. The invoice table has fields for other properties, but there is no field that holds the line items.
- On the child side (that is, in the line item class), the relationship is a simple reference property, and that is projected to SQL in the same way as other reference properties. The line item table has a field named `Invoice`, which points to the invoice table.
- Also on the child side, the IDs always include the ID of the parent record, even if you explicitly attempt to create an IDKey based exclusively on the child. Also, if the definition of the IDKey in the child class explicitly includes the parent relationship, the compiler recognizes this and does not add it again; this allows you to alter the sequence in which the parent reference appears as a subscript in the generated global references.

  As a consequence, it is not possible to add a bitmap index to this property, although other forms of index are permitted.

To query these tables together, you can query the invoice table and use arrow syntax, as in the following example:

SQL

```
SELECT
Invoice->Buyer, Invoice->InvoiceDate, ID, ProductSKU, UnitPrice
FROM MyApp.LineItem
```

Or you can perform an explicit join, as in the following example:

SQL

```
SELECT
i.Buyer, i.InvoiceDate, l.ProductSKU,l.UnitPrice
FROM MyApp.Invoice i, MyApp.LineItem l
WHERE i.ID = l.Invoice
```

Also, for the class on the child side, the projected table is adopted as a child table of the other table.

25.9 Creating Many-to-Many Relationships

InterSystems IRIS does not directly support many-to-many relationships, but this section describes how to model such a relationship indirectly.

To establish a many-to-many relationship between class A and class B, do the following:

1. Create a intermediate class that will define each relationship.
2. Define a one-to-many relationship between that class and class A.
3. Define a one-to-many relationship between that class and class B.

Then, create a record in the intermediate class for each relationship between an instance of class A and an instance of class B.

For example, suppose that class A defines doctors; this class defines the properties Name and Specialty. Class B defines patients; this class defines the properties Name and Address. To model the many-to-many relationship between doctors and patients, we could define an intermediate class as follows:

Class Definition

```
/// Bridge class between MN.Doctor and MN.Patient
Class MN.DoctorPatient Extends %Persistent
{

Relationship Doctor As MN.Doctor [ Cardinality = one, Inverse = Bridge ];

Index DoctorIndex On Doctor;

Relationship Patient As MN.Patient [ Cardinality = one, Inverse = Bridge ];

Index PatientIndex On Patient;
}
```

Then the doctor class looks like this:

Class Definition

```
Class MN.Doctor Extends %Persistent
{

Property Name;

Property Specialty;

Relationship Bridge As MN.DoctorPatient [ Cardinality = many, Inverse = Doctor ];

}
```

And the patient class looks like this:

Class Definition

```
Class MN.Patient Extends %Persistent
{

Property Name;

Property Address;

Relationship Bridge As MN.DoctorPatient [ Cardinality = many, Inverse = Patient ];

}
```

The easiest way to query both doctors and patients is to query the intermediate table. The following shows an example:

```
SELECT top 20 Doctor->Name as Doctor, Doctor->Specialty, Patient->Name as Patient
FROM MN.DoctorPatient order by doctor

Doctor  Specialty       Patient
Davis,Joshua M. Dermatologist   Wilson,Josephine J.
Davis,Joshua M. Dermatologist   LaRocca,William O.
Davis,Joshua M. Dermatologist   Dunlap,Joe K.
Davis,Joshua M. Dermatologist   Rotterman,Edward T.
Davis,Joshua M. Dermatologist   Gibbs,Keith W.
Davis,Joshua M. Dermatologist   Black,Charlotte P.
Davis,Joshua M. Dermatologist   Dunlap,Joe K.
Davis,Joshua M. Dermatologist   Rotterman,Edward T.
Li,Umberto R.   Internist       Smith,Wolfgang J.
```

```
Li,Umberto R.   Internist       Ulman,Mo O.
Li,Umberto R.   Internist       Gibbs,Keith W.
Li,Umberto R.   Internist       Dunlap,Joe K.
Quixote,William Q.      Surgeon Black,Charlotte P.
Quixote,William Q.      Surgeon LaRocca,William O.
Quixote,William Q.      Surgeon Black,Charlotte P.
Quixote,William Q.      Surgeon Smith,Wolfgang J.
Quixote,William Q.      Surgeon LaRocca,William O.
Quixote,William Q.      Surgeon LaRocca,William O.
Quixote,William Q.      Surgeon Black,Charlotte P.
Salm,Jocelyn Q. Allergist       Tsatsulin,Mark S.
```

As a variation, you can use a parent-child relationship in place of one of the one-to-many relationships. This provides the physical clustering of the data, but it means that you cannot use a bitmap index on that relationship.

25.9.1 Variation with Foreign Keys

Rather than defining relationships between the intermediate class and classes A and B, you can use reference properties and foreign keys, so that the intermediate class `MN.DoctorPatient` looks like this instead of the version shown previously:

Class Definition

```
Class MN.DoctorPatient Extends %Persistent
{

Property Doctor As MN.Doctor;

ForeignKey DoctorFK(Doctor) References MN.Doctor();

Property Patient As MN.Patient;

ForeignKey PatientFK(Patient) References MN.Patient();

}
```

Foreign keys are discussed in more detail in Using Foreign Keys and Syntax of Foreign Key Definitions.

One advantage to using a simple foreign key model is that no inadvertent swizzling of large numbers of objects will occur. One disadvantage is that no automatic swizzling is available.

25.10 See Also

- Defining and Using Properties
- Introduction to Persistent Objects
- Working with Persistent Objects
- Property Syntax and Keywords

26

Other Options for Persistent Classes

This topic describes other options that are available for persistent classes.

26.1 Defining a Sharded Class

If you are using sharding for horizontal scaling of data storage, you can define a sharded class by using the *Sharded* class keyword in the class definition. A sharded class is a persistent class where the data is spread among the data nodes of a sharded cluster, while the application accesses the data as if it were local.

Set the keyword `Sharded = 1` to create a sharded class, as in the following example:

```
Class MyApp.Person Extends %Persistent [ Sharded = 1 ]
```

Until sharded classes are fully implemented, InterSystems recommends creating sharded tables from SQL, not from the object side.

For more information on sharding, see Horizontally Scaling InterSystems IRIS for Data Volume with Sharding.

26.2 Defining a Read-Only Class

It is possible to define a persistent class whose objects can be opened but not saved or deleted. To do this, specify the *READONLY* parameter for the class as 1:

Class Member

```
Parameter READONLY = 1;
```

This is only useful for cases where you have objects that are mapped to preexisting storage (such as existing globals or an external database). If you call the **%Save()** method on a read-only object, it will always return an error code.

26.3 Adding Indexes

Indexes provide a mechanism for optimizing searches across the instances of a persistent class; they define a specific sorted subset of commonly requested data associated with a class. They are very helpful in reducing overhead for performance-critical searches.

Indexes automatically span the entire extent of the class in which they are defined. If a Person class has a subclass Student, all indexes defined in Person contain both Person objects and Student objects. Indexes defined in the Student class contain only Student objects.

Indexes can be sorted on one or more properties belonging to their class. This allows you a great deal of specific control of the order in which results are returned.

In addition, indexes can store additional data that is frequently requested by queries based on the sorted properties. By including additional data as part of an index, you can greatly enhance the performance of the query that uses the index; when the query uses the index to generate its result set, it can do so without accessing the main data storage facility. (See the Data keyword below.)

Note: The only way to simultaneously add an index definition to the class and build the data in the index is by using an ALTER TABLE, CREATE INDEX, or DROP INDEX statement. When adding an index to a class definition, the index must be manually built. To read about building indexes manually, refer to Building Indexes.

For additional information on indexes, refer to Defining and Building Indexes, especially the section Properties That Can Be Indexed. Also see Syntax of Index Definitions.

Important: Indices are inherited only from the primary superclass.

26.4 Adding Foreign Keys

To enforce referential integrity between tables you can define foreign keys in the corresponding persistent classes. When a table containing a foreign key constraint is modified, the foreign key constraints are checked. One way to add foreign keys is to add relationships between classes; see Defining and Using Relationships. You can also add explicit foreign keys to classes. For information, see Using Foreign Keys and Syntax of Foreign Key Definitions.

26.5 Adding Triggers

Because InterSystems SQL supports the use of triggers, any trigger associated with a persistent class is included as part of the SQL projection of the class.

Triggers are code segments executed when specific events occur in InterSystems SQL. InterSystems IRIS® data platform supports triggers based on the execution of **INSERT**, **UPDATE**, and **DELETE** commands. The specified code will be executed either immediately before or immediately after the relevant command is executed, depending on the trigger definition. Each event can have multiple triggers as long as they are assigned an execution order.

If a trigger is defined with `Foreach = row/object`, then the trigger is also called at specific points during object access. See Triggers and Transactions.

Triggers are also fired by the persistence methods used by the legacy storage class, %Storage.SQL because it uses SQL statements internally to implement its persistent behavior.

For more information on triggers, see Triggers and Trigger Definitions.

26.6 Referring to Fields from ObjectScript

Within a class definition, there are several places that may include ObjectScript code used in SQL. For example, SQL computed field code and trigger code is executed from within SQL. In these cases, there is no concept of a current object, so it is not possible to use dot syntax to access or set data within a specific instance. Instead, you can access the same data as fields within the current row using field syntax.

To reference a specific field of the current row, use the `{fieldname}` syntax where *fieldname* is the name of the field.

For example, the following code checks if the salary of an employee is less than 50000:

```
If {Salary} < 50000 {
   // actions here...
}
```

Note: In UPDATE trigger code, `{fieldname}` denotes the updated field value. In DELETE trigger code, `{fieldname}` denotes the value of the field on disk.

To refer to the current field in a SQL computed field, use the `{*}` syntax.

For example, the following code might appear in the computed code for a Compensation field to compute its value based on the values of Salary and Commission fields:

```
Set {*} = {Salary} + {Commission}
```

For trigger-specific syntax, see Special Trigger Syntax.

26.7 See Also

- Introduction to Persistent Objects
- Working with Persistent Objects
- Defining Persistent Classes
- Using the Object Synchronization Feature

27

Defining Method and Trigger Generators

A method generator is a specific kind of method that generates its own runtime code. Similarly, a trigger generator is a trigger that generates its own runtime code.

This topic primarily discusses method generators, but the details are similar for trigger generators.

27.1 Introduction

A powerful feature of InterSystems IRIS® data platform is the ability to define method generators: small programs that are invoked by the class compiler to generate the runtime code for a method. Similarly a trigger generator is invoked by the class compiler and generates the runtime code for a trigger.

Method generators are used extensively within the InterSystems IRIS class library. For example, most of the methods of the %Persistent class are implemented as method generators. This makes it possible to give each persistent class customized storage code, instead of less efficient, generic code. Most of the InterSystems IRIS data type class methods are also implemented as method generators. Again, this gives these classes the ability to provide custom implementations that depend on the context in which they are used.

You can use method and trigger generators within your own applications. For method generators, a common usage is to define one or more utility superclasses that provide specialized methods for the subclasses that use them. The method generators within these utility classes create special code based on the definition (properties, methods, parameter values, etc.) of the class that uses them. Good examples of this technique are the %Populate and %XML.Adaptor classes provided within the InterSystems IRIS library.

27.2 Basics

A method generator is simply a method of an InterSystems IRIS class that has its CodeMode keyword set to `objectgenerator`:

Class Definition

```
Class MyApp.MyClass Extends %RegisteredObject
{
Method MyMethod() [ CodeMode = objectgenerator ]
    {
        Do %code.WriteLine(" Write """ _ %class.Name _ """")
        Do %code.WriteLine(" Quit")
        Quit $$$OK
    }
}
```

When the class MyApp.MyClass is compiled, it ends up with a **MyMethod** method with the following implementation:

ObjectScript

```
 Write "MyApp.MyClass"
 Quit
```

You can also define trigger generators. To do so, use CodeMode = `objectgenerator` in the definition of a trigger. The values available within your trigger are slightly different than those in a method generator.

27.3 How Generators Work

A method generator takes effect when you compile a class. The operation of a method generator is straightforward. When you compile a class definition, the class compiler does the following:

1. It resolves inheritance for the class (builds a list of all inherited members).
2. It makes a list of all methods specified as method generators (by looking at the CodeMode keyword of each method).
3. It gathers the code from all method generators, copies it into one or more temporary routines, and compiles them (this makes it possible to execute the method generator code).
4. It creates a set of transient objects that represent the definition of the class being compiled. These objects are made available to the method generator code.
5. It executes the code for every method generator.

   If present, the compiler will arrange the order in which it invokes the method generators by looking at the value of the GenerateAfter keyword for each of the methods. This keyword gives you some control in cases where there may be compiler timing dependencies among methods.
6. It copies the results of each method generator (lines of code plus any changes to other method keywords) into the compiled class structure (used to generate the actual code for the class).

   Note that the original method signature (arguments and return type), as well as any method keyword values, are used for the generated method. If you specify a method generator as having a return type of %Integer, then the actual method will have a return type of %Integer.
7. It generates the executable code for the class by combining the code generated by the method generators along with the code from all the non-method generator methods.

The details are similar for trigger generators.

27.4 Values Available to Method Generators

The key to implementing method generators is understanding the context in which method generator code is executed. As described in the previous section, the class compiler invokes the method generator code at the point after it has resolved class inheritance but before it has generated code for the class. When it invokes method generator code, the class compiler makes the following variables available to the method generator code:

Table 27–1: Variables Available to Method Generators

Variable	Description
%code	An instance of the %Stream.MethodGenerator class. This is a stream into which you write the code for the method.
%class	An instance of the %Dictionary.ClassDefinition class. It contains the original definition of the class.
%method	An instance of the %Dictionary.MethodDefinition class. It contains the original definition of the method.
%compiledclass	An instance of the %Dictionary.CompiledClass class. It contains the *compiled* definition of the class being compiled. Thus, it contains information about the class after inheritance has been resolved (such as the list of all properties and methods, including those inherited from superclasses).
%compiledmethod or *%objcompiledmethod*	An instance of the %Dictionary class for the compiled method, for example,%Dictionary.CompiledMethod, %Dictionary.CompiledPropertyMethod or%Dictionary.CompiledIndexMethod. It contains the *compiled* definition of the method being generated.
%parameter	An array that contains the values of any class parameters indexed by parameter name. For example, *%parameter("MYPARAM")*, contains the value of the *MYPARAM* class parameter for the current class. This variable is provided as an easier alternative to using the list of parameter definitions available via the *%class* object.
%kind or *%membertype*	For member methods, the kind of class member that relates to this method, for example, a for property methods or `i` for index methods.
%mode	The type of method, for example, `method`, `propertymethod`, or `indexmethod`.
%pqname or *%member*	For member methods, the name of the class member that relates to this method.

27.5 Values Available to Trigger Generators

Like methods, triggers can be defined as generators. That is, you can use CodeMode = `objectgenerator` in the definition of a trigger. The following variables are available within the trigger generator:

Table 27–2: Added Variables Available to Trigger Generators

Variable	Description
%code, *%class*, *%compiledclass*, and *%parameter*	See the preceding section.
%trigger	An instance of the %Dictionary.TriggerDefinition class. It contains the original definition of the trigger.
%compiledtrigger or *%objcompiledmethod*	An instance of the %Dictionary.CompiledTrigger class. It contains the *compiled* definition of the trigger being generated.
%kind or *%membertype*	For triggers, this is the value `t`.
%mode	For triggers, this is the value `trigger`.
%pqname or *%member*	The name of this trigger.

27.6 Defining Method Generators

To define a method generator, do the following:

1. Define a method and set its CodeMode keyword to `objectgenerator`.
2. In the body of the method, write code that generates the actual method code when the class is compiled. This code uses the *%code* object to write out the code. It will most likely use the other available objects as inputs to decide what code to generate.

The following is an example of a method generator that creates a method that lists the names of all the properties of the class it belongs to:

Class Member

```
ClassMethod ListProperties() [ CodeMode = objectgenerator ]
{
    For i = 1:1:%compiledclass.Properties.Count() {
        Set prop = %compiledclass.Properties.GetAt(i).Name
        Do %code.WriteLine(" Write """ _ prop _ """,!")
    }
    Do %code.WriteLine(" Quit")
    Quit $$$OK
}
```

This generator will create a method with an implementation similar to:

ObjectScript

```
 Write "Name",!
 Write "SSN",!
 Quit
```

Note the following about the method generator code:

1. It uses the **WriteLine** method of the *%code* object to write lines of code to a stream containing the actual implementation for the method. (You can also use the **Write** method to write text without an end-of-line character).

2. Each line of generated code has a leading space character. This is required because ObjectScript does not allow commands within the first space of a line. This would not be the case if our method generator is creating Basic or Java code.
3. As the lines of generated code appear within strings, you have to be very careful about escaping quotation mark characters by doubling them up ("").
4. To find the list of properties for the class, it uses the *%compiledclass* object. It could use the *%class* object, but then it would only list properties defined within the class being compiled; it would not list inherited properties.
5. It returns a status code of $$$OK, indicating that the *method generator* ran successfully. This return value has nothing to do with the actual implementation of the method.

27.6.1 Specifying CodeMode within a Method Generator

By default, a method generator will create a code method (that is, the CodeMode keyword for the generated method is set to `code`). You can change this using the CodeMode property of the *%code* object.

For example, the following method generator will generate an ObjectScript expression method:

Class Member

```
Method Double(%val As %Integer) As %Integer [ CodeMode = objectgenerator ]
{
    Set %code.CodeMode = "expression"
    Do %code.WriteLine("%val * 2")
}
```

27.7 Generators and INT Code

For method and trigger generators, it can be very useful to display the corresponding INT code in your Integrated Development Environment (IDE) after compiling the class. If using InterSystems Studio, from the **View** menu, select **View Other Code**. If using InterSystems ObjectScript extension for Visual Studio Code, right click the class file in the editor and select **View Other**.

Note that if the generator is simple enough to be implemented in the kernel, no .INT code is generated for it.

27.8 Effect on Subclasses

This section discusses topics specific to generator methods in subclasses of the class in which they were defined.

It is necessary, of course, to compile any subclasses after compiling the superclass.

27.8.1 Method Regeneration in Subclasses

When you subclass a class that defines generator methods, InterSystems IRIS uses the same compilation rules that are described earlier. InterSystems IRIS does not, however, recompile a method in a subclass if the generated code looks the same as the superclass generated code. This logic does not consider whether the include files are the same for both classes. If the method uses a macro that is defined in an include file and if the subclass uses a different include file, InterSystems IRIS would not recompile the method in the subclass. You can, however, force the generator method to be recompiled in every class. To do so, specify the method keyword ForceGenerate for that method. There may be additional scenarios where this keyword is needed.

27.8.2 Invoking the Method in the Superclass

If you need a subclass to use the method generated for the superclass, rather than a locally generated method, do the following in the subclass: define the generator method so that it just returns $$$OK, as in the following example:

Class Member

```
ClassMethod Demo1() [ CodeMode = objectgenerator ]
{
    quit $$$OK
}
```

27.8.3 Removing a Generated Method

You can remove a generated method from a subclass, so that it cannot be invoked in that class. To do so, when you define the generator method in the superclass, include logic that examines the name of the current class and generates code only in the desired scenarios. For example:

```
ClassMethod Demo3() [ CodeMode = objectgenerator ]
{
    if %class.Name="RemovingMethod.ClassA" {
        Do %code.WriteLine(" Write !,""Hello from class: " _ %class.Name _ """")
    }
    quit $$$OK
}
```

If you try to invoke this method in any subclass, you receive the error <METHOD DOES NOT EXIST>.

Note that this logic is subtly different from that described in the previous section. If a generator method in a given class exists but has a null implementation, the method of the superclass, if any, is used instead. But if a generator method in a given class does not generate code for a given subclass, the method does not exist in that subclass and cannot be invoked.

27.9 See Also

- Defining and Calling Methods
- Adding Triggers

28
Defining and Using Class Queries

This page describes how to define and use *class queries*, which are a form of dynamic SQL to be used via the %SQL classes.

28.1 Introduction to Class Queries

A class query is a tool — contained in a class and meant for use with dynamic SQL — to look up records that meet specified criteria. With class queries, you can create predefined lookups for your application. For example, you can look up records by name, or provide a list of records that meet a particular set of conditions, such as all the flights from Paris to Madrid.

By creating a class query, you can avoid having to look up a particular object by its internal ID. Instead, you can create a query that looks up records based on any class properties that you want. These can even be specified from user input at runtime.

If you define a custom class query, your lookup logic can use ObjectScript and can be arbitrarily complex.

There are two kinds of class queries:

- Basic class queries, which use the class %SQLQuery and an SQL SELECT statement.
- Custom class queries, which use the class %Query and custom logic to execute, fetch, and close the query. These are discussed on another page.

Note that you can define class queries within *any* class; there is no requirement to contain them within persistent classes.

Important: You can define a class query that depends upon the results of another class query, but the ROWSPEC parameter must be specified in the queries.

28.2 Using Class Queries

Before looking at how to define class queries, it is useful to see how you can use them. In server-side code, you can use a class query as follows:

1. Use **%New()** to create an instance of %SQL.Statement.
2. Call the **%PrepareClassQuery()** method of that instance. As arguments, use the following, in order:
   a. Fully qualified name of the class that defines the query that you want to use.
   b. Name of the query in that class.

This method returns a %Status value, which you should check.

3. Call the **%Execute()** method of the %SQL.Statement instance. The **%Execute()** accepts any parameters needed by the query. Specify these in the order required by the query.

   This returns an instance of %SQL.StatementResult.

4. Use methods of %SQL.StatementResult to retrieve data from the result set. For details, see Dynamic SQL.

Suppose that Sample.Person has a query named **ByName**, which in turn accepts a string value; this query returns a result set containing people whose names include the input string.

The following shows a simple example that uses the **ByName** query of Sample.Person; error checking is not shown:

ObjectScript

```
set statement = ##class(%SQL.Statement).%New()
set status = statement.%PrepareClassQuery("Sample.Person","ByName")
set rset = statement.%Execute("Jo") //Look for names containing Jo

while rset.%Next() {
  write !, rset.%Get("Name")
}
```

You can also invoke the query as a stored procedure, thus executing it from an SQL context. See Defining and Using Stored Procedures.

28.3 Defining Basic Class Queries

To define a basic class query, define a query as follows:

- (For simple class queries) The type should be %SQLQuery.
- Optionally specify parameters of %SQLQuery, such as *ROWSPEC*. The *ROWSPEC* parameter for a query provides information on the names, data types, headings, and order of the fields in each row. For more information, see Parameters of the Class Query.
- In the argument list, specify any arguments that the query should accept.
- In the body of the definition, write an SQL SELECT statement.

  In this statement, to refer to an argument, precede the argument name with a colon (`:`).

  This SELECT statement should not include an INTO clause.
- Include the SqlProc keyword in the query definition.

  This causes the class query to be projected as a stored procedure, so that it can be visible via the INFORMATION_SCHEMA.Routines and INFORMATION_SCHEMA.Parameters tables.
- Optionally specify the SqlName keyword in the query definition, if you want the name of the stored procedure to be other than the default name.

  These are compiler keywords, so include them in square brackets after any parameters, after the query type (%SQLQuery).

28.4 Parameters of the Class Query

This section provides more details on the parameters you specify within a custom class query.

28.4.1 About ROWSPEC

The *ROWSPEC* parameter for a query provides information on the names, data types, headings, and order of the fields in each row. It is a quoted and comma-separated list of variable names and data types of the form:

```
ROWSPEC = "Var1:%Type1,Var2:%Type2[:OptionalDescription],Var3"
```

The *ROWSPEC* specifies the order of fields as a comma-separated list. The information for each field consists of a colon-separated list of its name, its data type (if it is different than the data type of the corresponding property), and an optional heading.

The number of elements in the *ROWSPEC* parameter must match the number of fields in the query. Otherwise, InterSystems IRIS® data platform returns a Cardinality Mismatch error.

For an example, the **ByName** query of the Sample.Person class is as follows:

Class Member

```
Query ByName(name As %String = "")
    As %SQLQuery(CONTAINID = 1, ROWSPEC = "ID:%Integer,Name,DOB,SSN", SELECTMODE = "RUNTIME")
   [ SqlName = SP_Sample_By_Name, SqlProc ]
{
        SELECT ID, Name, DOB, SSN
            FROM Sample.Person
            WHERE (Name %STARTSWITH :name)
            ORDER BY Name
}
```

Here, the *CONTAINID* parameter specifies that the row ID is the first field (the default); note that the first field specified in the SELECT statement is *ID*. The *ROWSPEC* parameter specifies that the fields are *ID* (treated as an integer), *Name*, *DOB*, and *SSN*; similarly, the SELECT statement contains the fields *ID*, *Name*, *DOB*, and *SSN*, in that order.

28.4.2 About CONTAINID

CONTAINID should be set to the number of the column returning the ID (1, by default) or to 0 if no column returns the ID.

Note: The system does not validate the value of *CONTAINID*. If you specify a non-valid value for this parameter, there is no error message. This means that if your query processing logic depends on this information, you may experience inconsistencies if the *CONTAINID* parameter is set improperly.

28.4.3 Other Parameters of the Query Class

In addition to *ROWSPEC* and *CONTAINID*, you can specify the following parameters of the query. These are class parameters for %SQLQuery:

- *SELECTMODE*
- *COMPILEMODE*

For details, see the class reference for %Library.SQLQuery and %Library.Query (its superclass).

28.5 Example

The following shows a simple example:

Class Member

```
Query ListEmployees(City As %String = "")
   As %SQLQuery (ROWSPEC="ID:%Integer,Name:%String,Title:%String", CONTAINID = 1) [SqlProc,
SqlName=MyProcedureName]
{
SELECT ID,Name,Title FROM Employee
 WHERE (Home_City %STARTSWITH :City)
 ORDER BY Name
}
```

28.6 Maximum Length of String Parameters

If you call a class query using ADO.NET, ODBC, or JDBC, any string parameters will be truncated to 50 characters by default. To increase the maximum string length for a parameter, specify a *MAXLEN* in the signature, as in the following example:

```
Query MyQuery(MyParam As %String(MAXLEN = 200)) As %SQLQuery [SqlProc]
```

This truncation does not occur if you call the query from the Management Portal or from ObjectScript.

28.7 See Also

- Defining Custom Class Queries
- Defining Classes

29

Defining Custom Class Queries

Class queries are a form of dynamic SQL to be used via the %SQL classes. Basic class queries perform all result set management for you, but in some cases, you may need custom code. This page describes how to define *custom class queries*, which are defined by a set of methods that work together.

Note: Custom class queries are not supported for sharded classes.

29.1 Defining Custom Class Queries

To define a custom query, use the instructions for defining basic class queries, with the following changes:

- Specify %Query for the query type.
- Specify the *ROWSPEC* parameter of the query (in parentheses, after the query type). This parameter provides information on the names, data types, headings, and order of the fields in each row of the result set of the query. See About ROWSPEC.
- Optionally specify the *CONTAINID* parameter of the query (in parentheses, after the query type). This parameter specifies the column number of the field, if any, that contains the ID for a particular row; the default is 1. See About CONTAINID.

  Together, the *ROWSPEC* and *CONTAINID* parameters are known as the *query specification*.
- Leave the body of the query definition empty. For example:

  Class Member

  ```
  Query All() As %Query(CONTAINID = 1, ROWSPEC = "Title:%String,Author:%String")
  {
  }
  ```

- Define the following class methods in the same class:
  - *queryname***Execute** — This method must perform any one-time setup.
  - *queryname***Fetch** — This method must return a row of the result set; each subsequent call returns the next row.
  - *queryname***Close** — This method must perform any cleanup operations.

  Where *queryname* is the name of the query.

  Each of these methods accepts an argument (*qHandle*), which is passed by reference. You can use this argument to pass information among these methods.

These methods define the query. The next section provides details.

These methods by default are written in ObjectScript.

29.2 Defining the Methods

For basic demonstration purposes, this section shows a simple example that *could* also be implemented as a basic class query. These methods implement the code for the following query:

Class Member

```
Query AllPersons() As %Query(ROWSPEC = "ID:%String,Name:%String,DOB:%String,SSN:%String")
{
}
```

The next section shows a more complex example. Also see Uses of Custom Queries, for information on other use cases.

29.2.1 Defining the querynameExecute() Method

The **querynameExecute()** method must provide all the setup logic needed. The name of the method must be *queryname***Execute**, where *queryname* is the name of the query. This method must have the following signature:

```
ClassMethod queryNameExecute(ByRef qHandle As %Binary,
                             additional_arguments) As %Status
```

Where:

- *qHandle* is used to communicate with the other methods that implement this query.

  This method should set *qHandle* as needed by the *queryname***Fetch** method.

  Although *qHandle* is formally of type `%Binary`, it can hold any value, including an OREF or a multidimensional array.

- *additional_arguments* is any runtime parameters that the query can use.

Within this implementation of method, use the following general logic:

1. Perform any one-time setup steps.

   For queries using SQL code, this method typically includes declaring and opening a cursor.

2. Set *qHandle* as needed by the *queryname***Fetch** method.

3. Return a status value.

The following shows a simple example, the **AllPersonsExecute()** method for the **AllPersons** query introduced earlier:

Class Member

```
ClassMethod AllPersonsExecute(ByRef qHandle As %Binary) As %Status
{
    set statement=##class(%SQL.Statement).%New()
    set status=statement.%PrepareClassQuery("Sample.Person","ByName")
    if $$$ISERR(status) { quit status }
    set resultset=statement.%Execute()
    set qHandle=resultset
    Quit $$$OK
}
```

In this scenario, the method sets *qHandle* equal to an OREF, specifically an instance of %SQL.StatementResult, which is the value returned by the **%Execute()** method.

As noted earlier, this class query could also be implemented as a basic class query rather than a custom class query. Some custom class queries do, however, use dynamic SQL as a starting point.

29.2.2 Defining the querynameFetch() Method

The **querynameFetch()** method must return a single row of data in **$List** format. The name of the method must be *queryname***Fetch**, where *queryname* is the name of the query. This method must have the following signature:

```
ClassMethod queryNameFetch(ByRef qHandle As %Binary,
                           ByRef Row As %List,
                           ByRef AtEnd As %Integer = 0) As %Status [ PlaceAfter = querynameExecute ]
```

Where:

- *qHandle* is used to communicate with the other methods that implement this query.

  When InterSystems IRIS® data platform starts executing this method, *qHandle* has the value established by the *queryname***Execute** method or by the previous invocation (if any) of this method. This method should set *qHandle* as needed by subsequent logic.

  Although *qHandle* is formally of type %Binary, it can hold any value, including an OREF or a multidimensional array.
- *Row* must be either a %List of values representing a row of data being returned or a null string if no data is returned.
- *AtEnd* must be 1 when the last row of data has been reached.
- The PlaceAfter method keyword controls the position of this method in the generated routine code. For *querynameExecute*, substitute the name of the specific **querynameExecute()** method. Be sure to include this if your query uses SQL cursors. (The ability to control this order is an advanced feature that should be used with caution. InterSystems does not recommend general use of this keyword.)

Within this implementation of method, use the following general logic:

1. Check to determine if it should return any more results.
2. If appropriate, retrieve a row of data and create a %List object and place that in the *Row* variable.
3. Set *qHandle* as needed by subsequent invocations (if any) of this method or needed by the **querynameClose()** method.
4. If no more rows exist, set *Row* to a null string and set *AtEnd* to 1.
5. Return a status value.

For the **AllPersons** example, the **AllPersonsFetch()** method could be as follows:

Class Member

```
ClassMethod AllPersonsFetch(ByRef qHandle As %Binary, ByRef Row As %List, ByRef AtEnd As %Integer = 0)
 As %Status
[ PlaceAfter = AllPersonsExecute ]
{
  set rset = $get(qHandle)
  if rset = "" quit $$$OK

  if rset.%Next() {
    set Row=$lb(rset.%GetData(1),rset.%GetData(2),rset.%GetData(3),rset.%GetData(4))
    set AtEnd=0
  } else {
    if (rset.%SQLCODE < 0) {write "%Next failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message
quit}
    set Row=""
    set AtEnd=1
  }
    quit $$$OK
}
```

Notice that this method uses the *qHandle* argument, which provides a %SQL.StatementResult object. The method then uses methods of that class to retrieve data. The method builds a $List and places that in the *Row* variable, which is returned as a single row of data. Also notice that the method contains logic to set the *AtEnd* variable when no more data can be retrieved.

As noted earlier, this class query could also be implemented as a basic class query rather than a custom class query. The purpose of this example is to demonstrate setting the *Row* and *AtEnd* variables.

29.2.3 The querynameClose() Method

The **querynameClose()** method must perform any needed clean up, after data retrieval has finished. The name of the method must be *queryname***Close**, where *queryname* is the name of the query. This method must have the following signature:

```
ClassMethod queryNameClose(ByRef qHandle As %Binary) As %Status [ PlaceAfter = querynameFetch ]
```

Where:

- *qHandle* is used to communicate with the other methods that implement this query.

  When InterSystems IRIS starts executing this method, *qHandle* has the value established by the last invocation of the *queryname***Fetch** method.

- The PlaceAfter method keyword controls the position of this method in the generated routine code. For *querynameFetch*, substitute the name of the specific **querynameFetch()** method. Be sure to include this if your query uses SQL cursors. (The ability to control this order is an advanced feature that should be used with caution. InterSystems does not recommend general use of this keyword.)

Within this implementation of method, remove variables from memory, close any SQL cursors, or perform any other cleanup as needed. The method must return a status value.

For the **AllPersons** example, the **AllPersonsClose()** method could be as follows:

For example, the signature of a **ByNameClose()** method might be:

Class Member

```
ClassMethod AllPersonsClose(ByRef qHandle As %Binary) As %Status [ PlaceAfter = AllPersonsFetch ]
{
        Set qHandle=""
        Quit $$$OK
}
```

29.2.4 Generated Methods for Custom Queries

The system automatically generates the **querynameGetInfo()** and **querynameFetchRows()**. Your application does not call any of these methods directly.

29.3 Defining Parameters for Custom Queries

If the custom query should accept parameters, do the following:

- Include them in the argument list of the query class member. The following example uses a parameter named `MyParam`:

  Class Member

  ```
  Query All(MyParam As %String) As %Query(CONTAINID = 1, ROWSPEC = "Title:%String,Author:%String")
  {
  }
  ```

- Include the same parameters in the argument list for *queryname***Execute** method, in the same order as in the query class member.
- In the implementation of the *queryname***Execute** method, use the parameters as appropriate for your needs.

Note: If you call a class query using ADO.NET, ODBC, or JDBC, any string parameters will be truncated to 50 characters by default. To increase the maximum string length for a parameter, specify a *MAXLEN* in the signature, as in the following example:

```
Query MyQuery(MyParam As %String(MAXLEN = 200)) As %Query [SqlProc]
```

This truncation does not occur if you call the query from the Management Portal or from ObjectScript.

29.4 When to Use Custom Queries

The following list suggests some scenarios when custom queries are appropriate:

- If it is necessary to use very complex logic to determine whether to include a specific row in the returned data. The **querynameFetch()** method can contain arbitrarily complex logic.
- If you have an API that returns data in format that is inconvenient for your current use case. In such a scenario, you would define the **querynameFetch()** method so that converts data from that format into a $List, as needed by the *Row* variable.
- If the data is stored in a global that does not have a class interface.
- If access to the data requires role escalation. In this scenario, you can perform the role escalation within the **querynameExecute()** method.
- If access to the data requires calling out to the file system (for example, when building a list of files). In this scenario, you can perform the callout within the **querynameExecute()** method and then stash the results either in *qHandle* or in a global.
- If it is necessary to perform a security check, check connections, or perform some other special setup work before retrieving data. You would do such work within the **querynameExecute()** method.

29.5 SQL Cursors and Class Queries

If a class query uses an SQL cursor, note the following points:

- Cursors generated from queries of type `%SQLQuery` automatically have names such as Q14.

  You must ensure that your cursors are given distinct names.

- Error messages refer to the internal cursor name, which typically has an extra digit. Therefore an error message for cursor Q140 probably refers to Q14.

- The class compiler must find a cursor declaration before making any attempt to use the cursor. This means that you must take extra care when defining a custom query that uses cursors.

  The DECLARE statement (usually in **querynameExecute()** method) must be in the same MAC routine as the Close and Fetch and must come before either of them. As shown earlier in this topic, use the method keyword PlaceAfter in both the **querynameFetch()** and **querynameClose()** method definitions to make sure this happens.

29.6 See Also

- Defining and Using Class Queries
- Defining Classes

30

Defining and Using XData Blocks

An *XData block* is a class member that consists of a name and a unit of data that you include in a class definition for use by the class after compilation.

30.1 Basics

An *XData block* is a named unit of data that you include in a class definition, typically for use by a method in the class. Most frequently, it is a well-formed XML document, but it could consist of other forms of data, such as JSON or YAML.

You can create an XData block by typing it directly in your Integrated Development Environment (IDE).

An XData block is a named class member (like properties, methods, and so on). The available XData block keywords include:

- SchemaSpec — Optionally specifies an XML schema against which the XData can be validated.
- XMLNamespace — Optionally specifies the XML namespace to which the XData block belongs. You can also, of course, include namespace declarations within the XData block itself.
- MimeType — The MIME type (more formally, the Internet media type) of the contents of the XData block. The default is `text/xml`.

If used to store XML, the XData block must consist of one root XML element, with any valid contents.

30.2 XML Example

To access an XML document in an arbitrary XData block programmatically, you use %Dictionary.CompiledXData and other classes in the %Dictionary package.

An XData block is useful if you want to define a small amount of system data. For example, suppose that the `EPI.AllergySeverity` class includes the properties `Code` (for internal use) and `Description` (for display to the users). This class could include an XData block like the following:

Class Member

```
XData LoadData
{
<table>
 <row>1^Minor</row>
 <row>2^Moderate</row>
 <row>3^Life-threatening</row>
 <row>9^Inactive</row>
 <row>99^Unable to determine</row>
</table>
}
```

The same class could also include a class method that reads this XData block and populates the table, as follows:

Class Member

```
ClassMethod Setup() As %Status
{
   //first kill extent
   do ..%KillExtent()

   // Get a stream of XML from the XData block contained in this class
   Set xdataID="EPI.AllergySeverity||LoadData"
   Set compiledXdata=##class(%Dictionary.CompiledXData).%OpenId(xdataID)
   Set tStream=compiledXdata.Data
   If '$IsObject(tStream) Set tSC=%objlasterror Quit

   set status=##class(%XML.TextReader).ParseStream(tStream,.textreader)
   //check status
   if $$$ISERR(status) do $System.Status.DisplayError(status) quit

   //iterate through document, node by node
   while textreader.Read()
   {
       if (textreader.NodeType="chars")
       {
           set value=textreader.Value
           set obj=..%New()
           set obj.Code=$Piece(value,"^",1)
           set obj.Description=$Piece(value,"^",2)
           do obj.%Save()
       }
   }
}
```

Notice the following:

- The XML within the XData is minimal. That it, instead of presenting the allergy severities as XML element with their own elements or attributes, the XData block simply presents rows of data as delimited strings. This approach allows you to write the setup data in a visually compact form.
- The `EPI.AllergySeverity` class is not XML-enabled and does not need to be XML-enabled.

30.3 JSON Example

A class could also include an XData block containing JSON content, like the following:

Class Member

```
XData LoadJSONData [MimeType = "application/json"]
{
   {
      "person":"John",
      "age":30,
      "car":"Ford"
   }
}
```

The same class could also include a class method that reads this XData block and populates a dynamic object, as follows:

Class Member

```
/// Reads a JSON XData block
ClassMethod SetupJSON() As %Status
{

   // Get a stream of JSON from the XData block contained in this class
   Set xdataID="MyPkg.MyClass||LoadJSONData"
   Set compiledXdata=##class(%Dictionary.CompiledXData).%OpenId(xdataID)
   Set tStream=compiledXdata.Data
   If '$IsObject(tStream) Set tSC=%objlasterror Quit

   // Create a dynamic object from the JSON content and write it as a string
   Set dynObject = {}.%FromJSON(tStream)
   Write dynObject.%ToJSON()

}
```

30.4 YAML Example

A class could also include an XData block containing YAML content, like the following Swagger API specification:

```
XData SampleAPI [mimetype = "application/yaml"]
{
swagger: "2.0"
info:
  title: Sample API
  description: API description in Markdown.
  version: 1.0.0
host: api.example.com
basePath: /v1
schemes:
  - https
paths:
  /users:
    get:
      summary: Returns a list of users.
      description: Optional extended description in Markdown.
      produces:
        - application/json
      responses:
        200:
          description: OK
}
```

This class method reads the XData block and writes its content, line by line:

Class Member

```
/// Reads a YAML XData block
ClassMethod SetupYAML() As %Status
{

   // Get a stream of YAML from the XData block contained in this class
   Set xdataID="MyPkg.MyClass||SampleAPI"
   Set compiledXdata=##class(%Dictionary.CompiledXData).%OpenId(xdataID)
   Set tStream=compiledXdata.Data
   If '$IsObject(tStream) Set tSC=%objlasterror Quit

   // Write the content from the stream, line by line
   While 'tStream.AtEnd {
      Write tStream.ReadLine(,.sc,.eol)
      If eol { Write ! }
   }
}
```

31 Defining Class Projections

This topic discusses *class projections*, which provide a way to customize what happens when a class is compiled or removed.

31.1 Introduction

Class projections provide a way to customize what happens when a class is compiled or removed. A class projection associates a class definition with a projection class. The projection class (derived from the %Projection.AbstractProjection class) provides methods that InterSystems IRIS® data platform uses to automatically generate additional code at two times:

- When the class is compiled
- When the class is deleted

This mechanism is used by the Java projections (hence the origin of the term *projection*) to automatically generate the necessary client binding code whenever a class is compiled.

31.2 Adding a Projection to a Class

To add a projection to a class definition, use the Projection statement within a class definition:

Class Definition

```
class MyApp.Person extends %Persistent
{
Projection JavaClient As %Projection.Java(ROOTDIR="c:\java");
}
```

This example defines a projection named `JavaClient` that will use the %Projection.Java projection class. When the methods of the projection class are called, they will receive the value of the *ROOTDIR* parameter.

A class can have multiple uniquely named projections. In the case of multiple projections, the methods of each projection class will be invoked when a class is compiled or deleted. The order in which multiple projections are handled is undefined.

InterSystems IRIS provides the following projection classes:

Class	Description
%Projection.Java	Generates a Java client class to enable access to the class from Java.
%Projection.Monitor	Registers this class as a routine that works with Log Monitor. Metadata is written to Monitor.Application, Monitor.Alert, Monitor.Item and Monitor.ItemGroup. A new persistent class is created called Monitor.Sample.
%Projection.MV	Generates an MV class that enables access to the class from MV.
%Projection.StudioDocument	Registers this class as a routine that works with Studio.
%Studio.Extension.Projection	Projects the XData `menu` block to the menu table.

You can also create your own projection classes and use them in the same way as you would any built-in projection class.

31.3 Creating a New Projection Class

To create a new projection class, create a subclass of the %Projection.AbstractProjection class, implement the projection interface methods (see the subsection), and define any needed class parameters. For example:

Class Definition

```
Class MyApp.MyProjection Extends %Projection.AbstractProjection
{

Parameter MYPARAM;

/// This method is invoked when a class is compiled
ClassMethod CreateProjection(cls As %String, ByRef params) As %Status
{
    // code here...
    QUIT $$$OK
}

/// This method is invoked when a class is 'uncompiled'
ClassMethod RemoveProjection(cls As %String, ByRef params, recompile as %Boolean) As %Status
{
    // code here...
    QUIT $$$OK
  }
}
```

31.3.1 The Projection Interface

Every projection class implements the *projection interface*, a set of methods that are called in response to certain events during the life cycle of a class. This interface consists of the following methods:

CreateProjection()

The **CreateProjection()** method is a class method that is invoked by the class compiler after it completes the compilation of a class definition. This method is passed the name of the class being compiled as well as an array containing the parameter values (subscripted by parameter name) defined for the projection.

RemoveProjection()

The **RemoveProjection()** method is a class method that is invoked either:

- When a class definition is deleted
- At the start of a recompilation of the class

  This method is passed the name of the class being removed, an array containing the parameter values (subscripted by parameter name) defined for the projection, and a flag indicating whether the method is being called as part of a recompilation or because the class definition is being deleted.

When a class definition containing a projection is compiled, the following events occur:

1. If the class has been compiled previously, it will be *uncompiled* before the new compile begins; that is, all the results of the previous compilation are removed. At this time, the compiler invokes the **RemoveProjection()** method for every projection with a flag indicating that a recompilation is about to occur.

   Note that you cannot call methods of the associated class from within the **RemoveProjection()** method, because the class does not exist at this point.

   Also note that if you add a new projection definition to a class that had been previously compiled (without the projection), then the compiler will call the **RemoveProjection()** method on the next compilation even though the **CreateProjection()** method has never been called. When implementing this method, be sure to plan for this possibility.

2. After the class is completely compiled (that is, it is ready for use), the compiler will invoke the **CreateProjection()** method for every projection.

When a class definition is deleted, the **RemoveProjection()** method is invoked for every projection with a flag indicating that a deletion has occurred.

31.4 See Also

- Defining Classes
- Projection Syntax and Keywords

32
Defining Callback Methods

This page describes how to define callback methods, which are called automatically by the methods you use to work with objects. Callback methods have names that begin with `%On` or `On`, typically followed by the name of the method that invokes them.

If a system method has an implemented callback method, then when the system method runs, that method invokes the callback method. For example, **%Delete()** invokes **%OnDelete()**, if **%OnDelete()** is implemented.

Important: Do not execute callback methods directly.

32.1 Available Callbacks in Object Classes

The following table lists all the available callbacks in object classes. Note that some are available for all object classes, while others are applicable only to persistent objects.

Table 32–1: Callback Methods

Callback Name	Implemented for	Method Type	Private Method?
%OnAddToSaveSet()	%RegisteredObject	Instance	Yes
%OnAfterBuildIndices()	%Persistent	Class	Yes
%OnAfterDelete()	%Persistent	Class	Yes
%OnAfterPurgeIndices()	%Persistent	Class	Yes
%OnAfterSave()	%Persistent	Instance	Yes
%OnBeforeBuildIndices()	%Persistent	Class	Yes
%OnBeforePurgeIndices()	%Persistent	Class	Yes
%OnBeforeSave()	%Persistent	Instance	Yes
%OnClose()	%RegisteredObject	Instance	Yes
%OnConstructClone()	%RegisteredObject	Instance	Yes
%OnDelete()	%Persistent	Class	Yes
%OnDeleteFinally()	%Persistent	Class	No

Callback Name	Implemented for	Method Type	Private Method?
%OnNew()	%RegisteredObject	Instance	Yes
%OnOpen()	%Persistent, %SerialObject	Instance	Yes
%OnOpenFinally()	%Persistent	Instance	No
%OnReload	%Persistent	Instance	Yes
%OnRollBack	%Persistent	Instance	Yes
%OnSaveFinally()	%Persistent	Instance	No
%OnValidateObject()	%RegisteredObject	Instance	Yes
%OnDetermineClass()	%Storage.Persistent	Class	No

Note: For all callbacks that are private methods, documentation for them is only visible in the Class Reference if the **Private** check box in the upper-right corner of the Class Reference is selected.

32.2 %OnAddToSaveSet()

This instance method is called whenever the current object is being added to a SaveSet by **%AddToSaveSet()**. **%AddToSaveSet()** can be called by:

- **%Save()** for an instance of %Persistent
- **%GetSwizzleObject()** for an instance of %SerialObject
- **%AddToSaveSet()** for a referencing object

If **%OnAddToSaveSet()** modifies another object, then it is the responsibility of **%OnAddToSaveSet()** to invoke **%AddToSaveSet()** on that modified object. When calling **%AddToSaveSet()** from **%OnAddToSaveSet()**, pass the depth as the first argument and 1 (literal one) as the second argument.

When you invoke **%Save()** on an object, called, for example, *MyPerson*, the system generates a list of objects that *MyPerson* references. A SaveSet is the list of objects consisting of the object to be saved and all the objects that it references. In the example, the SaveSet might include referenced objects *MySpouse*, *MyDoctor*, and so on. For a fuller discussion, see Saving Objects.

The signature of **%OnAddToSaveSet()** is:

Class Member

```
Method %OnAddToSaveSet(depth As %Integer,
                       insert As %Integer,
                       callcount As %Integer)
                        As %Status [ Private, ServerOnly = 1 ]
{
    // body of method here...
}
```

where:

depth

An integer value passed in from **%AddToSaveSet()** that represents the internal state of SaveSet construction. If you use **%OnAddToSaveSet()** to add any other objects to the SaveSet, pass this value to **%AddToSaveSet()** without change.

insert

A flag indicating if the object being saved is being inserted into the extent (1) or that it is already part of the extent (0).

callcount

The number of times that **%OnAddToSaveSet** has been called for this object. Due to the networked nature of object references, it is possible that **%AddToSaveSet** can be invoked on the same object multiple times.

The method returns a %Status code, where a failure status causes the save to fail and the transaction to be rolled back.

You can update objects, create new objects, delete objects and ask objects to include themselves in the current SaveSet by calling **%AddToSaveSet()**. If you modify the current instance or any of its descendants, you must let the system know that you have done this; to do so, call **%AddToSaveSet()** for the modified instance(s) and specify the *Refresh* argument as 1.

None of the modification restrictions imposed on **%OnAfterSave()**, **%OnBeforeSave()**, or **%OnValidateObject()** are in place for **%OnAddToSaveSet()**.

If you delete an object using **%OnAddToSaveSet()**, be sure to call **%RemoveFromSaveSet()** to clean up any dangling references to it.

This method can be overridden in any subclass of %Library.RegisteredObject.

32.3 %OnAfterBuildIndices()

This class method is called by the **%BuildIndices()** method after that method builds the indexes and executes $SortEnd and just before the method releases the extent lock (if one had been requested).

Its signature is:

```
ClassMethod %OnAfterBuildIndices(indexlist As %String(MAXLEN="") = "") As %Status [ Abstract, Private,
 ServerOnly = 1 ]
{
    // body of method here...
}
```

where:

indexlist

A $List of the index names.

32.4 %OnAfterDelete()

This class method is called by the **%Delete()** or **%DeleteId()** method just after a specified object is deleted (immediately after a successful call to **%DeleteData()**). This method allows you to perform actions outside the scope of the object being saved, such as queuing a later notification action.

Its signature is:

Class Member

```
ClassMethod %OnAfterDelete(oid As %ObjectIdentity) As %Status [ Private, ServerOnly = 1 ]
{
    // body of method here...
}
```

where:

oid

The object being deleted.

The method returns a %Status code, where a failure status causes **%Delete()** or **%DeleteId()** to fail and, if there is an active transaction, to roll it back. If **%Delete()** or **%DeleteId()** return an error (either its own error or one originating in **%DeleteData()**), then there is no call to **%OnAfterDelete()**.

Subclasses of %Library.Persistent have the option of overriding this method.

32.5 %OnAfterPurgeIndices()

This class method is called by the **%PurgeIndices()** method after that method has completed all its processing.

Its signature is:

```
ClassMethod %OnAfterPurgeIndices(indexlist As %String(MAXLEN="") = "") As %Status [ Abstract, Private,
 ServerOnly = 1 ]
{
    // body of method here...
}
```

where:

indexlist

A $List of the index names.

32.6 %OnAfterSave()

This instance method is called by the **%Save()** method just after an object is saved. This method allows you to perform actions outside the scope of the object being saved, such as queueing a later notification action. An example is a bank using the deposit in excess of a certain amount to cause it to send the customer an explanation of its deposit policies.

Its signature is:

Class Member

```
Method %OnAfterSave(insert as %Boolean)As %Status [ Private, ServerOnly = 1 ]
{
    // body of method here...
}
```

where:

insert

A flag indicating if the object being saved is being inserted into the extent (1) or that it is an update of an existing object (0).

The method returns a %Status code, where a failure status causes **%Save()** to fail and ultimately roll back the transaction.

Subclasses of %Library.Persistent have the option of overriding this method.

32.7 %OnBeforeBuildIndices()

This class method is called by the **%BuildIndices()** method after that method acquires the extent lock (if one had been requested) and before that method starts to build indexes.

Its signature is:

```
ClassMethod %OnBeforeBuildIndices(ByRef indexlist As %String(MAXLEN="") = "") As %Status [ Abstract,
Private, ServerOnly = 1 ]
{
    // body of method here...
}
```

where:

indexlist

A $List of the index names. This parameter is passed by reference. If the implementation of **%OnBeforeBuildIndices()** alters this value, then **%BuildIndices()** receives the changed value.

32.8 %OnBeforePurgeIndices()

This class method is called by the **%PurgeIndices()** method before that method starts work. If this method returns an error, then **%PurgeIndices()** will exit immediately without purging any index structures, returning the error to the caller of **%PurgeIndices()**.

Its signature is:

```
ClassMethod %OnBeforePurgeIndices(ByRef indexlist As %String(MAXLEN="") = "") As %Status [ Abstract,
Private, ServerOnly = 1 ]
{
    // body of method here...
}
```

where:

indexlist

A $List of the index names. This parameter is passed by reference. If the implementation of **%OnBeforePurgeIndices()** alters this value, then **%PurgeIndices()** receives the changed value.

32.9 %OnBeforeSave()

This instance method is called by the **%Save()** method just before an object is saved. This method allows you to request user confirmation before completing an action before saving the instance to disk.

Important: It is not valid to modify the current object in **%OnBeforeSave()**. If you wish to modify the object before saving it, implement the **%OnAddToSaveSet()** callback instead and include your logic in that method.

Its signature is:

Class Member

```
Method %OnBeforeSave(insert as %Boolean) As %Status [ Private, ServerOnly = 1 ]
{
    // body of method here...
}
```

where:

insert

A flag indicating if the object being saved is being inserted into the extent (1) or that it is already part of the extent (0).

The method returns a %Status code, where a failure status causes the save to fail.

Subclasses of %Library.Persistent have the option of overriding this method.

32.10 %OnClose()

This instance method is called immediately before an object is destructed, thereby providing the user with an opportunity to perform operations on any ancillary items, such as releasing locks or removing temporary data structures.

Its signature is:

Class Member

```
Method %OnClose() As %Status [ Private, ServerOnly = 1 ]
{
    // body of method here...
}
```

The method returns a %Status code, where a failure status is only informational and does nothing to prevent the object from being destructed.

Subclasses of %Library.RegisteredObject have the option of overriding this method.

32.11 %OnConstructClone()

This instance method is called by the **%ConstructClone()** method immediately after the structures have been allocated for the cloned object and all the data has been copied into it. The method allows you to perform any additional actions related to the cloned object, such as taking out a lock or resetting any of the property values of the clone.

Its signature is:

Class Member

```
Method %OnConstructClone(object As %RegisteredObject,
                         deep As %Boolean,
                         ByRef cloned As %String)
                         As %Status [ Private, ServerOnly = 1 ]
{
    // body of method here...
}
```

where:

object

The OREF of the object that was cloned.

deep

How “deep” the cloning process is, where 0 specifies that the clone points to the same related objects as the original; 1 causes objects related to the object being cloned to also be cloned, so that the clone gets its own set of related objects.

cloned

An argument whose use varies according to how **%OnConstructClone()** is being invoked. See class documentation on %Library.RegisteredObject for details.

The method returns a %Status code, where a failure status prevents the clone from being created.

Subclasses of %Library.RegisteredObject have the option of overriding this method.

32.12 %OnDelete()

This class method is called by the **%Delete()** or **%DeleteId()** method just before an object is deleted. This method can be used to ensure that deleting an object does not corrupt data integrity, such as by ensuring that an object designed to contain other objects is only deleted when it is empty.

Its signature is:

Class Member

```
ClassMethod %OnDelete(oid As %ObjectIdentity) As %Status [ Private, ServerOnly = 1 ]
{
    // body of method here...
}
```

where:

oid

An object identifier for the object being deleted.

The method returns a %Status code, where a failure status stops the deletion.

To access the properties of the object that is to be deleted, convert the OID that is passed in to an ID, and then open the object with that ID, as in the following example:

```
ClassMethod %OnDelete(oid As %ObjectIdentity) As %Status [ Private, ServerOnly = 1 ]
{
    set id = $$$oidPrimary(oid)
    set obj = ..%OpenId(id)
    write "Deleting object with name ", obj.Name, !
    return 1
}
```

Subclasses of %Library.Persistent have the option of overriding this method.

32.13 %OnDeleteFinally()

This class method is called by the **%Delete()** or **%DeleteId()** method after all processing is complete and just before returning to the caller. If the **%Delete()** or **%DeleteId()** method started a transaction then that transaction is completed prior to invoking the callback. This method does not return any value and cannot change the result of the method that invoked it.

Its signature is:

Class Member

```
ClassMethod %OnDeleteFinally(oid As %ObjectIdentity, status As %Status) [ ServerOnly = 1 ]
{
    // body of method here...
}
```

where:

oid

An object identifier for the object being deleted.

status

The status that will be reported to the user. The status cannot be modified by this callback method.

Subclasses of %Library.Persistent have the option of overriding this method.

32.14 %OnNew()

This instance method is called by the **%New()** method at the point when the memory for an object has been allocated and properties are initialized.

Its signature is:

Class Member

```
Method %OnNew(initvalue As %String) As %Status [ Private, ServerOnly = 1 ]
{
    // body of method here...
}
```

where:

initvalue

A string that the method uses in setting up the object, unless being overridden, as described in the next note.

Important: The arguments for **%OnNew()** must match those of **%New()**. When customizing this method, override the arguments with whatever variables and types that you expect to receive from **%New()**. For example, if **%New()** accepts two arguments — *dob* for a date of birth and *name* for a first name and surname, the signature of **%OnNew()** might be:

Class Member

```
Method %OnNew(dob as %Date = "", name as %Name = "") as %Status [ Private, ServerOnly =
 1 ]
{
 // body of method here...
}
```

The method returns a %Status code, where a failure status stops the creation of the object.

For example, with a class whose instances must have a value for their Name property, the callback might be of the form:

Class Member

```
Method %OnNew(initvalue As %String) As %Status
{
    If initvalue="" Quit $$$ERROR($$$GeneralError,"Must supply a name")
    Set ..Name=initvalue
    Quit $$$OK
}
```

Subclasses of %Library.RegisteredObject have the option of overriding this method.

32.15 %OnOpen()

This instance method is called by the **%Open()** or **%OpenId()** method just before an object is opened. It allows you to verify the state of an instance compared to any relevant entities.

Its signature is:

Class Member

```
Method %OnOpen() As %Status [ Private, ServerOnly = 1 ] {
    // body of method here...
}
```

The method returns a %Status code, where a failure status stops the opening of the object.

Subclasses of %Library.Persistent and %SerialObject have the option of overriding this method.

32.16 %OnOpenFinally()

This class method is called by the **%Open()** or **%OpenId()** method after all processing is complete and just before returning to the caller. This method does not return any value and cannot change the result of the method that invoked it.

Its signature is:

Class Member

```
ClassMethod %OnOpenFinally(oid As %ObjectIdentity, status As %Status) [ ServerOnly = 1 ]
{
    // body of method here...
}
```

where:

oid

An object identifier for the object being opened.

status

The status that will be reported to the user. The status cannot be modified by this callback method.

Subclasses of %Library.Persistent have the option of overriding this method.

32.17 %OnReload()

This instance method is called by the **%Reload** method to provide notification that the object specified by *oid* was reloaded. Note that **%Open** calls **%Reload** when the object identified by *oid* is already in memory. If this method returns an error, the object is not opened.

Its signature is:

Class Member

```
Method %OnReload() As %Status [ Private, ServerOnly = 1 ]
  {
    // body of method here...
  }
```

The method returns a %Status code, where a failure status stops the rollback operation.

Subclasses of %Library.Persistent have the option of overriding this method.

32.18 %OnRollBack()

InterSystems IRIS® data platform calls this instance method when it rolls back an object that it had previously successfully serialized as part of a SaveSet. (See Saving Objects.)

When you invoke **%Save()** for a persistent object or a stream or when you invoke **%GetSwizzleObject()** for a serial object, the system starts a *save transaction* which includes all the objects in the SaveSet. If the **%Save()** fails (because properties do not pass validation, for example), InterSystems IRIS rolls back all objects that it had previously successfully serialized as part of a SaveSet. That is, for each of these objects, InterSystems IRIS invokes **%RollBack()**, which calls **%OnRollBack()**.

InterSystems IRIS does not invoke this method for an object that has not been successfully serialized, that is, an object that is not valid.

%RollBack() restores the on-disk state of data for that object to its pre-transaction state, but does not affect the in-memory state of any properties of that object that you have set, apart from its ID assignment. (For more details, see Saving Objects.) If you want to revert in-memory changes, do so in **%OnRollBack()**.

The signature of this method is:

Class Member

```
Method %OnRollBack() As %Status [ Private, ServerOnly = 1 ]
{
    // body of method here...
}
```

The method returns a %Status code, where a failure status stops the rollback operation.

Subclasses of %Library.Persistent have the option of overriding this method.

32.19 %OnSaveFinally()

This class method is called by the **%Save()** method after all processing is complete and just before returning to the caller. If the **%Save()** method started a transaction then that transaction is completed prior to invoking the callback. This method does not return any value and cannot change the result of the method that invoked it.

Its signature is:

Class Member

```
ClassMethod %OnSaveFinally(oref As %ObjectHandle, status As %Status) [ ServerOnly = 1 ]
{
    // body of method here...
}
```

where:

oid

An object identifier for the object being saved.

status

The status that will be reported to the user. The status cannot be modified by this callback method.

Subclasses of %Library.Persistent have the option of overriding this method.

32.20 %OnValidateObject()

This instance method is called by the **%ValidateObject()** method just after all validation has occurred. This allows you to perform custom validation, such as where valid values for one property vary according to the value of another property.

Its signature is:

Class Member

```
Method %OnValidateObject() As %Status [ Private, ServerOnly = 1 ]
{
    // body of method here...
}
```

The method returns a %Status code, where a failure status causes the validation to fail.

Subclasses of %Library.RegisteredObject have the option of overriding this method.

32.21 %OnDetermineClass()

The **%OnDetermineClass()** class method returns the most specific type class of that object. (For an introduction to the most specific type class, see %ClassName() and the Most Specific Type Class (MSTC).) **%OnDetermineClass()** is implemented by the default storage class. If you use custom storage or SQL storage, there is no default implementation for this method, but you can implement it.

Its signature is:

```
ClassMethod %OnDetermineClass(
        oid As %ObjectIdentity,
        ByRef class As %String)
    As %Status [ ServerOnly = 1 ]
```

where:

oid

An object identifier for the object.

class

The most specific type class of the instance identified by *oid*. The *most specific type class* of an object is the class of which the object is an instance and the object is not an instance of any subclass of that class.

The return value is a status value indicating success or failure.

Subclasses of %Library.SwizzleObject can override this method.

32.21.1 Invoking %OnDetermineClass()

%OnDetermineClass() can be invoked in either of two ways:

```
Set status = ##class(APackage.AClass).%OnDetermineClass(myoid, .myclass)
Set status = myinstance.%OnDetermineClass(myoid, .myclass)
```

where *myoid* is the object whose most specific type class is being determined and *myclass* is the class identified. APackage.AClass is the class from which the method is being invoked and *myinstance* is the instance from which the method is being invoked.

In this case, the method is computing the most specific type class for *myoid* and setting *myclass* equal to that value. If *myoid* is not an instance of the current class, an error is returned.

Consider the example of using **%OnDetermineClass()** with Sample.Employee, which is a subclass of Sample.Person. If there is a call of the form

```
Set status = ##class(Sample.Employee).%OnDetermineClass(myoid, .class)
```

and *myoid* refers to an object whose most specific type class is Sample.Person, then the call returns an error.

32.21.2 An Example of Results of Calls to %OnDetermineClass()

Suppose there is a MyPackage.GradStudent class that extends a MyPackage.Student class that extends a MyPackage.Person class. The following shows the results of invoking **%OnDetermineClass()**, passing in the OID of an object whose most specific type class is MyPackage.Student:

- `##class(MyPackage.Person).%OnDetermineClass(myOid,.myClass)`

  - Return value: $$$OK
  - *myClass* set to: MyPackage.Student

- `##class(MyPackage.Student).%OnDetermineClass(myOid,.myClass)`
  - Return value: $$$OK
  - *myClass* set to: MyPackage.Student

- `##class(MyPackage.GradStudent).%OnDetermineClass(myOid,.myClass)`
  - Return value: error status
  - *myClass* set to: `""`

32.22 Triggers as an Alternative

For an application that uses both SQL and object access, if you implement a trigger, it is generally desirable to call the same logic at an equivalent point in object access. For example, if you insert an audit record when a row is deleted, you should probably also insert an audit record if an object in that same class is deleted.

In many cases, the easiest way to call the same logic from both SQL and object access is to define a specific type of trigger. If a trigger is defined with `Foreach = row/object`, then the trigger is called during either object or SQL access. See Triggers and Transactions.

For more information, see Using Triggers or CREATE TRIGGER.

32.23 See Also

- Defining Classes
- Working with Registered Objects
- Introduction to Persistent Objects

33

Overriding Property Methods

This topic describes how to override property methods, which are the actual methods that InterSystems IRIS® data platform uses when you use OREFs to work with the properties of objects. These methods are not used when you access the data via SQL.

A closely related topic is defining data type classes, which are used for literal properties; when defining data type classes, it may be necessary to override property methods as described on this page.

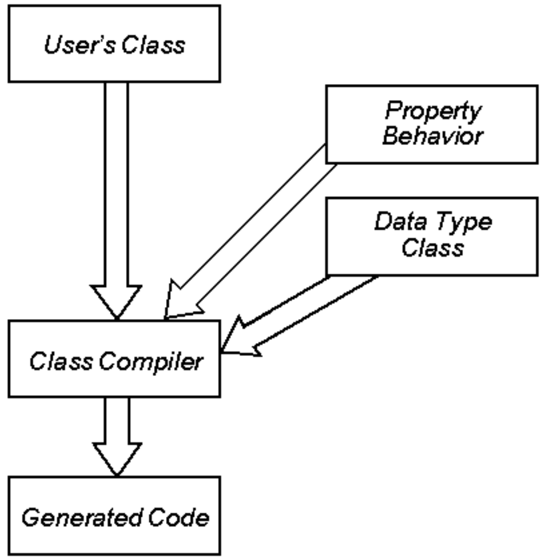
33.1 How Property Methods Are Defined

When a class has properties, the class compiler automatically generates a number of methods for each property and uses those internally. The names of the methods are based on the property names.

For any given property, the property methods are generated via a special mechanism that uses logic from two places:

- An internal property behavior class, which provides certain built-in property behavior, such as **Get()**, **Set()**, and validation code. There are different implementations based on the kind of property. Literal properties, object-valued properties, collection properties, and serial object properties have different property behavior classes; these are internal and cannot be redefined.
- Methods defined in the data type class used by the property, if applicable. Many of these methods are method generators.

Figure 33–1: Property Behavior



33.2 Overriding a Property Getter Method

To override the getter method for a property, modify the class that contains the property and add a method as follows:

- It must have the name *PropertyName*`Get`, where *PropertyName* is the name of the corresponding property.
- It takes no arguments.
- Its return type must be the same as the type of the property.
- It must return the value of the property.
- To refer to the value of this property, this method must use the variable `i%`*PropertyName*. This name is case-sensitive.

  Important: Within this getter method for a given property, do not use `..`*PropertyName* syntax to refer to the value of that property. If you attempt to do so, the result is a <FRAMESTACK> error, caused by a recursive series of references. You can, however, use `..`*PropertyName* to refer to other properties, because doing so does not cause any recursion.

  The variable `i%`*PropertyName* is an *instance variable*. For more information on instance variables, see i%PropertyName.

Note: Note that it is not supported to override accessor methods for object-typed properties or for multidimensional properties. Also because the maximum length of a method name is 220 characters, it is not possible to create accessor methods for properties that are 218, 219, or 220 characters long.

The following shows an example, a setter method for a property named HasValue, which is of type %Boolean:

Class Member

```
Method HasValueGet() As %Boolean
{
  If ((i%NodeType="element")||(i%NodeType="")) Quit 0
  Quit 1
}
```

33.3 Overriding a Property Setter Method

To override the setter method for a property, modify the class that contains the property and add a method as follows:

- It must have the name *PropertyName*`Set`, where *PropertyName* is the name of the corresponding property.
- It takes one argument, which contains the value of the property.

  Specifically, this is the value specified in the SET command, when the property is being set.
- It must return a %Status value.
- To set the value of this property, this method must set the variable `i%`*PropertyName*. This name is case-sensitive.

  Important: Within this setter method for a given property, do not use `..`*PropertyName* syntax to refer to the value of that property. If you attempt to do so, the result is a <FRAMESTACK> error, caused by a recursive series of references. You can, however, use `..`*PropertyName* to refer to other properties, because doing so does not cause any recursion.

  The variable `i%`*PropertyName* is an *instance variable*. For more information on instance variables, see i%PropertyName.

Note: Note that it is not supported to override accessor methods for object-typed properties or for multidimensional properties. Also because the maximum length of a method name is 220 characters, it is not possible to create accessor methods for properties that are 218, 219, or 220 characters long.

For example, suppose that MyProp is of type %String. We could define the following setter method:

Class Member

```
Method MyPropSet(value as %String) As %Status
{
    if i%MyProp="abc" {
        set i%MyProp="corrected value"
    }
    quit $$$OK
}
```

The following shows another example, a setter method for a property named DefaultXmlns, which is of type %String:

Class Member

```
Method DefaultXmlnsSet(value As %String) As %Status
{
    set i%DefaultXmlns = value
    If ..Namespaces'="" Set ..Namespaces.DefaultXmlns=value
    quit $$$OK
}
```

Notice that this example refers to the Namespaces property of the same object by using the `..`*PropertyName* syntax. This usage is not an error, because it does not cause any recursion.

33.4 Defining an Object-Valued Property with a Custom Accessor Method

As noted earlier, it is not supported to override accessor methods for object-typed properties. If you need to define a property that holds object values and you need to define custom accessor methods, define the property with the type %RawString..

This is not an object class but is rather a generic class, and it is permitted to override the accessor methods for this property. When using the property, set it equal to an instance of the desired class.

For example, the following class includes the property Zip, whose formal type is %RawString. The property description indicates that the property is meant to be an instance of Sample.USZipCode. The class also defines the **ZipGet()** and **ZipSet()** property methods.

Class Definition

```
Class PropMethods.Demo Extends %Persistent
{

/// Timestamp for viewing Zip
Property LastTimeZipViewed As %TimeStamp;

/// Timestamp for changing Zip
Property LastTimeZipChanged As %TimeStamp;

/// When setting this property, set it equal to instance of Sample.USZipCode.
/// The type is %RawString rather than Sample.USZipCode, so that it's possible
/// to override ZipGet() and ZipSet().
Property Zip As %RawString;

Method ZipGet() As %RawString [ ServerOnly = 1 ]
{
    // get id, swizzle referenced object
    set id = i%Zip
    if (id '= "") {
        set zip = ##class(Sample.USZipCode).%OpenId(id)
        set ..LastTimeZipViewed = $zdt($zts)
    }
    else {
        set zip = ""
    }
    return zip
}

Method ZipSet(zip As %RawString) As %Status [ ServerOnly = 1 ]
{
    // set i% for new zip
    if ($isobject(zip) && zip.%IsA("Sample.USZipCode")) {
        set id = zip.%Id()
        set i%Zip = id
        set ..LastTimeZipChanged = $zdt($zts)
    }
    else {
        set i%Zip = ""
    }
    quit $$$OK
}

}
```

The following Terminal session demonstrates the use of this class:

```
SAMPLES>set demo=##class(PropMethods.Demo).%New()

SAMPLES>write demo.LastTimeZipChanged

SAMPLES>set zip=##class(Sample.USZipCode).%OpenId(10001)

SAMPLES>set demo.Zip=zip

SAMPLES>w demo.LastTimeZipChanged
10/14/2015 19:21:08
```

33.5 See Also

- Using Property Methods

34

Defining Data Type Classes

This topic describes how data type classes work and describes how to define them.

34.1 Introduction

A data type class is to be used as the type of a literal property in an object class. Data type classes provide the following features:

- They provide for SQL and client interoperability by providing SQL logical operation, client data type, and translation information.
- They provide validation for literal data values, which you can extend or customize by using data type class parameters.
- They manage the translation of literal data for its stored (on disk), logical (in memory), and display formats.

Data type classes define a specific set of methods (typically method generators) that the compiler uses to define the behavior of the properties. Data type classes are not object classes (that is, they do not inherit from %RegisteredObject).

When a class has properties, the class automatically possesses a set of methods for accessing and working with property values. These are called property methods and their names are based on the property names. For example, if a class contains a string property called `Name`, the class includes methods called **NameDisplayToLogical()**, **NameGet()**, **NameGetStored()**, **NameIsValid()**, **NameLogicalToDisplay()**, **NameLogicalToOdbc()**, **NameNormalize()**, and **NameSet()**. InterSystems IRIS uses the property methods internally to access and work with property values. You can also call these methods directly; see Using Property Methods.

34.2 Parameters in Data Type Classes

A data type class can include class parameters that define default behavior, and this default behavior can be overridden for specific properties based on that data type class. For example, the %Integer data type class has a class parameter, *MAXVAL*, which specifies the maximum valid value for a property of type %Integer. If you define a class with the property NumKids as follows:

Class Member

```
Property NumKids As %Integer(MAXVAL=10);
```

This specifies that the NumKids property cannot exceed 10.

Internally, this works as follows: the validation methods for the standard data type classes are all implemented as method generators and use their various class parameters to control the generation of these validation methods. In this example, this property definition generates a **NumKidsIsValid()** method that tests whether the value of NumKids exceeds 10. Without the use of class parameters, creating this functionality would require the definition of an IntegerLessThanTen class.

34.3 Defining a Data Type Class

To easily define a data type class, first identify an existing data type class that is close to your needs. Create a subclass of this class. In your subclass:

- Specify suitable values for the keywords SqlCategory, ClientDataType, and OdbcType.
- Override any class parameters as needed. For example, you might override the *MAXLEN* parameter so that there is no length limit for the property.

  If needed, add your own class parameters as well.
- Override the methods of the data type class as needed. In your implementations, refer to the parameters of this class as needed.

If you do not base your data type class on an existing data type class, be sure to add `[ ClassType=datatype ]` to the class definition. This declaration is not needed if you do base your class on another data type class.

Note: When defining a data type class, do not extend %Persistent, %RegisteredObject, or other object classes, as data type classes cannot contain properties.

34.4 Defining Class Methods in Data Type Classes

Depending on your needs, you should define some or all of the following class methods in your data type classes:

- **IsValid()** — performs validation of data for the property, using property parameters if appropriate. As noted earlier, the **%ValidateObject()** instance method of any object class invokes the **IsValid()** method for each property. This method has the following signature:

  ```
  ClassMethod IsValid(%val) As %Status
  ```

  Where *%val* is the value to be validated. This method should return an error status if the value is invalid and should otherwise return $$$OK.

  Note: It is standard practice in InterSystems IRIS not to invoke validation logic for null values.

- **Normalize()** — converts the data for the property into a standard form or format. The **%NormalizeObject()** instance method of any object class invokes the **Normalize()** method for each property. This method has the following signature:

  ```
  ClassMethod Normalize(%val) As Type
  ```

  Where *%val* is the value to be validated and *Type* is a suitable type class.

- **DisplayToLogical()** — converts a display value into a logical value. (For information on formats, see Data Formats.) This method has the following signature:

  ```
  ClassMethod DisplayToLogical(%val) As Type
  ```

Where *%val* is the value to be converted and *Type* is a suitable type class.

The other format conversion methods have the same general form.

- **LogicalToDisplay()** — converts a logical value to a display value.
- **LogicalToOdbc()** — converts a logical value into an ODBC value.

  Note that the ODBC value must be consistent with the ODBC type specified by the OdbcType class keyword of the data type class.
- **LogicalToStorage()** — converts a logical value into a storage value.
- **LogicalToXSD()** — converts a logical value into the appropriate SOAP-encoded value.
- **OdbcToLogical()** — converts an ODBC value into a logical value.
- **StorageToLogical()** — converts a database storage value into a logical value.
- **XSDToLogical()** — converts a SOAP-encoded value into a logical value.

When implementing these methods, note the following requirements:

- Each method must be a class method.
- Each method must accept an argument: the value to check for validity, to normalize, to convert, and so on.
- If the data type class includes the *DISPLAYLIST* and *VALUELIST* parameters, these methods must first check for the presence of these class parameters and include code to process these lists if present.
- The data format and translation methods cannot include embedded SQL. If you need to call embedded SQL within this logic, then you can place the embedded SQL in a separate routine, and the method can call this routine.
- These methods are used within a procedure block and thus cannot use NEW to provide a new scope for a variable, other than a percent variable.

In most cases, it is best to define these methods as method generators.

34.5 Defining Instance Methods in Data Type Classes

You can also add instance methods to the data type class, and these methods can use the variable *%val*, which contains the current value of the property. The compiler uses these to generate additional property methods in any class that uses the data type class. For example, consider the following example data type class:

Class Definition

```
Class Datatypes.MyDate Extends %Date
{

Method ToMyDate() As %String [ CodeMode = expression ]
{
$ZDate(%val,4)
}
}
```

Suppose that we have another class as follows:

Class Definition

```
Class Datatypes.Container Extends %Persistent
{

Property DOB As Datatypes.MyDate;

/// additional class members
}
```

When we compile these classes, InterSystems IRIS adds the instance method **DOBToMyDate()** to the container class. Then when we create an instance of the container class, we can invoke this method. For example:

```
TESTNAMESPACE>set instance=##class(Datatypes.Container).%New()

TESTNAMESPACE>set instance.DOB=+$H

TESTNAMESPACE>write instance.DOBToMyDate()
30/10/2024
```

34.6 See Also

- Defining Classes
- Working with Registered Objects
- Defining and Using Literal Properties
- Using Property Methods

35

Implementing Dynamic Dispatch

This topic discusses dynamic dispatch in InterSystems IRIS® data platform classes.

35.1 Introduction to Dynamic Dispatch

InterSystems IRIS classes can include support for what is called *dynamic dispatch*. If dynamic dispatch is in use and a program references a property or method that is not part of the class definition, then a method (called a *dispatch method*) is called that attempts to resolve the undefined method or property. For example, dynamic dispatch can return a value for a property that is not defined or it can invoke a method for a method that is not implemented. The dispatch destination is dynamic in that it does not appear in the class descriptor and is not resolved until runtime.

InterSystems IRIS makes a number of dispatch methods available that you can implement. Each method attempts to resolve an element that is missing under different circumstances.

If you implement a dispatch method, it has the following effects:

- During application execution, if InterSystems IRIS encounters an element that is not part of the compiled class, it invokes the dispatch method to try to resolve the encountered element.
- The application code that uses the class does not do anything special to make this happen. InterSystems IRIS automatically checks for the existence of the dispatch method and, if that method is present, invokes it.

35.2 Content of Methods Implementing Dynamic Dispatch

As the application developer, you have control over the content of dispatch methods. The code within them can be whatever is required to implement the methods or properties that the class is attempting to resolve.

Code for dynamic dispatch might include attempts to locate a method based on other classes in the same extent, package, database, on the same file system, or by any other criteria. If a dispatch method provides a general case, it is recommended that the method also create some kind of log for this action, so that there is a record of any continued operation that includes this general resolution.

For example, the following implementation of **%DispatchClassMethod()** allows the application user to invoke a method to perform whatever action was intended:

Class Member

```
ClassMethod %DispatchClassMethod(Class As %String, Method As %String, args...)
{
    WRITE "The application has attempted to invoke the following method: ",!,!
    WRITE Class,".",Method,!,!
    WRITE "This method does not exist.",!
    WRITE "Enter the name of the class and method to call",!
    WRITE "or press Enter for both to exit the application.",!,!


    READ "Class name (in the form 'Package.Class'): ",ClassName,!
    READ "Method name: ",MethodName,!

    IF ClassName = "" && MethodName = "" {
        // return a null string to the caller if a return value is expected
        QUIT:$QUIT "" QUIT
    } ELSE {
        // checking $QUIT ensures that a value is returned
        // if and only if it is expected
        IF $QUIT {
            QUIT $CLASSMETHOD(ClassName, MethodName, args...)
        } ELSE {
            DO $CLASSMETHOD(ClassName, MethodName, args...)
            QUIT
        }
    }
}
```

By including this method in a class that is a secondary superclass of all classes in an application, you can establish application-wide handling of calls to nonexistent class methods.

35.2.1 Return Values

None of the dispatch methods have specified return values. This is because each should provide output that is of the same type of the call that originally created the need for the dispatch.

If the dispatch method cannot resolve the method or property, it can use **$SYSTEM.Process.ThrowError()** to throw a <METHOD DOES NOT EXIST> or <PROPERTY DOES NOT EXIST> error — or whatever else may be appropriate.

35.3 The Dynamic Dispatch Methods

The following methods may be implemented to resolve unknown methods and properties:

- %DispatchMethod()
- %DispatchClassMethod()
- %DispatchGetProperty()
- %DispatchSetProperty()
- %DispatchSetMultidimProperty()

35.3.1 %DispatchMethod()

This method implements an unknown method call. Its syntax is:

```
Method %DispatchMethod(Method As %String, Args...)
```

where its first argument is the name of the referenced method and the second argument is an array that holds all the arguments passed to the original method. Since the number of arguments and their types can vary depending on the method being resolved, the code in **%DispatchMethod()** needs to handle them correctly (since the class compiler cannot make any assumptions about the type). The `Args...` syntax handles this flexibly.

Because **%DispatchMethod()** attempts to resolve any unknown instance method associated with the class, it has no specified return value; if successful, it returns a value whose type is determined by the method being resolved and whether the caller expects a return value.

%DispatchMethod() can also resolve an unknown multidimensional property reference — that is, to get the value of a property. However, only direct multidimensional property references are supported for dynamic dispatch. **$DATA**, **$ORDER**, and **$QUERY** are not supported, nor is a **SET** command with a list of variables.

35.3.2 %DispatchClassMethod()

This method implements an unknown class method call. Its syntax is:

```
ClassMethod %DispatchClassMethod(Class As %String, Method As %String, Args...)
```

where its first two arguments are the name of the referenced class and the name of the referenced method. Its third argument is an array that holds all the arguments passed to the original method. Since the number of arguments and their types can vary depending on the method being resolved, the code in **%DispatchClassMethod()** needs to handle them correctly (since the class compiler cannot make any assumptions about the type). The `Args...` syntax handles this flexibly.

Because **%DispatchClassMethod()** attempts to resolve any unknown class method associated with the class, it has no specified return value; if successful, it returns a value whose type is determined by the method being resolved and whether the caller expects a return value.

35.3.3 %DispatchGetProperty()

This method gets the value of an unknown property. Its syntax is:

```
Method %DispatchGetProperty(Property As %String)
```

where its argument is the referenced property. Because **%DispatchGetProperty()** attempts to resolve any unknown property associated with the class, it has no specified return value; if successful, it returns a value whose type is that of the property being resolved

35.3.4 %DispatchSetProperty()

This method sets the value of an unknown property. Its syntax is:

```
Method %DispatchSetProperty(Property As %String, Value)
```

where its arguments are the name of the referenced property and the value to set for it.

35.3.5 %DispatchSetMultidimProperty()

This method sets the value of an unknown multidimensional property. Its syntax is:

```
Method %DispatchSetMultidimProperty(Property As %String, Value, Subs...)
```

where its first two arguments are the name of the referenced property and the value to set for it. The third argument, *Subs*, is an array that contains the subscript values. *Subs* has an integer value that specifies the number of subscripts, *Subs(1)* has the value of the first subscript, *Subs(2)* has the value of the second, and so on. If no subscripts are given, then *Subs* is undefined.

Only direct multidimensional property references are supported for dynamic dispatch. **$DATA**, **$ORDER**, and **$QUERY** are not supported, nor is a **SET** command with a list of variables.

Note: Note that there is no **%DispatchGetMultidimProperty()** dispatch method. This is because a multidimensional property reference is identical to a method call. Thus, such a reference invokes **%DispatchMethod()**, which must include code to differentiate between method names and multidimensional property names.

35.4 See Also

- Defining Classes

A

Object-Specific ObjectScript Features

This page summarize the ObjectScript features that are specific to working with classes and objects.

A.1 Relative Dot Syntax (..)

The relative dot syntax (..) provides a mechanism for referencing a method or property in the current context. The context for an instance method or a property is the current instance; the context for a class method is the class in which the method is implemented. You cannot use relative dot syntax in a class method to reference properties or instance methods, because these require the instance context.

For example, suppose there is a *Bricks* property of type %Integer:

Class Member

```
Property Bricks As %Integer;
```

A **CountBricks()** method can then refer to Bricks with relative dot syntax:

Class Member

```
Method CountBricks()
{
    Write "There are ",..Bricks," bricks.",!
}
```

Similarly, a **WallCheck()** method can refer to **CountBricks()** and *Bricks*:

Class Member

```
Method WallCheck()
{
    Do ..CountBricks()
    If ..Bricks < 100 {
        Write "Your wall will be small."
    }
}
```

A.2 ##class Syntax

The *##class* syntax allows you to:

- Invoke a class method when there is no existing or open instance of a class.
- Cast a method from one class as a method from another.
- Access a class parameter

Note: *##class* is not case-sensitive.

A.2.1 Invoking a Class Method

To invoke a class method, the syntax is either of the following:

```
>Do ##class(Package.Class).Method(Args)
>Set localname = ##class(Package.Class).Method(Args)
```

It is also valid to use ##class as part of an expression, as in

ObjectScript

```
 Write ##class(Class).Method(args)*2
```

without setting a variable equal to the return value.

A frequent use of this syntax is in the creation of new instances:

```
>Set LocalInstance = ##class(Package.Class).%New()
```

A.2.2 Casting a Method

To cast a method of one class as a method of another class, the syntax is either of the following:

```
>Do ##class(Package.Class1)Class2Instance.Method(Args)
>Set localname = ##class(Package.Class1)Class2Instance.Method(Args)
```

You can cast both class methods and instance methods.

For example, suppose that two classes, MyClass.Up and MyClass.Down, both have **Go()** methods. For **MyClass.Up**, this method is as follows

Class Member

```
Method Go()
{
    Write "Go up.",!
}
```

For **MyClass.Down**, the **Go()** method is as follows:

Class Member

```
Method Go()
{
    Write "Go down.",!
}
```

You can then create an instance of MyClass.Up and use it to invoke the **MyClass.Down.Go** method:

```
>Set LocalInstance = ##class(MyClass.Up).%New()

>Do ##class(MyClass.Down)LocalInstance.Go()
Go down.
```

It is also valid to use ##class as part of an expression, as in

ObjectScript

```
 Write ##class(Class).Method(args)*2
```

without setting a variable equal to the return value.

A more generic way to refer to other methods are the $METHOD and $CLASSMETHOD functions, which are for instance and class methods, respectively; these are described in Dynamically Accessing Objects. These provide a mechanism for referring to packages, classes, and methods programmatically.

A.2.3 Accessing a Class Parameter

To access a class parameter, you can use the following expression:

```
##class(Package.Class).#PARMNAME
```

Where *Package.Class* is the name of the class and *PARMNAME* is the name of the parameter. For example:

ObjectScript

```
 w ##class(%XML.Adaptor).#XMLENABLED
```

displays whether methods generated by the XML adaptor are XML enabled, which by default is set to 1.

You can also use the $PARAMETER functions, which is described in Dynamically Accessing Objects.

A.3 $this Syntax

The *$this* variable provides a handle to the OREF of the current instance, such as for passing it to another class or for another class to refer to the properties or methods of the current instance. When an instance refers to its own properties or methods, relative dot syntax is faster and thus is preferred.

Note: *$this* is not case-sensitive; hence, *$this*, *$This*, *$THIS*, or any other variant all have the same value.

For example, suppose there is an application with an Accounting.Order class and an Accounting.Utils class. The **Accounting.Order.CalcTax()** method calls the **Accounting.Utils.GetTaxRate()** and **Accounting.Utils.GetTaxableSubtotal()** methods, passing the city and state values of the current instance to the **GetTaxRate()** method and passing the list of items ordered and relevant tax-related information to **GetTaxableSubtotal()**. **CalcTax()** then uses the values returned to calculate the sales tax for the order. Hence, its code is something like:

Class Member

```
Method CalcTax() As %Numeric
{
    Set TaxRate = ##Class(Accounting.Utils).GetTaxRate($this)
    Write "The tax rate for ",..City,", ",..State," is ",TaxRate*100,"%",!
    Set TaxableSubtotal = ##class(Accounting.Utils).GetTaxableSubTotal($this)
    Write "The taxable subtotal for this order is $",TaxableSubtotal,!
    Set Tax = TaxableSubtotal * TaxRate
    Write "The tax for this order is $",Tax,!
}
```

The first line of the method uses the *##Class* syntax (described above) to invoke the other method of the class; it passes the current object to that method using the *$this* syntax. The second line of the method uses relative dot syntax to get the values of the *City* and *State* properties. The action on the third line is similar to that on the first line.

In the Accounting.Utils class, the **GetTaxRate()** method can then use the handle to the passed-in instance to get handles to various properties — for both getting and setting their values:

Class Member

```
ClassMethod GetTaxRate(OrderBeingProcessed As Accounting.Order) As %Numeric
{
    Set LocalCity = OrderBeingProcessed.City
    Set LocalState = OrderBeingProcessed.State
    // code to determine tax rate based on location and set
    // the value of OrderBeingProcessed.TaxRate accordingly
    Quit OrderBeingProcessed.TaxRate
}
```

The **GetTaxableSubtotal()** method also uses the handle to the instance to look at its properties and set the value of its TaxableSubtotal property.

Hence, the output at the Terminal from invoking the **CalcTax()** method for *MyOrder* instance of the Accounting.Order class would be something like:

```
>Do MyOrder.CalcTax()
The tax rate for Cambridge, MA is 5%
The taxable subtotal for this order is $79.82
The tax for this order is $3.99
```

A.4 ##super Syntax

Suppose that a subclass method overrides a superclass method. From within the subclass method, you can use the *##super()* syntax to invoke the overridden superclass method.

Note: *##super* is not case-sensitive.

For example, suppose that the class MyClass.Down extends MyClass.Up and overrides the **Simple** class method. If the code for **MyClass.Up.Simple()** is:

Class Member

```
ClassMethod Simple()
{
    Write "Superclass.",!
}
```

and the code for **MyClass.Down.Simple()** is:

Class Member

```
ClassMethod Simple()
{
    Write "Subclass.",!
    Do ##super()
}
```

then the output for subclass method, **MyClass.Down.Simple()**, is:

```
>Do ##Class(MyClass.Down).Simple()
Subclass.
Superclass.
>
```

A more generic way to refer to other methods are the $METHOD and $CLASSMETHOD functions, which are for instance and class methods, respectively; these are described in Dynamically Accessing Objects. These provide a mechanism for referring to packages, classes, and methods programmatically.

A.4.1 Calls That ##super Affects

##super only affects the current method call. If that method makes any other calls, those calls are relative to the current object or class, *not* the superclass. For example, suppose that MyClass.Up has **MyName()** and **CallMyName()** methods:

```
Class MyClass.Up Extends %Persistent
{

ClassMethod CallMyName()
{
    Do ..MyName()
}

ClassMethod MyName()
{
    Write "Called from MyClass.Up",!
}

}
```

and that MyClass.Down overrides those methods as follows:

```
Class MyClass.Down Extends MyClass.Up
{

ClassMethod CallMyName()
{
    Do ##super()
}

ClassMethod MyName()
{
    Write "Called from MyClass.Down",!
}

}
```

then invoking the **CallMyName()** methods have the following results:

```
USER>d ##class(MyClass.Up).CallMyName()
Called from MyClass.Up

USER>d ##class(MyClass.Down).CallMyName()
Called from MyClass.Down
```

MyClass.Down.CallMyName() has different output from **MyClass.Up.CallMyName()** because its **CallMyName()** method includes *##super* and so calls the **MyClass.Up.CallMyName()** method, which then calls the uncast **MyClass.Down.MyName()** method.

A.4.2 ##super and Method Arguments

##super also works with methods that accept arguments. If the subclass method does not specify a default value for an argument, make sure that the method passes the argument by reference to the superclass.

For example, suppose the code for the method in the superclass (**MyClass.Up.SelfAdd()**) is:

Class Member

```
ClassMethod SelfAdd(Arg As %Integer)
{
    Write Arg,!
    Write Arg + Arg
}
```

then its output is:

```
>Do ##Class(MyClass.Up).SelfAdd(2)
2
4
>
```

The method in the subclass (**MyClass.Down.SelfAdd()**) uses *##super* and passes the argument by reference:

Class Member

```
ClassMethod SelfAdd(Arg1 As %Integer)
{
    Do ##super(.Arg1)
    Write !
    Write Arg1 + Arg1 + Arg1
}
```

then its output is:

```
>Do ##Class(MyClass.Down).SelfAdd(2)
2
4
6
>
```

In **MyClass.Down.SelfAdd()**, notice the period before the argument name. If we omitted this and we invoked the method without providing an argument, we would receive an `<UNDEFINED>` error.

A.5 $CLASSNAME and Other Dynamic Access Functions

InterSystems IRIS® data platform supplies several functions that support generalized processing of objects. They do this by allowing a reference to a class and one of its methods or properties to be computed at runtime. (This is known as reflection in Java.) These functions are:

- $CLASSNAME — Returns the name of a class.
- $CLASSMETHOD — Supports calls to a class method.
- $METHOD — Supports calls to an instance method.
- $PARAMETER — Returns the value of a class parameter of the specified class.
- $PROPERTY — Supports references to a particular property of an instance.

The function names are shown here in all uppercase letters, but they are, in fact, not case-sensitive.

A.5.1 $CLASSNAME

This function returns the name of a class. The signature is:

```
$CLASSNAME(Instance)
```

where *Instance* is an OREF.

For more information, see $CLASSNAME.

A.5.2 $CLASSMETHOD

This function executes a named class method in the designated class. The signature is:

```
$CLASSMETHOD (Classname, Methodname, Arg1, Arg2, Arg3, ... )
```

where:

Classname

An existing class.

Methodname

A method of the class specified by the first argument.

***Arg1*, *Arg2*, *Arg3*, ...**

A series of expressions to be substituted for the arguments to the designated method.

For more information, see $CLASSMETHOD.

A.5.3 $METHOD

This function executes a named instance method for a specified instance of a designated class. The signature is:

```
$METHOD (Instance, Methodname, Arg1, Arg2, Arg3, ... )
```

where:

Instance

An OREF of instance of a class.

Methodname

A method of the class specified by the instance in the first argument.

***Arg1*, *Arg2*, *Arg3*, ...**

A series of expressions to be substituted for the arguments to the designated method.

For more information, see $METHOD.

A.5.4 $PARAMETER

This function returns the value of a class parameter of the designated class. The signature is:

```
$PARAMETER(Instance,Parameter)
```

where:

Instance

An OREF of instance of a class.

Instance

Either the fully qualified name of a class or an OREF of an instance of the class.

Parameter

A parameter of the given class.

For more information, see $PARAMETER.

A.5.5 $PROPERTY

This function gets or sets the value of a property in an instance of the designated class. If the property is multidimensional, the following arguments after the property name are used as indexes in accessing the value of the property. The signature is:

```
$PROPERTY (Instance, PropertyName, Index1, Index2, Index3... )
```

where:

Instance

An OREF of instance of a class.

PropertyName

A property of the class specified by the instance in the first argument.

***Index1*, *Index2*, *Index3*, ...**

For multidimensional properties, indexes into the array represented by the property.

For more information, see $PROPERTY.

A.6 i%<PropertyName> Syntax

This section provides some additional information on instance variables. You do not need to refer to these variables unless you override an *accessor method* for a property; see Using and Overriding Property Methods.

When you create an instance of any class, the system creates an instance variable for each non-calculated property of that class. The instance variable holds the value of the property. For the property *PropName*, the instance variable is named *i%PropName*, and this variable name is case-sensitive. These variables are available within any instance method of the class.

For example, if a class has the properties Name and DOB, then the instance variables *i%Name* and *i%DOB* are available within any instance method of the class.

Internally, InterSystems IRIS also uses additional instance variables with names such as *r%PropName* and *m%PropName*, but these are not supported for direct use.

Instance variables have process-private, in-memory storage allocated for them. Note that these variables are not held in the local variable symbol table and are not affected by the **Kill** command.

A.7 ..#<Parameter> Syntax

The ..#<Parameter> syntax allows for references to class parameters from within methods of the same class.

For example, if a class definition include the following parameter and method:

Class Member

```
Parameter MyParam = 22;
```

and the following method:

Class Member

```
ClassMethod WriteMyParam()
{
    Write ..#MyParam
}
```

Then the **WriteMyParam()** method invokes the **Write** command with the value of the *MyParam* parameter as its argument.

B

Using the Object Synchronization Feature

This topic describes the object synchronization feature, which you can use to synchronize specific tables in databases that are on occasionally connected systems.

B.1 Introduction to Object Synchronization

Object synchronization is a set of tools available with InterSystems IRIS® data platform objects that allows application developers to set up a mechanism to synchronize databases on occasionally connected systems. By this process, each database updates its objects. Object synchronization offers complementary functionality to InterSystems IRIS system tools that provide high availability. Object synchronization is not designed to provide support for real-time updates; rather, it is most useful for a system that needs updates at discrete intervals.

For example, a typical object synchronization application would be in an environment where there is a master copy of a database on a central server and secondary copies on client machines. Consider the case of a sales database, where each sales representative has a copy of the database on a laptop computer. When Mary, a sales representative, is off site, she makes updates to her copy of the database. When she connects her machine to the network, the central and remote copies of the database are synchronized. This can occur hourly, daily, or at any interval.

Object synchronization between two databases involves updating each of them with data from the other. However, InterSystems IRIS does not support bidirectional synchronization as such. Rather, updates from one database are posted to the other; then updates are posted in the opposite direction. For a typical application, if there is a main database and one or more local databases (as in the previous sales database example), it is recommended that updates are from the local to the main database first, and then from the main database to the local one.

For object synchronization, the idea of client and server is by convention only. For any two databases, you can perform bidirectional updates; if there are more than two databases, you can choose what scheme you use to update all of them (such as local databases synchronizing with a main database independently).

This section addresses the following topics:

- The GUID
- How updates work
- The SyncSet and SyncTime objects

B.1.1 The GUID

To ensure that updates work properly, each object in a database should be uniquely distinguishable. To provide this functionality, InterSystems IRIS gives each individual object instance a *GUID* — a globally unique ID. The GUID makes each object universally unique.

The GUID is optionally created, based on the value of the *GUIDENABLED* parameter. If *GUIDENABLED* has a value of 1, then a GUID is assigned to each new object instance.

Consider the following example. Two databases are synchronized and each has the same set of objects in it. After synchronization, each database has a new object added to it. If the two objects share a common GUID, object synchronization considers them the same object in two different states; if each has its own GUID, object synchronization considers them to be different objects.

B.1.2 How Updates Work

Each update from one database to another is sent as a set of transactions. This ensures that all interdependent objects are updated together. The content of each transaction depends on the contents of the journal for the source database. The update can include one or more transactions, up to all transactions that have occurred since the last synchronization.

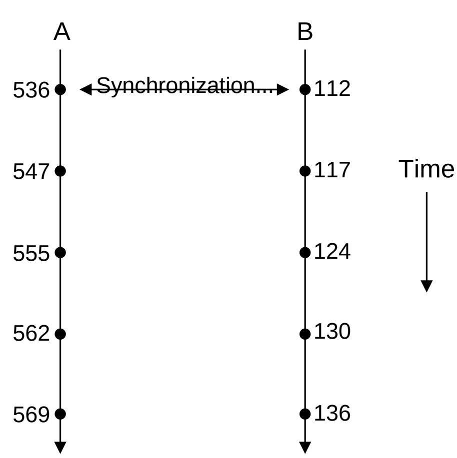
Resolution of the following conditions is the responsibility of the application:

- If two instances that share a unique key have different GUIDs. This requires determining if the two records describe a single object or two unique objects.
- If two changes require reconciliation. This requires determining if the two changes were to a common property or to non-intersecting sets of properties.

B.1.3 The SyncSet and SyncTime Objects

When two databases are to be synchronized, each has transactions in it that the other lacks. This is illustrated in the following diagram:

Figure II–1: Two Unsynchronized Databases



Here, database A and database B have been synchronized at transaction 536 for database A and transaction 112 for database B. The subsequent transactions for each database need to be updated from each to the other. To do this, InterSystems IRIS uses what is called a SyncSet object. This object contains a list of transactions that are used to update a database. For example, when synchronizing database B with database A, the default contents of the SyncSet object are transactions 547, 555, 562, and 569. Analogously, when synchronizing database A with database B, the default contents of the SyncSet object are transactions 117, 124, 130, and 136. (The transactions do not use a continuous set of numbers, because each transaction encapsulates multiple inserts, updates, and deletes — which themselves use the intermediate numbers.)

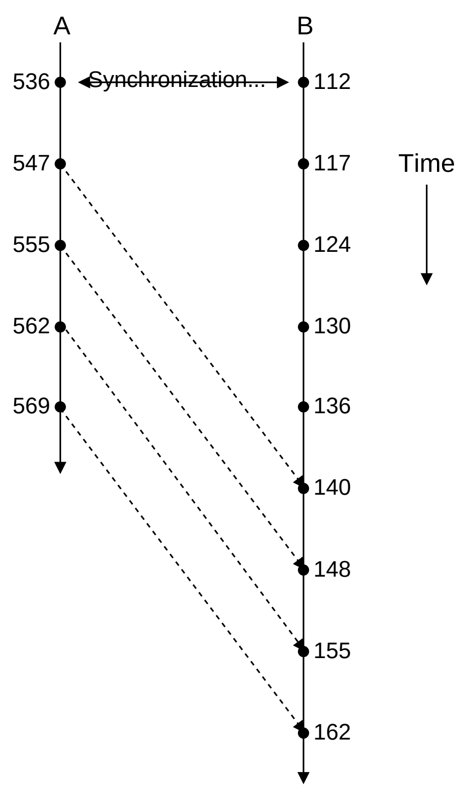
Each database holds a record of its synchronization history with the other. This record is called a SyncTime table. For database, its contents are of the form:

```
Database     Namespace     Last Transaction Sent     Last Transaction Received
------------------------------------------------------------------------------
B            User          536                       112
```

Note: The numbers associated with each transaction do not provide any form of a time stamp. Rather, they indicate the sequence of filing for transactions within an individual database.

Once database B has been synchronized with database A, the two databases might appear as follows:

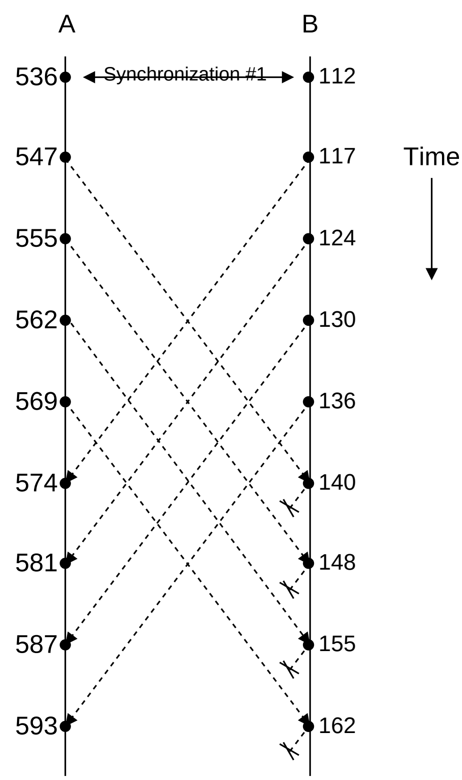
Figure II–2: Two Databases, Where One Has Been Synchronized with the Other



Because the transactions are being added to database B, they result in new transaction numbers in that database.

Analogously, the synchronization of database B with database A results in 117, 124, 130, and 136 being added to database A (and receiving new transaction numbers there):

Figure II–3: Two Synchronized Databases



Note that the transactions from database B that have come from database A (140 through 162) are *not* updated back to database A. This is because the update from B to A uses a special feature that is part of the synchronization functionality. It works as follows:

1. Each transaction in a database is labeled with what can be called a database of origin. In this example, transaction 140 in database B would be marked as originating in database A, while its transaction 136 would be marked as originating in itself (database B).
2. The **SyncSet.AddTransactions()** method, which bundles a set of transactions for synchronization, allows you to exclude transactions that originate in a particular database. Hence, when updating from B to A, **AddTransactions()** excludes all transactions that originate in database A — because those have already been added to the transaction list for database B.

This functionality prevents creating infinite loops in which two databases continually update each other with the same set of transactions.

B.2 Modifying the Classes to Support Synchronization

Object synchronization requires that the sites have data with matching sets of GUIDs. If you are starting with an already existing database that does not yet have GUIDs assigned for its records, you need to assign a GUID to each instance (record) in the database, and then make sure there are matching copies of the database on each site. In detail, the process is:

1. For each class being synchronized, set the value of the *OBJJOURNAL* parameter to 1.

Class Member

```
Parameter OBJJOURNAL = 1;
```

This activates the logging of filing operations (that is, insert, update, or delete) within each transaction; this information is stored in the *^OBJ.JournalT* global. An *OBJJOURNAL* value of 1 specifies that the property values that are changed in filing operations are stored in the system journal file; during synchronization, data that needs to be synchronized is retrieved from that file.

Note: *OBJJOURNAL* can also have a value of 2, though the possible use of this value is restricted to special cases. It is *never* for classes using the default storage mechanism (%Storage.Persistent). A value of 2 specifies that property values that are changed in filing operations are stored in the *^OBJ.Journal* global; during synchronization, data that needs to be synchronized is retrieved from that global. Also, storing information in the global increases the size of the database very quickly.

2. Optionally also set the value of the *JOURNALSTREAM* parameter to 1.

   Class Member

   ```
   Parameter JOURNALSTREAM = 1;
   ```

   By default, object synchronization does not support synchronization of file streams. The *JOURNALSTREAM* parameter controls whether or not streams are journaled when *OBJJOURNAL* is true:

   - If *JOURNALSTREAM* is false and *OBJJOURNAL* is true, then objects are journaled but the streams are not.
   - If *JOURNALSTREAM* is true and *OBJJOURNAL* is true, then streams are journaled. Object synchronization tools will process journaled streams when the referencing object is processed.

3. For each class being synchronized, set the value of its *GUIDENABLED* parameter to 1; this tells InterSystems IRIS to allow the class to be stored with GUIDs.

   Class Member

   ```
   Parameter GUIDENABLED = 1;
   ```

   Note that if this value is not set, the synchronization does not work properly. Also, you must set *GUIDENABLED* for serial classes, but not for embedded objects.

4. Recompile the class.
5. For each class being synchronized, give each object instance its own GUID by running the **AssignGUID()** method:

   ObjectScript

   ```
    Set Status = ##Class(%Library.GUID).AssignGUID(className,displayOutput)
   ```

   where:

   - *className* is the name of class whose instances are receiving GUIDs, such as `"Sample.Person"`.
   - *displayOutput* is an integer where zero specifies that no output is displayed and a nonzero value specifies that output is displayed.

   The method returns a %Status value, which you should check.

6. Put a copy of the database on each site.

B.3 Performing the Synchronization

This section describes how to perform the synchronization. The database providing the updates is known as the *source database*; and the database receiving the updates is the *target database*. To perform the actual synchronization, the process is:

1. Each time you wish to synchronize the two databases, go to the instance with the source database. On the source database, create a new SyncSet using the **%New()** method of the %SYNC.SyncSet class:

   ObjectScript

   ```
   Set SrcSyncSet = ##class(%SYNC.SyncSet).%New("unique_value")
   ```

   The integer argument to **%New()**, *unique_value*, should be an easily identified, unique value. This ensures that each addition to the transaction log on each site can be differentiated from the others.

2. Call the **AddTransactions()** method of the SyncSet instance:

   ObjectScript

   ```
   Do SrcSyncSet.AddTransactions(FirstTransaction,LastTransaction,ExcludedDB)
   ```

   Where:

   - *FirstTransaction* is the first transaction number to synchronize.
   - *LastTransaction* is the last transaction number to synchronize.
   - *ExcludedDB* specifies a namespace within a database whose transactions are not included in the SyncSet.

   This method collects the synchronization data and puts it in a global, ready for export.

   Or, to synchronize all transactions since the last synchronization, omit the first and second arguments:

   ObjectScript

   ```
   Do SrcSyncSet.AddTransactions(,,ExcludedDB)
   ```

   This gets all transactions, beginning with the first unsynchronized transaction to the most recent transaction. The method uses information in the SyncTime table to determine the values.

   ExcludedDB is a $LIST created as follows:

   ObjectScript

   ```
   Set ExcludedDB = $ListBuild(GUID,namespace)
   ```

   Where:

   - *GUID* is the system GUID of the target system. This value is available through the %SYS.System.InstanceGUID class method; to invoke this method, use the `##class(%SYS.System).InstanceGUID()` syntax.
   - *namespace* is the namespace on the target system.

3. Call the **ErrCount()** method to determine how many errors were encountered. If there have been errors, the SyncSet.Errors query provides more detailed information.

4. Export the data to a local file using the **ExportFile()** method:

ObjectScript

```
Do SrcSyncSet.ExportFile(file,displaymode,bUpdate)
```

Where:

- *file* is the file to which the transactions are being exported; it is a name with a relative or absolute path.
- *displaymode* specifies whether or not the method writes output to the current device. Specify d for output or -d for no output.
- *bUpdate* is a boolean value that specifies whether or not the SyncTime table is updated (where the default is 1, meaning True). It may be helpful to explicitly set this to 0 at this point, and then set it to 1 after the source receives assurance that the target has indeed received the data and performed the synchronization.

5. Move the exported file from the source machine to the target machine.
6. Create a SyncSet object on the target machine using the **SyncSet.%New()** method. Use the same value for the argument of **%New()** as on the source machine — this is what identifies the source of the synchronized transactions.
7. Read the SyncSet object into the InterSystems IRIS instance on the target machine using the **Import()** method:

ObjectScript

```
Set Status = TargetSyncSet.Import(file,lastSync,maxTS,displaymode,errorlog,diag)
```

Where:

- *file* is the file containing the data for import.
- *lastSync* is the last synchronized transaction number (default from synctime table).
- *maxTS* is the last transaction number in the SyncSet object.
- *displaymode* specifies whether or not the method writes output to the current device. Specify d for output or -d for no output.
- *errorlog* provides a repository for any error information (and is called by reference to provide information for the application).
- *diag* provides more detailed diagnostic information about what is happening when importing

This method puts data into the target database. It behaves as follows:

a. If the method detects that the object has been modified on both the source and target databases since the last synchronization, it invokes the **%ResolveConcurrencyConflict()** callback method; like other callback methods, the content of **%ResolveConcurrencyConflict()** is user-supplied. (Note that this can occur if either the two changes both modified a common property or the two changes each modified non-intersecting sets of properties.) If the **%ResolveConcurrencyConflict()** method is not implemented, then the conflict remains unresolved.

b. If, after the **Import()** method executes, there are unsuccessfully resolved conflicts, these remain in the SyncSet object as unresolved items. Be sure to take the appropriate action regarding the remaining conflicts; this may involve resolution, leaving the items in an unresolved state, and so on.

Important: The **Import()** method returns a status value but that status value simply indicates that the method completed without encountering an error that prevented the SyncSet from being processed. It does not indicate that every object in the SyncSet was processed successfully without encountering any errors. For information on synchronization error reporting, see **Import()** in the class reference for %SYNC.SyncSet.

8. Once the first database updates the second database, perform the same process in the other direction so that the second database can update the first one.

B.4 Translating Between GUIDs and OIDs

To determine the OID of an object from its GUID or vice versa, there are two methods available:

- **%GUIDFind()** is a class method of the %GUID class that takes a GUID of an object instance and returns the OID associated with that instance.
- **%GUID()** is a class method of the %Persistent class that takes an OID of an object instance and returns the GUID associated with that instance; the method can only be run if the *GUIDENABLED* parameter is TRUE for the corresponding class. This method dispatches polymorphically and determines the most specific type class if the OID does not contain that information. If the instance has no GUID, the method returns an empty string.

B.5 Manually Updating a SyncTime Table

To perform a manual update on the SyncTime table for a database, invoke the **SetlTrn()** method, which sets the last transaction number:

ObjectScript

```
Set Status=##class(%SYNC.SyncTime).SetlTrn(syncSYSID, syncNSID, ltrn)
```

Where:

- *syncSYSID* is the system GUID of the target system. This value is available through the %SYS.System.InstanceGUID class method; to invoke this method, use the `##class(%SYS.System).InstanceGUID()` syntax.
- *syncNSID* is the namespace on the target system, which is held in the **$Namespace** variable.
- *ltrn* is the highest transaction number known to have been imported. You can get this value by invoking the **GetLastTransaction()** method of the SyncSet.

The **SetlTrn()** method sets the highest transaction number synchronized on the target system instead of the default behavior (which is to set the highest transaction number exported from the source system). Either approach is fine and is a choice available during application development.